

Gegevensabstractie en -structuren

Thema 1 Gegevensabstractie: muren Abstract Data Types (ADTs)

1

Abstract Data Types (ADT)

- Definitie

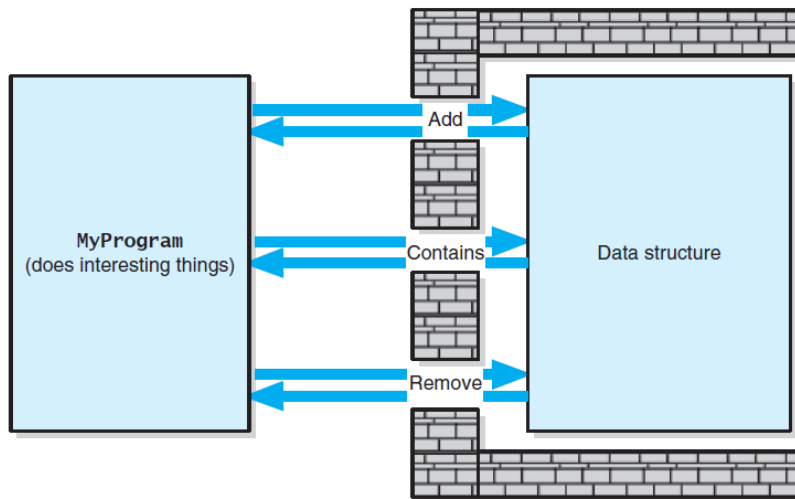
Een ADT bestaat uit

- een verzameling gegevens
- een verzameling bewerkingen op die gegevens
 - gegevens toevoegen
 - gegevens verwijderen
 - vragen stellen over gegevens
- Specificaties van een ADT = het **abstracte niveau**, het plan, het contract
 - beschrijven nauwkeurig **wat** de bewerkingen doen, **niet hoe** ze te implementeren
- Implementatie van een ADT = het **concrete niveau**, de uitvoering van het contract
 - omvat het kiezen van een **gegevensstructuur**, d.i. een constructie, gedefinieerd in een programmeertaal, om gegevens op te slaan

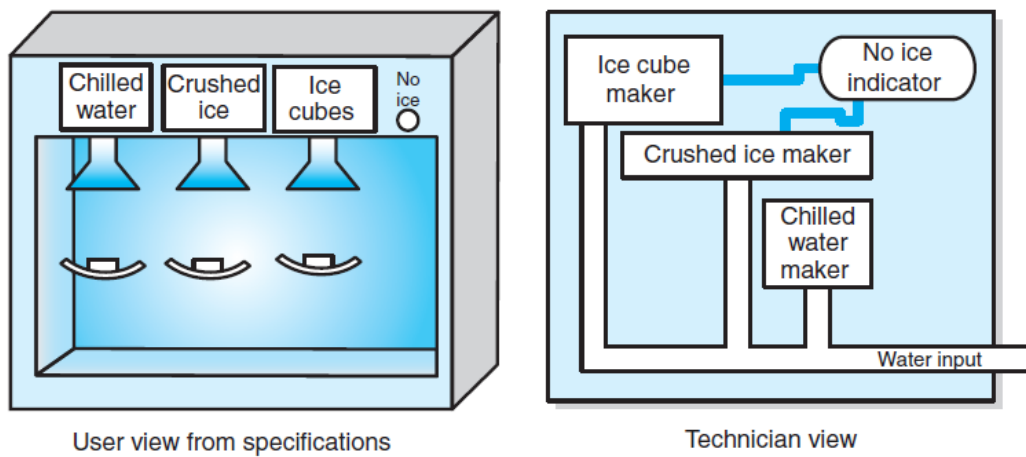
2

ADT versus gegevensstructuur

- **ADT** = de specificatie van gegevens en bewerkingen op die gegevens
- **gegevensstructuur** = de implementatie van een ADT in een programmeertaal
- verstopt gegevensstructuren achter een '**muur**' van ADT bewerkingen (zie ook Thema 11)



3



4

Een ADT ontwerpen

- het ontwerp van een ADT ontstaat tijdens het problem-solving proces
- 2 sleutelvragen:
 - welke gegevens heeft het probleem nodig?
 - welke bewerkingen zijn er nodig?

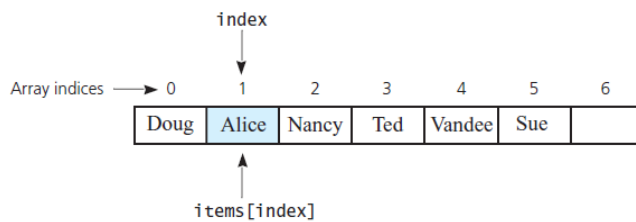
Een ADT implementeren

- hoe de gegevens van een ADT opslaan?
 - kies een gegevensstructuur
- hoe de bewerkingen van een ADT uitvoeren?
 - kies algoritmen
- de keuzes hangen af van
 - de ADT bewerkingen
 - de context waarin de bewerkingen gebruikt worden

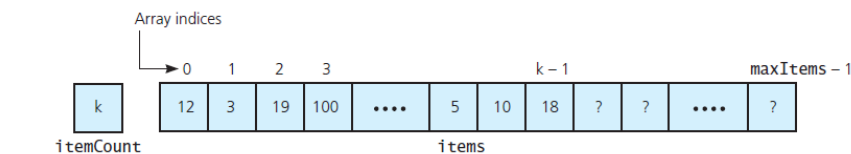
Gegevensabstractie en -structuren

Thema 2 Arrays en zoekalgoritmen Recurisie: spiegels

Een array of vakjesblok met vaste grootte



- de opeenvolgende locaties (vakjes) in de array (vakjesblok) krijgen een nummer, de **index**, van 0 tot de grootte van de array-1,
- notatie: stel we hebben een array met naam *items*, dan staat **items[k]** voor de inhoud van het vakje met index *k* in de array *items*
- voorbeeld:



$\text{items}[0] = 12, \text{items}[1] = 3, \dots, \text{items}[k-1] = 18$

7

Sequentieel zoeken in een array of vakjesblok

Algoritme

- begin het zoeken naar een element bij het eerste vakje
- loop de achtereenvolgende vakjes af tot het element gevonden is

Pseudocode geeft goed leesbaar de stappen in een algoritme weer, in een formulering tussen een tekst in natuurlijke taal en een computerprogramma

Pseudocode 1

kijk of het element in het eerste vakje $\text{items}[0]$ zit

zo niet: kijk of het element in het tweede vakje $\text{items}[1]$ zit

...

zo niet: kijk of het element in het laatste vakje $\text{items}[\text{maxItems}-1]$ zit

Pseudocode 2

gevonden = false

index = 0

while (index < maxItems en niet gevonden)

if (het element in het vakje $\text{items}[\text{index}]$ zit)

 gevonden = true

else

 index = index + 1

8

Binair zoeken

Voorbeeld: een woord opzoeken in een woordenboek

Pseudocode Search

```
if (the dictionary contains only one page)
    Scan the page for word
else
{   Open the dictionary to a point near the middle
    Determine which half of the dictionary contains the word

    if (the word is in the first half of the dictionary)
        Search the first half of the dictionary for the word
    else
        Search the second half of the dictionary for the word
}
```

Lukt binair zoeken altijd of is er een voorwaarde?

9

Recursie, spiegels

Recursie

- een uitzonderlijk krachtige techniek voor problem-solving
- lost een probleem op door kleinere problemen van hetzelfde type op te lossen (cfr. spiegels):
 - splitst een probleem op in kleinere identieke problemen,
 - splitst deze nieuwe problemen opnieuw op in nog kleinere identieke problemen
 - ...
 - tot uiteindelijk de nieuwe problemen zo klein zijn dat de oplossing ervan evident is

10

Gegevensabstractie en -structuren

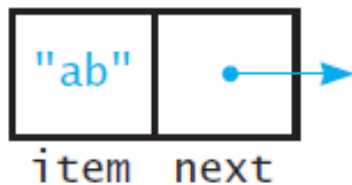
Thema 3 Gelinkte kettingen

11

Een gelinkte ketting (of kort ketting)

node of knoop =

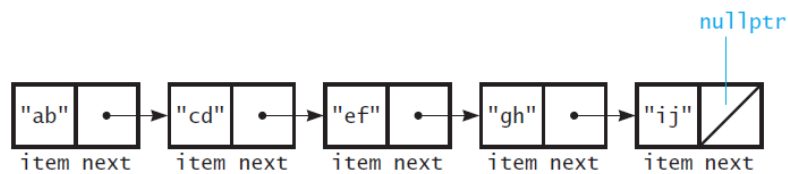
- component die kan gelinkt worden aan andere componenten
- bevat een item of gegevenselement en een wijzertje



~ een wagonnetje van een speelgoedtreintje

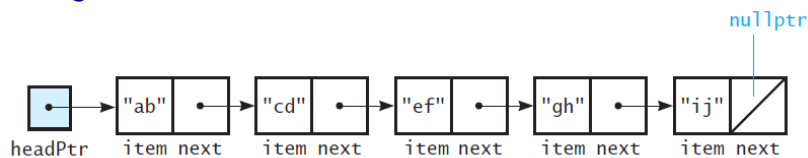
12

- knopen kunnen aan elkaar gelinkt (of met elkaar verbonden) worden tot een gelinkte ketting (linked chain)



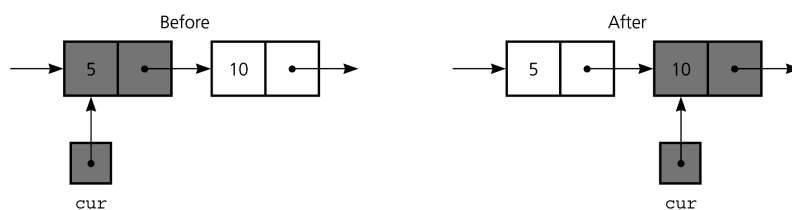
- een nullpointer geeft het einde van de gelinkte ketting weer

- een extra wijzertje (head pointer) geeft toegang tot de eerste knoop van een gelinkte ketting



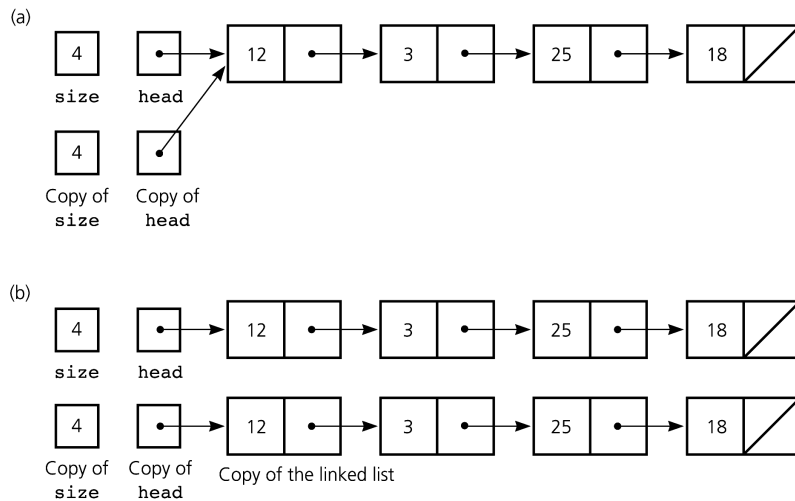
13

De inhoud van een gelinkte ketting doorlopen of zoeken naar een element



14

Oppervlakkige (a) vs. diepe kopie (b) van een gelinkte ketting



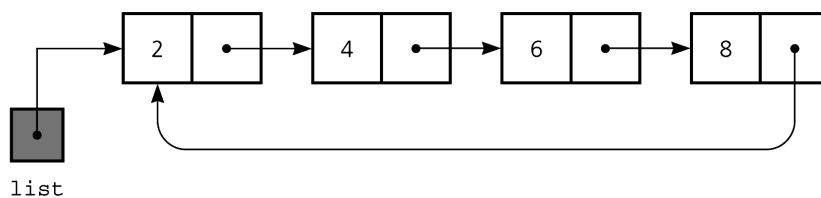
15

Variaties: circulaire gelinkte ketting

De laatste knoop wijst naar de eerste.

Elke knoop heeft een voorganger.

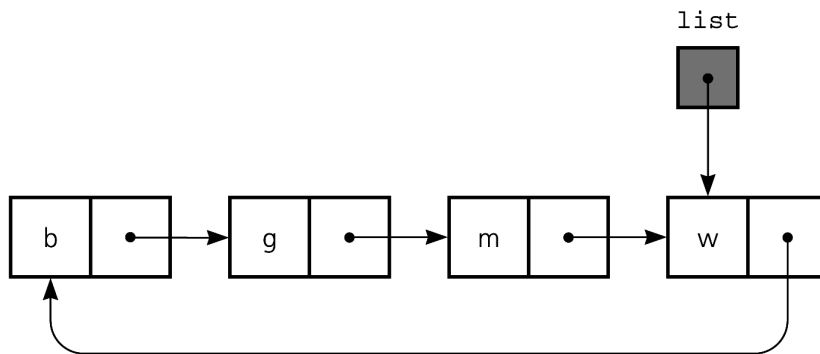
Geen enkele knoop bevat de nullpointer.



16

Variaties: circulaire gelinkte ketting

Laat de wijzer naar de laatste knoop wijzen om zowel eerste als laatste knoop snel te bereiken.

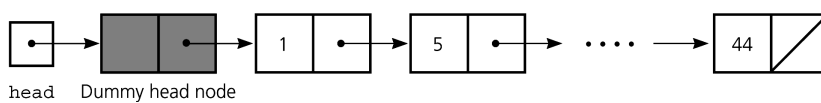


17

Variaties: dummy head knoop

dummy head knoop

- blijft aanwezig, ook als de gelinkte ketting leeg is
- elimineert speciaal geval: eerste element toevoegen en verwijderen



18

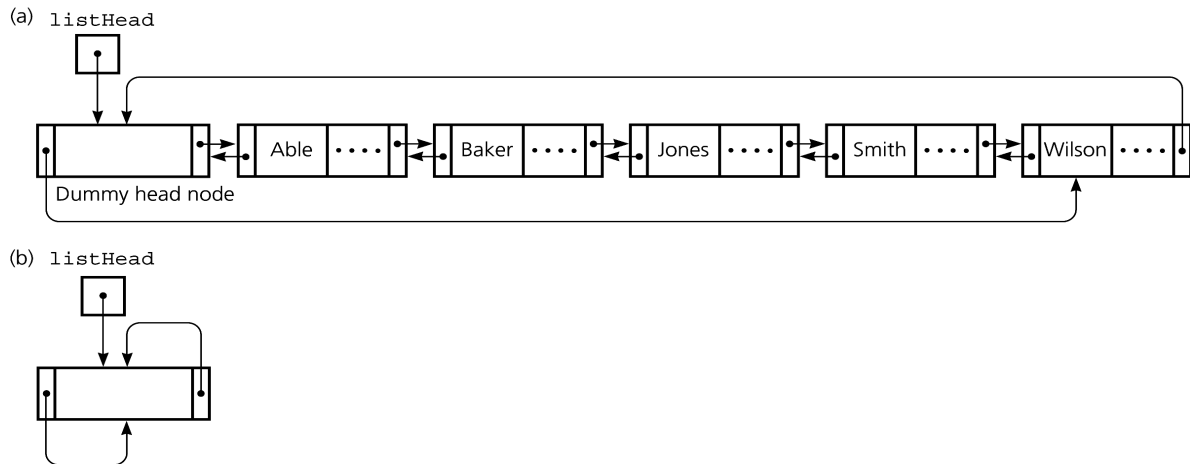
Variaties: dubbel gelinkte kettingen

Elke knoop wijst naar z'n voorganger EN z'n opvolger:

- preceede wijzer en next wijzer

(a) Een circulaire dubbel gelinkte ketting met dummy head knoop

(b) Een lege ketting met dummy head knoop



19

Gegevensabstractie en -structuren

Thema 4 Het ADT lijst

20

Het ADT lijst

- Alle elementen, behalve het eerste en het laatste, hebben een unieke voorganger en een unieke opvolger
- Het eerste element (head/front) heeft geen voorganger
- Het laatste element (tail/end) heeft geen opvolger
- We verwijzen naar de elementen via hun positie in de lijst
- **Specificaties** van de ADT bewerkingen van het ADT lijst
 - definieer een **bewerkingencontract** (operation contract)
 - vermeld **wat** de bewerkingen doen
 - vermeld **niet hoe** de lijst zal worden bewaard of **hoe** de bewerkingen zullen worden uitgevoerd
- ADT bewerkingen kunnen gebruikt worden in een toepassing op basis van de specificaties, zonder kennis van de implementatiedetails!

21

De ADT lijst gegevens

- een eindig aantal objecten
 - niet noodzakelijk verschillend
 - van hetzelfde type
 - gesorteerd volgens de door de gebruiker bepaalde posities

De ADT lijst bewerkingen

1. Creëer een lege lijst (create)
2. Wis een lijst (destroy)
3. Bepaal of een lijst leeg is
4. Bepaal het aantal elementen in een lijst
5. Voeg een element toe op een bepaalde positie in een lijst: **insert**
6. Verwijder het element op een bepaalde positie uit een lijst: **delete**
7. Vraag het element op een bepaalde positie in een lijst op: **retrieve**

22

Specificatie van de ADT bewerkingen van het ADT lijst

= bewerkingencontract of operation contract

// ListItemType is het type van de elementen in de lijst

+createList()

// Creëert een lege lijst.

+destroyList()

// Wist een lijst.

+isEmpty(): boolean {query}

// Bepaalt of een lijst leeg is.

+getLength(): integer {query}

// Geeft het aantal elementen in de lijst.

23

+insert(in positie:integer, in newItem:ListItemType, out success:boolean)

// Voegt het element 'newItem' toe op 'positie' in een lijst, indien

// $1 \leq \text{positie} \leq \text{getLength}() + 1$.

// Als $\text{positie} \leq \text{getLength}()$, dan worden de elementen als volgt hernummerd:

// het element op 'positie' wordt het element op 'positie+1', het element op 'positie+1'

// wordt het element op 'positie+2', enz.. = **shift** rechts

// 'Success' duidt aan of het toevoegen gelukt is.

+delete(in positie:integer, out success:boolean)

// Verwijdert het element op 'positie' uit een lijst, indien

// $1 \leq \text{positie} \leq \text{getLength}()$.

// Als $\text{positie} < \text{getLength}()$, dan worden de elementen als volgt hernummerd:

// het element op 'positie+1' wordt het element op 'positie', het element op 'positie+2'

// wordt het element op 'positie+1', enz.. = **shift** links

// 'Success' duidt aan of het verwijderen gelukt is.

**+retrieve(in positie:integer, out dataItem:ListItemType, out success:boolean)
{query}**

// Plaatst het element op 'positie' van een lijst in 'dataItem', indien

// $1 \leq \text{positie} \leq \text{getLength}()$. De lijst blijft onveranderd bij deze bewerking.

// 'Success' duidt aan of het opvragen gelukt is.

Zie Thema 10 voor pré- en postcondities.

24

Gebruik van de ADT lijst bewerkingen

Deze specificatie volstaat om het ADT te kunnen gebruiken – we hoeven de implementatie niet te kennen:

Voorbeeld 1.

```
boodschappenlijst.createList()      // creëert een lege lijst 'boodschappenlijst'
boodschappenlijst.insert(1, appelen, success)
boodschappenlijst.insert(2, rozijnen, success)
boodschappenlijst.insert(3, volkorenbloem, success)
boodschappenlijst.insert(4, ahornsiroop, success)
boodschappenlijst.insert(5, tofu, success)
boodschappenlijst.insert(6, zonnebloemolie, success)
boodschappenlijst.delete(4, success)
boodschappenlijst.insert(4, rijststroop, success)
boodschappenlijst.insert(3, kaneel, success)
```

⇒ boodschappenlijst bevat appelen, rozijnen, kaneel, volkorenbloem, rijststroop, tofu, zonnebloemolie (in deze volgorde)

25

Gegevensabstractie en -structuren

Thema 5 ADT Lijst implementaties

26

Verband ADT lijst, array en gelinkte ketting

Als we het ADT lijst willen **implementeren** kunnen we gebruik maken van

- een array of
- een gelinkte ketting

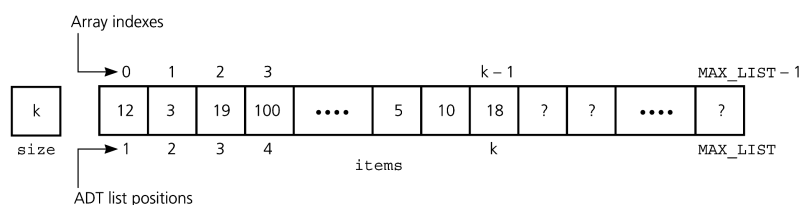
Beiden zijn geschikt om het ADT lijst voor te stellen:

- Alle elementen, behalve het eerste en het laatste, hebben een unieke voorganger en een unieke opvolger
- Het eerste element (head/front) heeft geen voorganger
- Het laatste element (tail/end) heeft geen opvolger
- We verwijzen naar de elementen via hun positie in de lijst (ook van de gelinkte ketting kunnen we aflezen wat het eerste element is, het tweede, het derde, enz.)

27

Een array implementatie van het ADT Lijst

- een array en een lijst verwijzen beiden naar hun elementen via een getal
 - vb. het element op positie 4 van de **lijst**
 - vb. het element met index 3 van de **array**.
- plaats de elementen van een lijst in een array *items*:
 - het k^{de} element van de lijst zit in `items[k-1]`
- `size` = het aantal elementen in de lijst



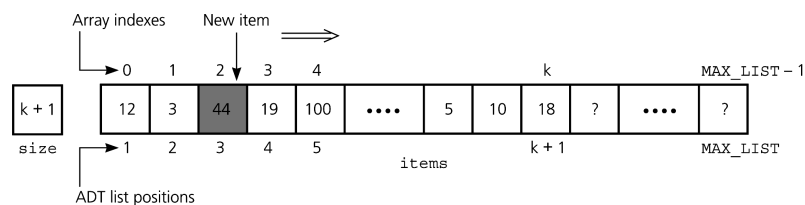
28

De ADT lijst bewerkingen in een array implementatie

- ADT bewerking isEmpty: return (size == 0)
- ADT bewerking getLength: return size
- ADT bewerking retrieve: dataItem = items[index]

ADT bewerking insert

- insert het element 44 op positie 3 van de lijst:

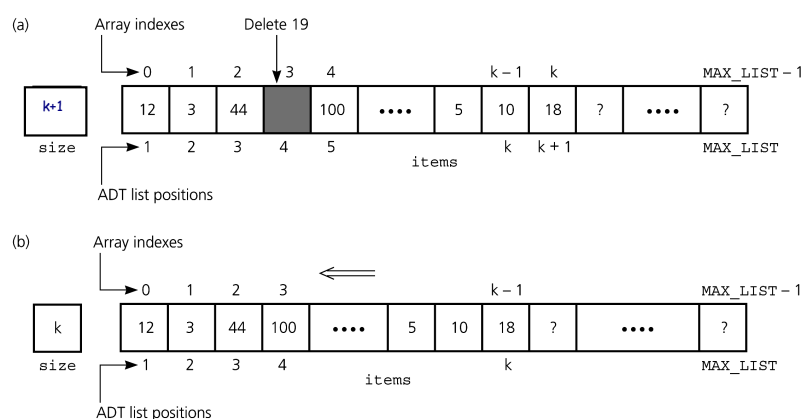


- gegevens naar rechts shiften om plaats te maken: $\text{items}[k+1] = \text{items}[k]$

29

ADT bewerking delete (of remove)

- delete het element op positie 4 van de lijst:



- gegevens naar links shiften om lege plaats te vullen: $\text{items}[k] = \text{items}[k+1]$

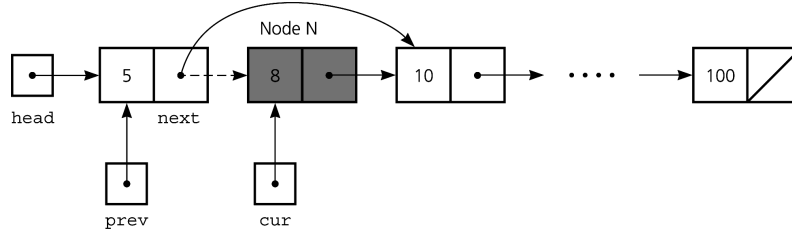
30

Een gelinkte ketting implementatie van het ADT Lijst

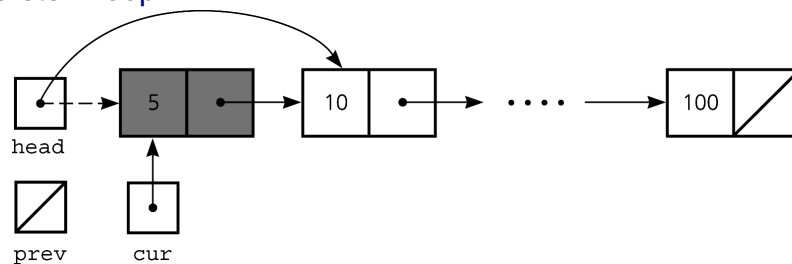
ADT delete

een element verwijderen uit de lijst = een knoop verwijderen uit de gelinkte ketting

(a) Een interne knoop:



(b) De eerste knoop:

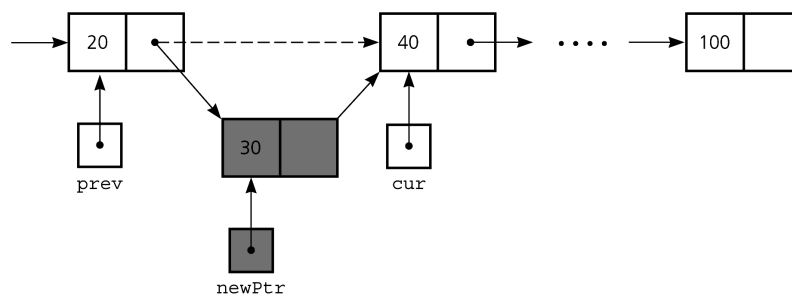


31

ADT insert

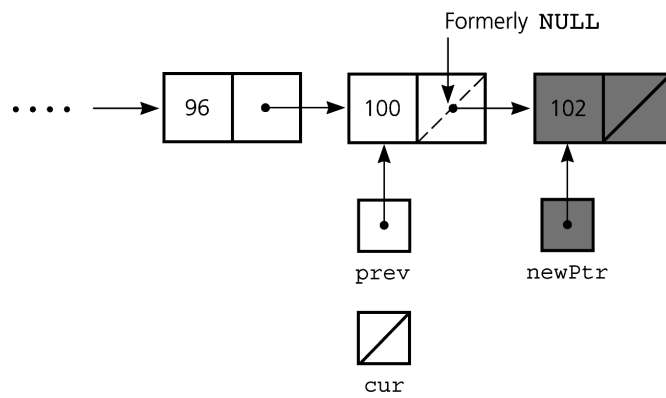
een element toevoegen op een bepaalde positie in de lijst = een knoop toevoegen op een bepaalde positie in een gelinkte ketting

(a) Tussen 2 knopen:



32

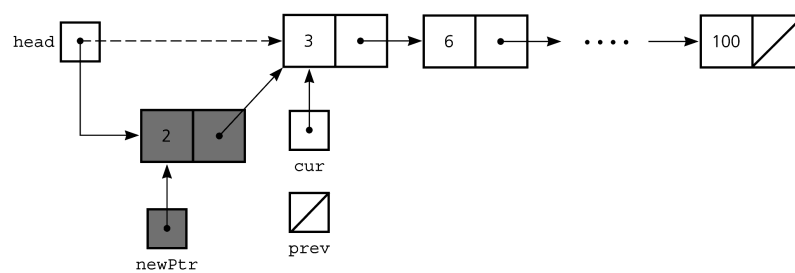
(b) Aan het eind:



d.i. zoals in (a), dus geen speciaal geval.

33

(c) Vooraan:



34

Vergelijking array- en kettingstructuren

Array ~ vakjesblok

- o Eenvoudig in gebruik
- o Vaste grootte
- o Gegevens shiften bij toevoegen en verwijderen
- o Gebruikt minder geheugen per element dan een ketting implementatie
- o 1 stap om het i-de element te bereiken: onmiddellijke toegang
- o opmerking: het vergroten van een dynamische array kan verspilling van opslag en tijd betekenen

Gelinkte ketting ~ speelgoedtreintje

- o Grootte varieert dynamisch
- o Geen gegevens shiften tijdens toevoegen en verwijderen
- o i stappen om het i-de element te bereiken: deel van de gelinkte ketting doorlopen

35

Gegevensabstractie en -structuren

Thema 6
Bomen

36

Trees - Bomen

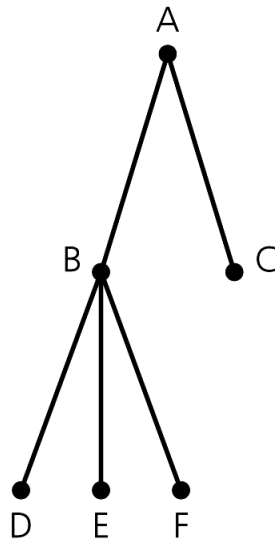


Figure 10-1 A general tree

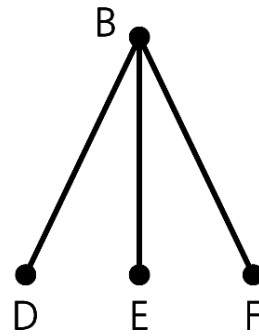
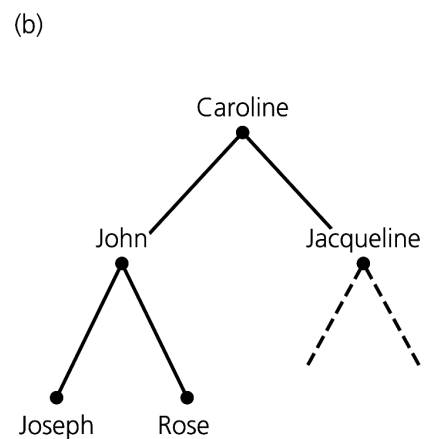
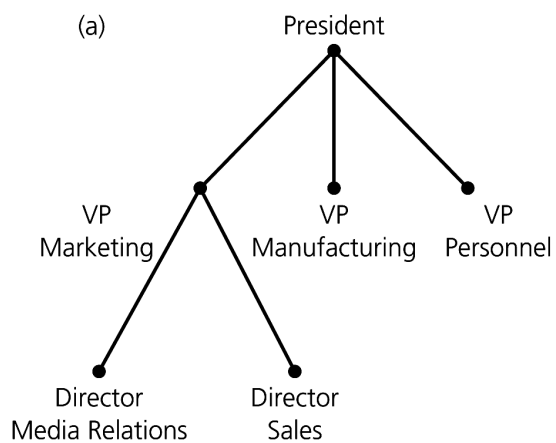


Figure 10-2

A subtree of the tree in Figure 10-1

37

Algemene bomen en binaire bomen



38

Definitie algemene boom

Een **algemene boom** is een verzameling T van één of meerdere knopen zodat

- T is verdeeld in disjuncte deelverzamelingen:
 - $\{r\}$, met r de **wortel** (root)
 - verzamelingen die zelf **algemene bomen** zijn, **deelbomen** van de wortel r .

Definitie binaire boom

Een **binaire boom** is een verzameling T van knopen zodat

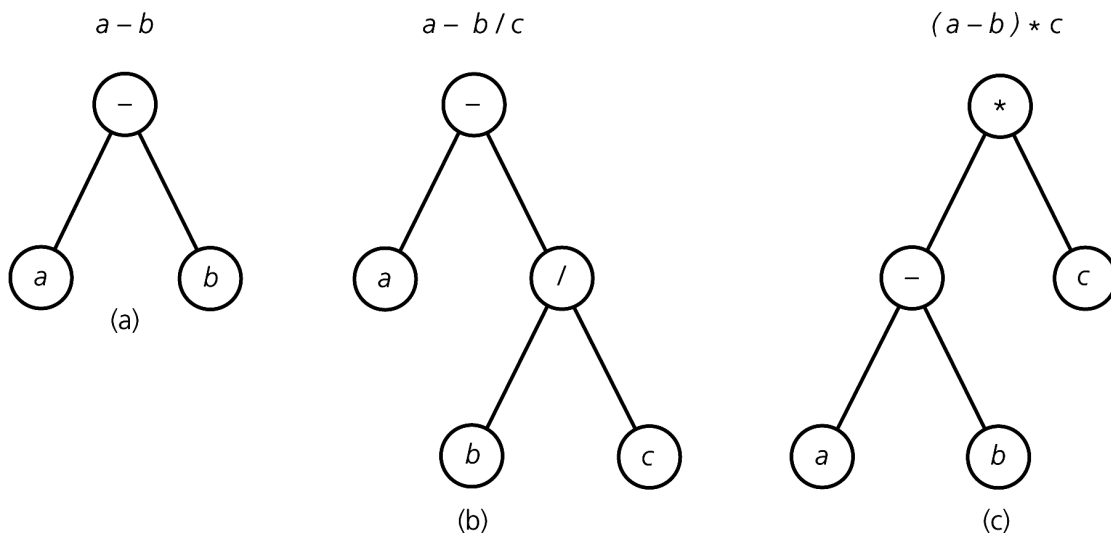
- T is leeg, of
- T is verdeeld in 3 disjuncte deelverzamelingen:
 - $\{r\}$, met r de **wortel** (root)
 - 2 verzamelingen die zelf **binaire bomen** zijn, de **linkerdeelboom** T_L en de **rechterdeelboom** T_R van de wortel r .

39

De wortel w van de linkerdeelboom T_L van r is het **linkerkind** van r en r is de **ouder** van w .

De wortel w van de rechterdeelboom T_R van r is het **rechterkind** van r en r is de **ouder** van w .

Een knoop zonder kinderen is een **blad**. Een knoop zonder ouder is een **wortel**. Een knoop met minstens één kind is een **interne knoop**.



40

Pad: een opeenvolging van knopen $k_1, k_2, \dots, k_n$ zodat elke k_{i+1} een kind is van k_i

Voorouder van een knoop k : een knoop op het pad van de wortel tot k . De ouder van een knoop is z'n eerste voorouder, de ouder van de ouder is z'n volgende voorouder, enz. tot de wortel aan de beurt is. De wortel is een voorouder van elke andere knoop.

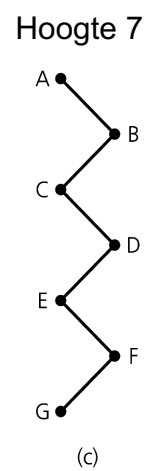
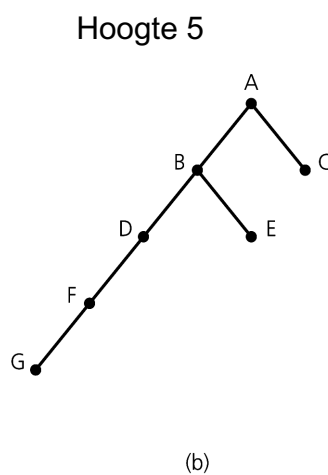
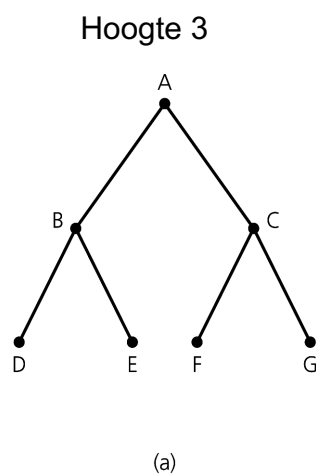
Afstameling van een knoop k : een knoop op een pad van k tot een blad. De kinderen van een knoop zijn z'n eerste afstammelingen. De kinderen van de kinderen zijn z'n volgende afstammelingen, ...

Knopen met dezelfde ouder zijn **siblings**.

41

De hoogte van een boom

De **hoogte** van een boom B is het aantal knopen op het langste pad van de wortel tot een blad.



42

De hoogte van een boom - definities

Het **niveau** van een knoop k in een boom B is

- 1, indien k de wortel van B is
- 1 groter dan het niveau van k 's ouder, indien k geen wortel is.

De **hoogte** van een boom B is

- 0 indien B leeg is
- het maximum niveau van k 's knopen indien B niet leeg is

of

De **hoogte** van een boom B is

- 0, indien B leeg is
- 1 meer dan de hoogte van de hoogste deelboom van de wortel van B , indien B niet leeg is: $hoogte(B) = 1 + \max(hoogte(B_L), hoogte(B_R))$

43

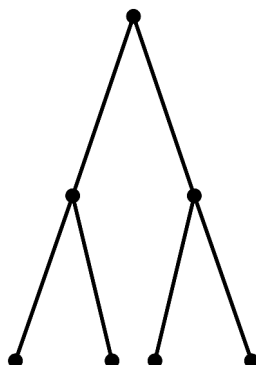
Volle binaire bomen

Een binaire boom van hoogte h is vol als

- alle knopen op niveau $< h$ twee kinderen hebben

Recursieve definitie **volle** binaire boom:

- als een binaire boom T leeg is, dan is T een **volle binaire boom** van hoogte 0
- als een binaire boom T niet leeg is met hoogte $h > 0$, dan is T een **volle binaire boom** als de deelbomen van k 's wortel volle binaire bomen zijn van hoogte $h-1$

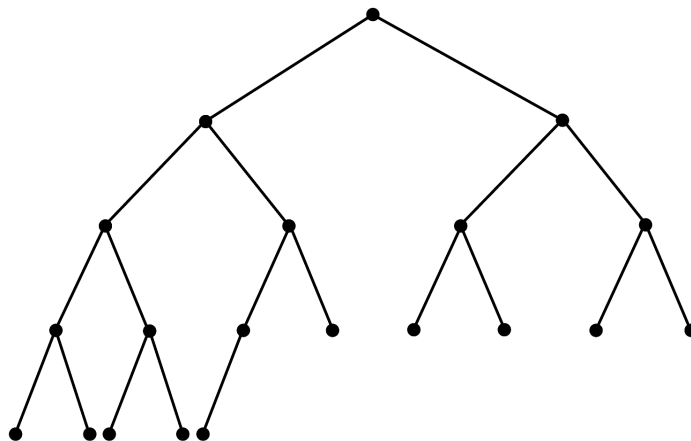


44

Complete binaire bomen

Een binaire boom van hoogte h is **compleet** als hij vol is tot niveau $h-1$ en als niveau h opgevuld is van links naar rechts. Formeel:

- alle knopen op niveau $h-2$ en hoger hebben elk 2 kinderen, en
- als een knoop op niveau $h-1$ minstens 1 kind heeft, dan hebben alle knopen links ervan op hetzelfde niveau elk 2 kinderen, en
- als een knoop op niveau $h-1$ 1 kind heeft, dan is het een linkerkind.



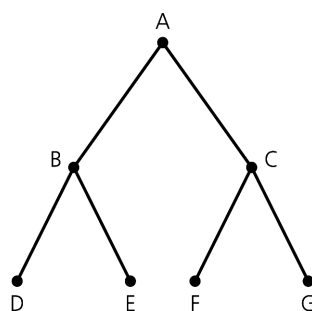
45

Gebalanceerde binaire bomen

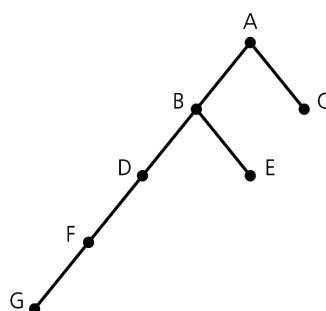
Een binaire boom van hoogte h is **gebalanceerd** als voor elke knoop, het verschil in hoogte tussen zijn 2 deelbomen niet meer is dan 1.

Complete binaire bomen zijn gebalanceerd.

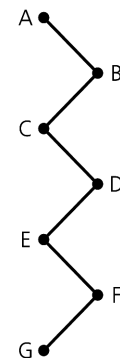
Volle binaire bomen zijn compleet en gebalanceerd.



(a)



(b)

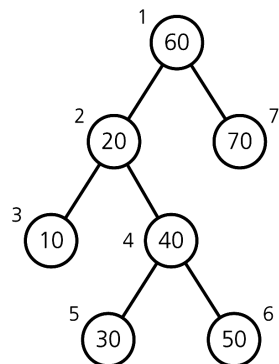


(c)

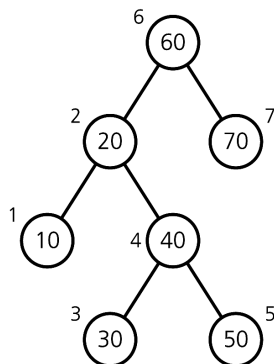
46

Het doorlopen van een binaire boom

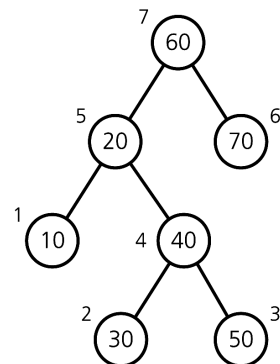
Het doorlopen bezoekt **elke** knoop in de boom en roept telkens een **functie** aan om een bepaalde actie uit te voeren op de bezochte knoop



(a) Preorder: 60, 20, 10, 40, 30, 50, 70



(b) Inorder: 10, 20, 30, 40, 50, 60, 70



(c) Postorder: 10, 30, 50, 40, 20, 70, 60

(Numbers beside nodes indicate traversal order.)

Preorder traversal bezoekt **eerst** de wortel en dan de deelbomen.

preorder(in binTree: BinaryTree)

// Doorloopt de binaire boom binTree in preorder en toont de gegevens

// van elke bezochte knoop

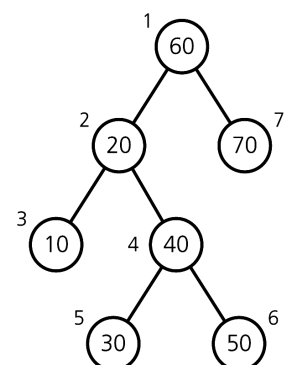
If (binTree is not empty)

{ display the data in the root of binTree

preorder(left subtree of binTree's root)

preorder(right subtree of binTree's root)

}



(a) Preorder: 60, 20, 10, 40, 30, 50, 70

Inorder traversal bezoekt de wortel **tussen** het doorlopen van de deelbomen.

inorder(in binTree: BinaryTree)

// Doorloopt de binaire boom binTree in inorder en toont de gegevens

// van elke bezochte knoop

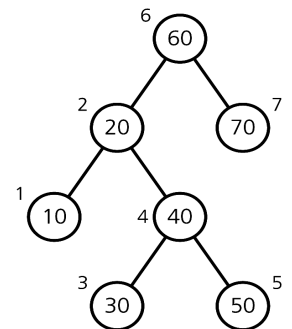
If (binTree is not empty)

{ ***inorder***(left subtree of binTree's root)

display the data in the root of binTree

inorder(right subtree of binTree's root)

}



(b) Inorder: 10, 20, 30, 40, 50, 60, 70

49

Postorder traversal bezoekt de wortel **na** het doorlopen van de deelbomen

postorder(in binTree: BinaryTree)

// Doorloopt de binaire boom binTree in postorder en toont de gegevens

// van elke bezochte knoop

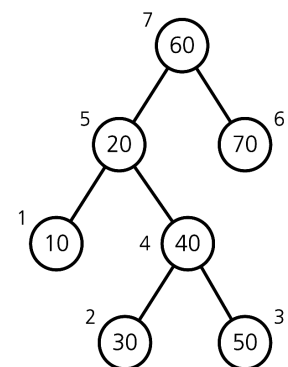
If (binTree is not empty)

{ ***postorder***(left subtree of binTree's root)

postorder(right subtree of binTree's root)

display the data in the root of binTree

}



(c) Postorder: 10, 30, 50, 40, 20, 70, 60

50

Zoeken in een binaire boom

- binaire boom: niet geschikt om snel een bepaald item te zoeken.
- de gegevens organiseren volgens **waarde**
- binaire **zoekboom** – maar welke waarde?
- Elk item is een **record** met verschillende gegevens**velden** (vb. naam, adres, studentnummer, ...)
 - Een **zoeksleutel** (search key) is een veld dat een record kan identificeren (vb. studentnummer).
 - We nemen hier aan dat één enkel veld als zoeksleutel volstaat om een item te identificeren.

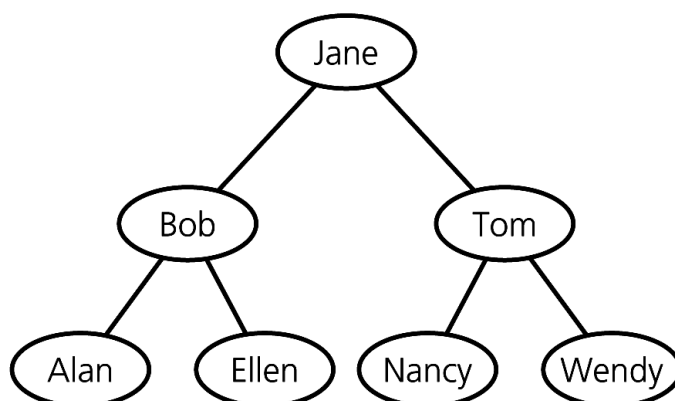
51

Binaire zoekbomen

Definitie.

Een **binaire zoekboom** is een binaire boom zodat voor elke knoop k geldt:

- de waarde van k is $>$ elke waarde in k 's linkerdeelboom
- de waarde van k is $<$ elke waarde in k 's rechterdeelboom
- de linker- en rechterdeelbomen zijn zelf **binaire zoekbomen**



52

Het ADT Binaire Zoekboom

De bewerkingen:

1. Creëer een lege binaire zoekboom.
2. Wis een binaire zoekboom.
3. Bepaal of een binaire zoekboom leeg is.
4. Voeg een nieuw item toe aan de binaire zoekboom.
5. Verwijder het item met een gegeven zoeksleutelwaarde uit de binaire zoekboom.
6. Vraag het item met een bepaalde zoeksleutelwaarde op.
7. Doorloop de knopen in een binaire zoekboom in preorder, inorder of postorder.

53

ADT binaire zoekboom: bewerkingencontract

```
// TreeltemType is het type van de elementen in de binaire zoekboom.  
// Een element van dit type heeft een zoeksleutel veld van het type KeyType.  
  
+createSearchTree()  
  
+destroySearchTree()  
  
+isEmpty(): boolean {query}  
  
+searchTreeInsert(in newltem: TreeltemType, out success: boolean)  
// Voegt newltem toe aan een binaire zoekboom met items met unieke  
// zoeksleutels verschillend van de zoeksleutel van newltem. Success  
// geeft weer of het toevoegen gelukt is.
```

54

```

+searchTreeDelete(in searchKey: KeyType, out success: boolean)
// ...

+searchTreeRetrieve(in searchKey: KeyType, out treeltem: TreeltemType,
                    out success: boolean)
//

+preorderTraverse(in visit: FunctionType)
//

+inorderTraverse(in visit: FunctionType)
//

+postorderTraverse(in visit: FunctionType)
//

```

55

ADT binaire zoekboom: zoekalgoritme

```

search(in binTree: BinarySearchTree, in searchKey: KeyType)
// Zoekt in de binaire zoekboom binTree naar het item met searchKey als
// zoeksleutel.

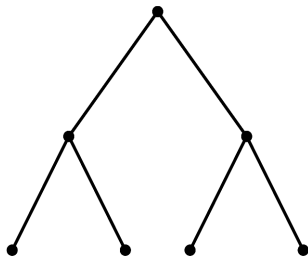
if (binTree is empty)
    gezochte record is niet gevonden
else if (searchKey == zoeksleutel van item in root)
    gezochte record is gevonden
else if (searchKey < zoeksleutel van item in root)
    search(Left subtree of binTree, searchKey)
else
    search(Right subtree of binTree, searchKey)

```

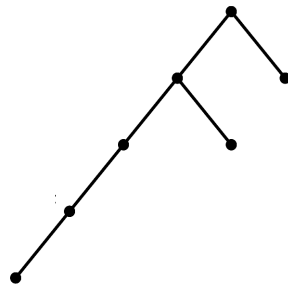
56

ADT binaire zoekboom: zoekalgoritme

- zoeken is basis voor insert, delete en retrieve
- er zijn verschillende binaire zoekbomen met dezelfde gegevens
 - verschillende efficiënties



(a)



(b)



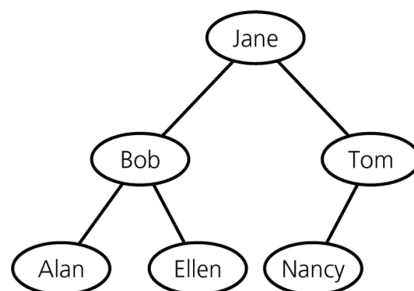
(c)

57

ADT Binaire zoekboom: insert algoritme

- gebruik het zoekalgoritme om de plaats van het toe te voegen item te zoeken
 - zoeken stopt bij een lege boom
 - item kan daar als nieuw blad worden toegevoegd
 - item zit dan op een plaats waar het zoekalgoritme het later zal terugvinden
 - binaire-zoekboom-eigenschappen blijven bewaard.

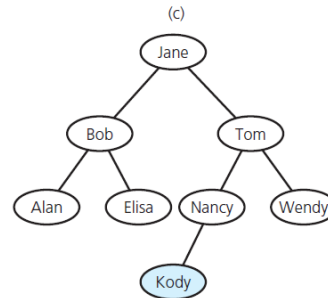
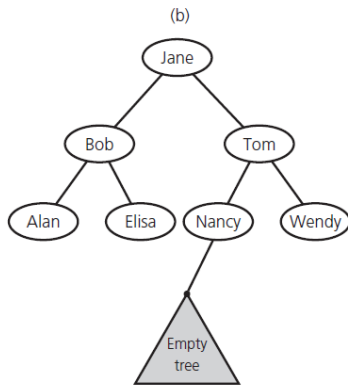
(a)



58

ADT Binaire zoekboom - insert algoritme

- stel dat we het zoekalgoritme toepassen op het nieuwe item 'Kody'
 - zoeken stopt bij een lege deelboom van Nancy
 - item kan als nieuw blad en linkerkind van Nancy worden toegevoegd
 - Kody zit dan op een plaats waar het zoekalgoritme het later zal terugvinden; binaire-zoekboom-eigenschappen blijven bewaard.



Welke boom krijgen we indien we de volgorde van het resultaat van een preorder traversal van een binaire zoekboom gebruiken om een binaire zoekboom te maken met behulp van searchTreeInsert?

59

ADT Binaire zoekboom: delete algoritme

Vind de te verwijderen knoop N, met het zoekalgoritme.

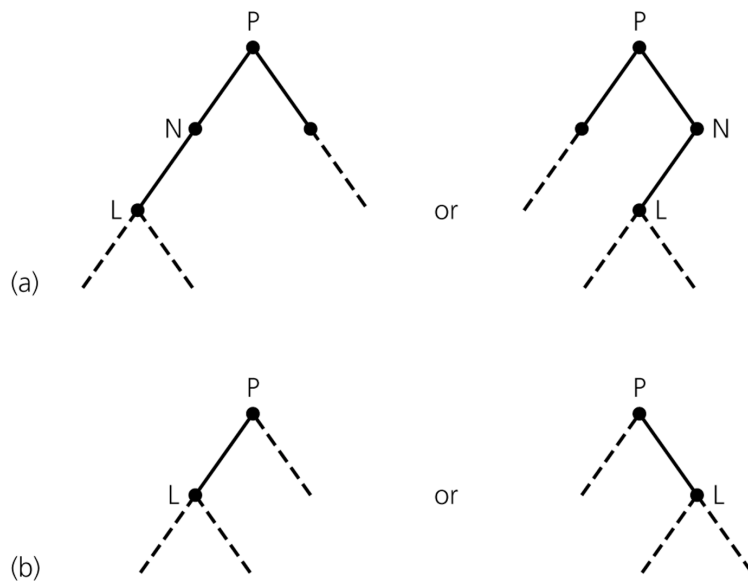
Drie mogelijkheden

1. N is een blad → verwijder het blad
2. N heeft 1 kind → laat het kind de plaats van N innemen.
3. N heeft 2 kinderen → ...

60

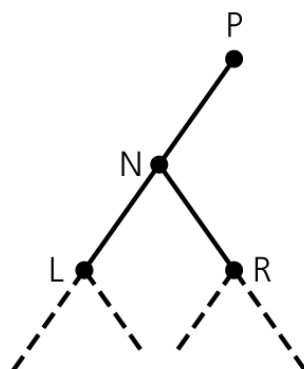
Mogelijkheid 2: N heeft 1 kind.

- Laat het kind de plaats van N innemen. Delete N uit fig.(a) geeft fig.(b)



61

Mogelijkheid 3: N heeft 2 kinderen.



Een item verwijderen kan ook zonder de knoop N van dat item te verwijderen ...

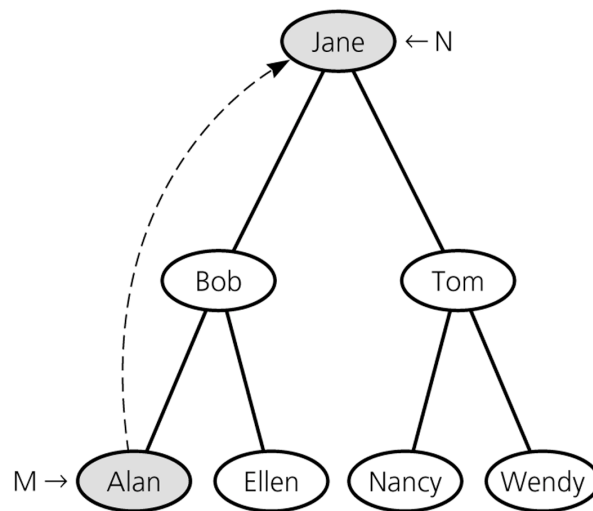
Strategie:

- Vind een geschikte knoop M die eenvoudiger te verwijderen is.
- Kopieer item in M naar N; originele item in N wordt zo overschreven (verwijderd).
- Verwijder knoop M van de boom.

62

MAAR: de eigenschappen van de binaire zoekboom moeten bewaard blijven !

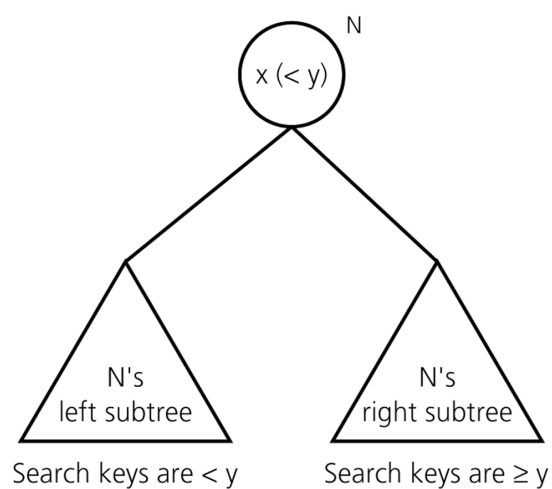
In volgende figuur is M geen goede keuze!



63

Welke knoop is geschikt als knoop M?

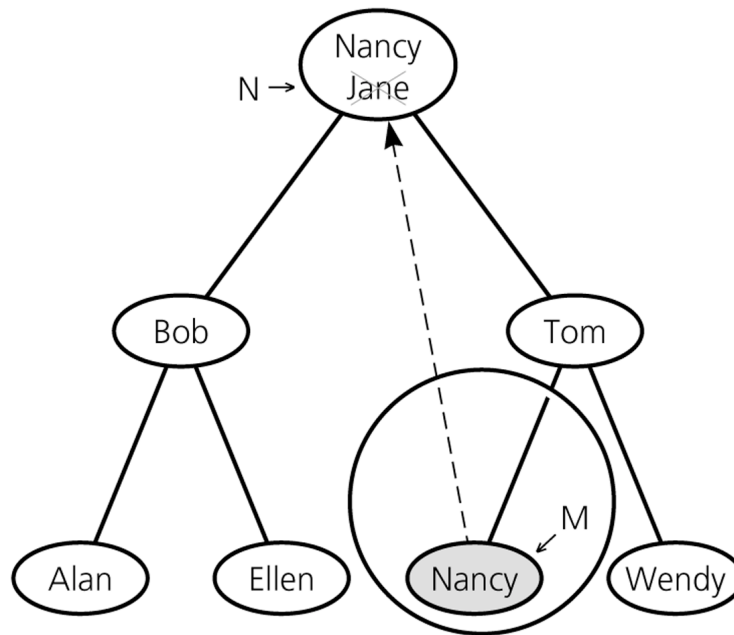
- een blad of een knoop met 1 kind.
- zoeksleutel van item in M moet juist voor (= inorder predecessor) of juist na (= **inorder successor**) zoeksleutel van item in N komen
- Waar zitten de inorder predecessor en successor van knoop N?



M is de meest rechtse knoop in N's linkerdeelboom of de meest linkse knoop in N's rechterdeelboom.

64

Voorbeeld. Waar zit de inorder successor van knoop N?



65

ADT Binaire zoekboom: retrieve (of getEntry) algoritme

Verfijn het zoekalgoritme: return het gezochte item, indien gevonden

if (binTree is empty)

 gezochte record is niet gevonden

else if (searchKey == zoeksleutel van item in root)

 gezochte record is gevonden: **return record**

else if (searchKey < zoeksleutel van item in root)

 retrieve(Left subtree of binTree, searchKey)

else

 retrieve(Right subtree of binTree, searchKey)

66

Gegevensabstractie en -structuren

Thema 7 Gebalanceerde zoekbomen

67

(On)efficiëntie binaire zoekbomen

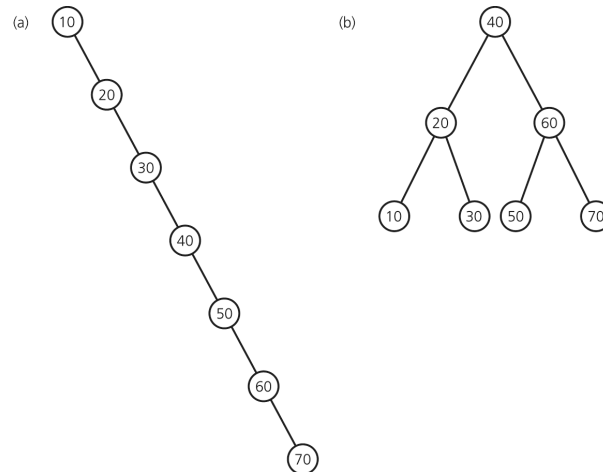
De efficiëntie van de bewerkingen in een binaire zoekboom hangt af van de hoogte van de boom:

retrieve, insert en delete

- volgen elk een pad van de wortel van de boom tot een knoop
- vergelijken voor elke knoop k op dit pad, k 's zoek sleutel met een gegeven zoek sleutel om te bepalen of, en in welke deelboom, de bewerking wordt verdergezet
- voeren elk een aantal vergelijkingen uit – dit aantal is kleiner dan of gelijk aan de hoogte van de boom

68

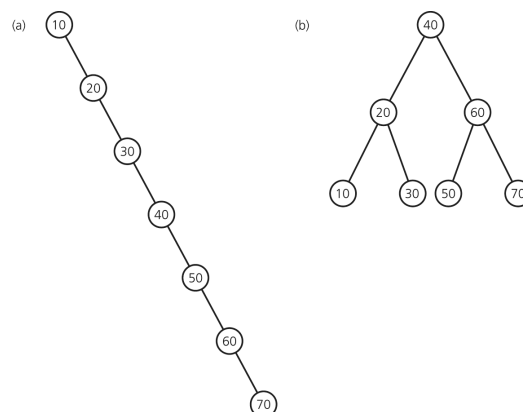
- ➔ het zoeken in een binaire zoekboom kan even onefficiënt zijn als sequentieel zoeken (figuur a) en even efficiënt als het binaire zoeken in een array (figuur b)



69

Een binaire zoekboom met n items heeft hoogte

- maximum: n
- minimum: $\lceil \log_2(n + 1) \rceil$
- afhankelijk van de **volgorde** van toevoegen en verwijderen van items



70

Gebalanceerde zoekbomen

Het is belangrijk de best mogelijke efficiëntie te realiseren ...

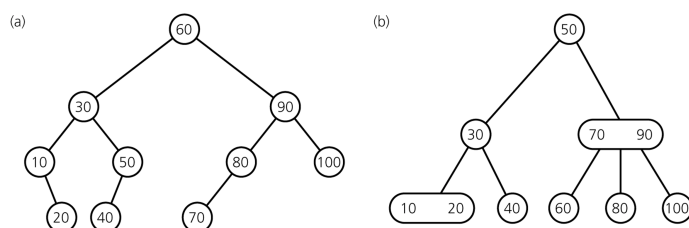
Variaties op de binaire zoekbomen

- 2-3 bomen
- 2-3-4 bomen
- rood-zwart bomen

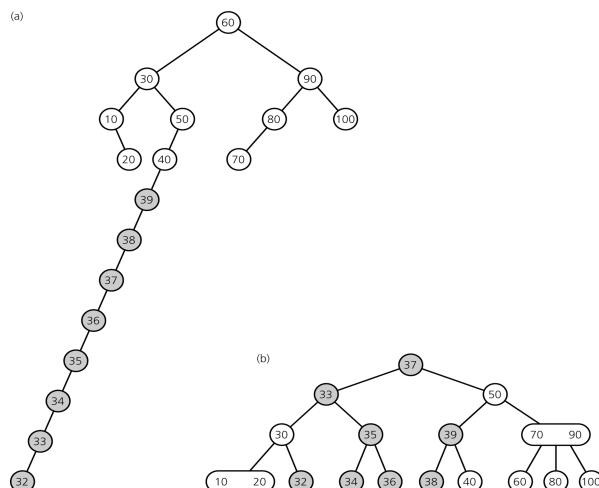
- behouden het gebalanceerd zijn na toevoegen en verwijderen
- zijn makkelijker te onderhouden dan gebalanceerde binaire zoekbomen

71

Een voorbeeldje: toevoegen in binaire zoekbomen vs. in 2-3 bomen

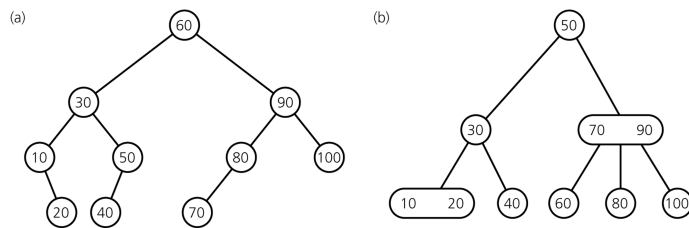


Achtereenvolgens 39, 38, 37, 36, 35, 34, 33 en 32 toevoegen aan deze bomen geeft volgende bomen:

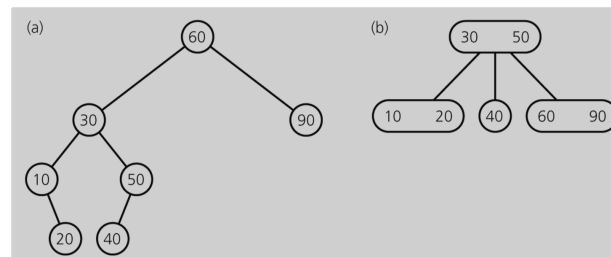


72

Een voorbeeldje: verwijderen uit binaire zoekbomen vs. uit 2-3 bomen



Achtereenvolgens 70, 100, 80 verwijderen uit deze bomen geeft volgende bomen:

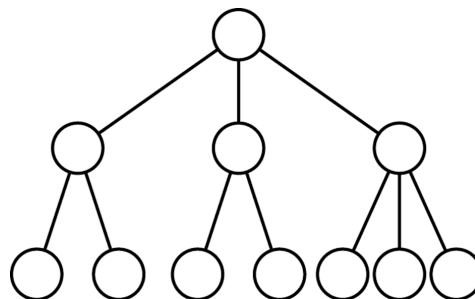


2-3 bomen

2-3 bomen blijven steeds gebalanceerd bij toevoegen en verwijderen.

Alle bladeren zitten op hetzelfde niveau.

Elke interne knoop (geen blad) heeft ofwel 2 kinderen (een **2-knoop**) ofwel 3 kinderen (een **3-knoop**).



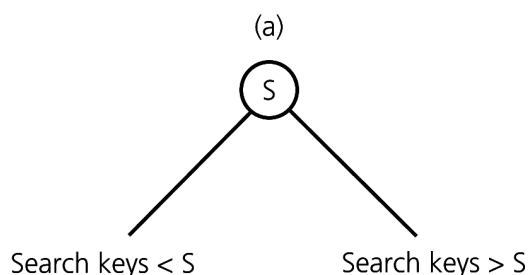
Een 2-3 boom zonder 3-knopen is een volle binaire boom.

Een 2-3 boom is nooit hoger dan een binaire boom met hetzelfde aantal knopen en minimale hoogte.

Een **2-3 boom** van hoogte h is een verzameling T van knopen zodat

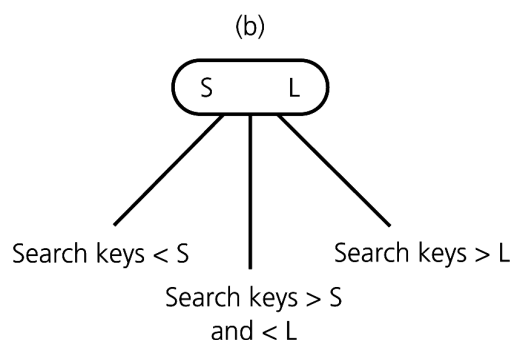
- ofwel is T leeg, een 2-3 boom van hoogte 0.
- ofwel is T verdeeld in 3 disjuncte deelverzamelingen

de wortel is een 2-knoop ...



- ofwel is T verdeeld in 4 disjuncte deelverzamelingen:

de wortel is een 3-knoop ...



75

Definitie: Een **2-3 boom** van hoogte h is een verzameling T van knopen zodat

- ofwel is T leeg, een 2-3 boom van hoogte 0.
- ofwel is T verdeeld in 3 disjuncte deelverzamelingen:
 - $\{r\}$, met r de wortel die **1 data item** bevat
 - 2 verzamelingen die zelf **2-3 bomen** van hoogte $h-1$ zijn, de **linkerdeelboom** T_L en de **rechterdeelboom** T_R van de wortel r .
De zoeksleutel van het data item in r is groter dan elke zoeksleutel in de linkerdeelboom T_L en kleiner dan elke zoeksleutel in de rechterdeelboom T_R .
- ofwel is T verdeeld in 4 disjuncte deelverzamelingen:
 - $\{r\}$, met r de wortel die **2 data items** bevat
 - 3 verzamelingen die zelf **2-3 bomen** van hoogte $h-1$ zijn, de **linkerdeelboom** T_L , de **middelste deelboom** T_M en de **rechterdeelboom** T_R van de wortel r .
De kleinste (linker) zoeksleutel in r is groter dan elke zoeksleutel in de linkerdeelboom T_L en kleiner dan elke zoeksleutel in de middelste deelboom T_M . De grootste (rechter) zoeksleutel in r is groter dan elke zoeksleutel in de middelste deelboom T_M en kleiner dan elke zoeksleutel in de rechterdeelboom T_R .

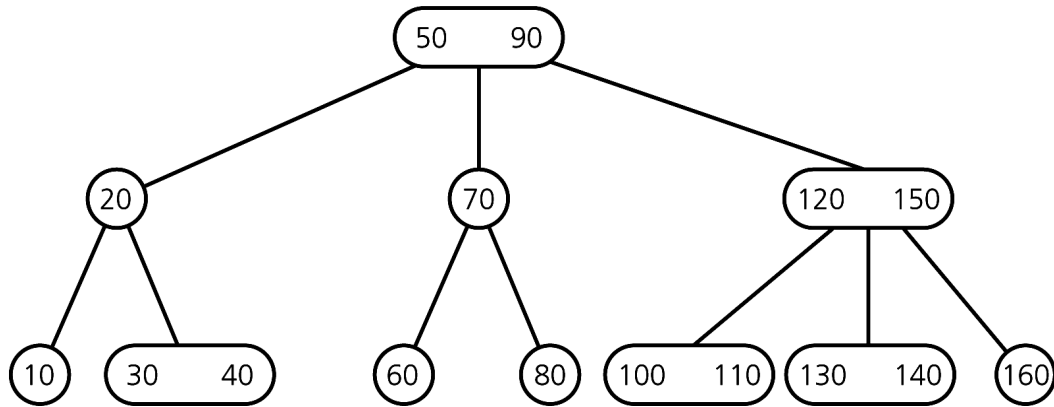
76

Eigenschappen van 2-3 bomen:

Een knoop met 2 kinderen, moet 1 item bevatten.

Een knoop met 3 kinderen, moet 2 items bevatten.

Een blad mag 1 of 2 items bevatten (heeft 2 of 3 lege deelbomen).



77

Bewerkingen op 2-3 bomen

De algoritmen voor het

- doorlopen van een 2-3 boom,
- zoeken in een 2-3 boom
- toevoegen in een 2-3 boom
- verwijderen uit een 2-3 boom

zijn

- recursief met basisgeval een blad → de input boom mag niet leeg zijn.

78

Het doorlopen van een 2-3 boom

inorder(in tree: TwoThreetree)

// doorloopt de niet-lege 2-3 boom tree in volgorde van de zoeksleutels

if (de wortel r van tree is een blad)
 bezoek de data item(s) van r

else if (r heeft 2 data items)
{ **inorder**(linkerdeelboom van r)
 bezoek het kleinste data item van r
 inorder(middelste deelboom van r)
 bezoek het grootste data item van r
 inorder(rechterdeelboom van r)
}
else // r heeft 1 data item
{ **inorder**(linkerdeelboom van r)
 bezoek het data item van r
 inorder(rechterdeelboom van r)
}

79

Zoeken in een 2-3 boom

retrieveItem(in tree: TwoThreetree, in searchkey: KeyType,
 out treeltem: TreeltemType): boolean

// Plaatst het item met zoeksleutel searchKey uit de niet-lege 2-3 boom tree
// in treeltem. De functie geeft de waarde true als het item gevonden is en
// false als het niet aanwezig is.

if (searchKey is in de wortel r van tree)
{ // item is gevonden
 treeltem := het gevonden item
 return true
}
else if (r is een blad)
 return false

80

```

// zoek verder in de juiste deelboom
else if (r heeft 2 data items)
{ if (searchkey < kleinste zoeksleutel in r)
    return retrieveItem(linkerdeelboom van r, searchKey, treeltem)
  else if (searchkey < grootste zoeksleutel in r)
    return retrieveItem(middelste deelboom van r, searchKey, treeltem)
  else
    return retrieveItem(rechterdeelboom van r, searchKey, treeltem)
}

else // r heeft 1 data item
{ if (searchkey < zoeksleutel in r)
    return retrieveItem(linkerdeelboom van r, searchKey, treeltem)
  else
    return retrieveItem(rechterdeelboom van r, searchKey, treeltem)
} // end if

```

81

De zoekefficiëntie in een 2-3 boom vs. in een zo goed mogelijk gebalanceerde binaire zoekboom met dezelfde elementen

De hoogte van een 2-3 boom is kleiner dan of gelijk aan de hoogte in een zo goed mogelijk gebalanceerde binaire zoekboom.

→ het aantal bezochte knopen in een 2-3 boom is kleiner dan of gelijk aan het aantal bezochte knopen in een zo goed mogelijk gebalanceerde binaire zoekboom (1)

Maar: het aantal vergelijkingen per knoop kan groter zijn in een 2-3 boom (2)

(1)+(2) → Het aantal vergelijkingen bij het zoeken naar een item in een 2-3 boom is *ongeveer gelijk* aan het aantal vergelijkingen bij het zoeken naar een item in een binaire zoekboom **die zo goed mogelijk gebalanceerd is**.

De zoekefficiëntie in een 2-3 boom is ongeveer gelijk aan de zoekefficiëntie in een binaire zoekboom die zo goed mogelijk gebalanceerd is.

Waarom dan 2-3 bomen?

Omdat het moeilijk is om een binaire zoekboom zo goed mogelijk gebalanceerd te houden bij het toevoegen en verwijderen van elementen.

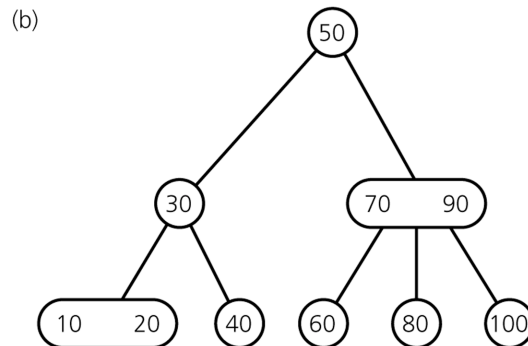
Omdat het relatief eenvoudig is om de vorm van een 2-3 boom te behouden.

82

Een element toevoegen aan een 2-3 boom

Voorbeeld.

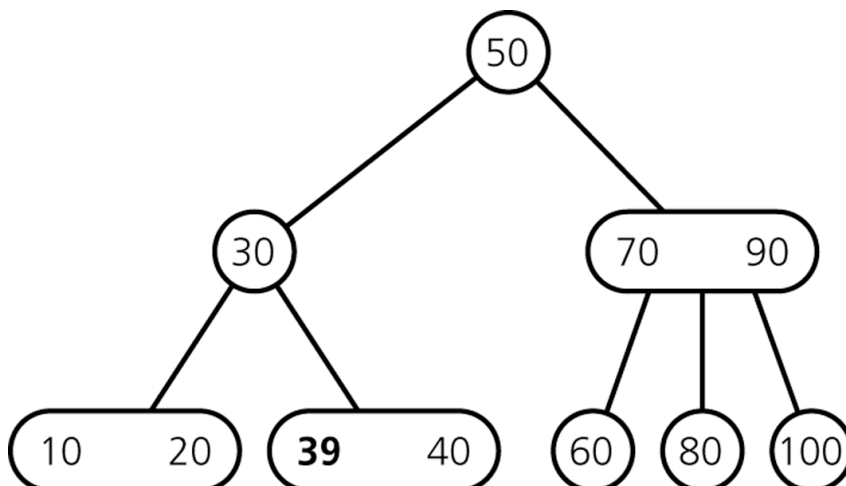
Voeg achtereenvolgens 39, 38, 37, 36, 35, 34, 33 en 32 toe aan volgende 2-3 boom:



83

Insert 39

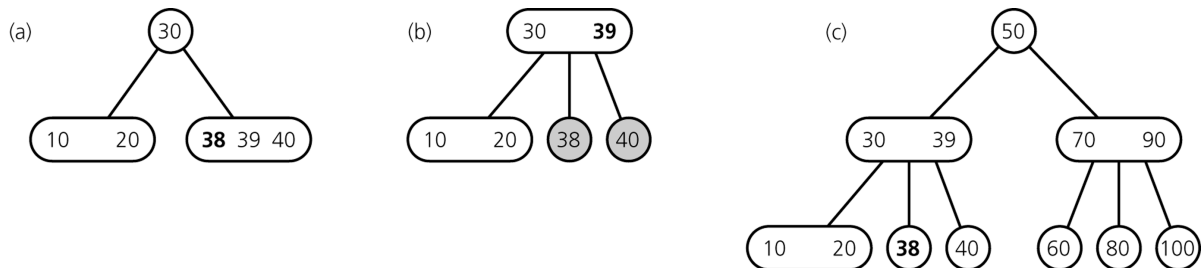
voeg 39 toe in het blad waar het zoeken naar 39 zou stoppen



84

Insert 38

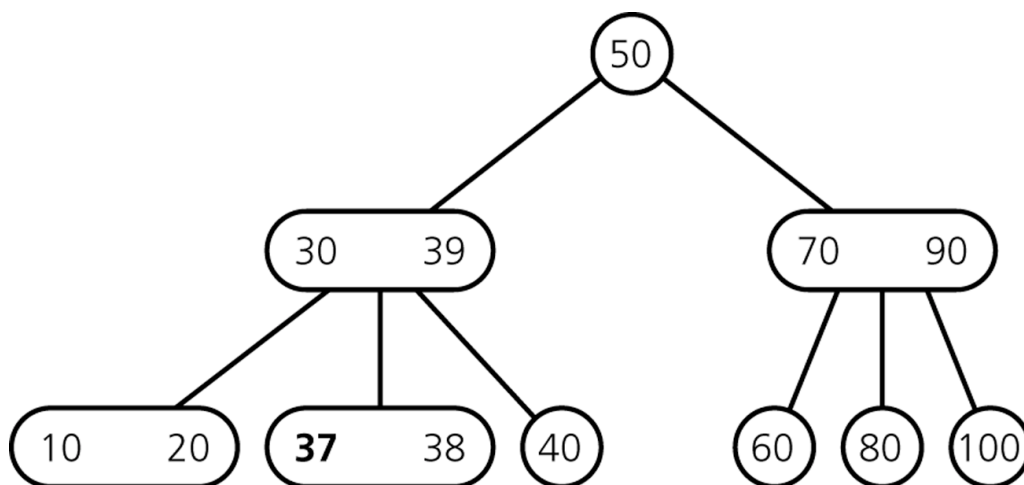
- (a) voeg 38 toe in het blad waar het zoeken naar 38 zou stoppen
→ blad met 3 items! → geen 2-3 boom!
- (b) splits het overvolle blad: het middelste item gaat naar de ouder, de andere 2 elk in een nieuw blad.



85

Insert 37

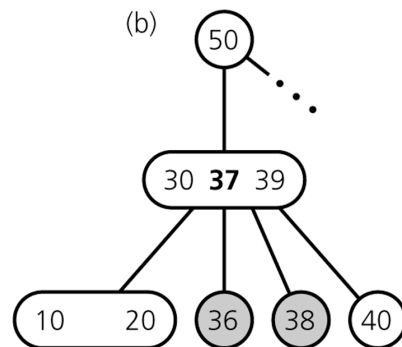
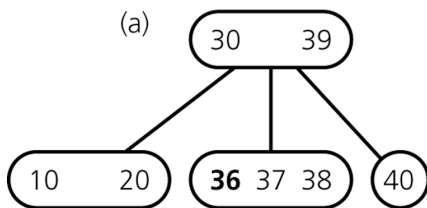
voeg 37 toe in het blad waar het zoeken naar 37 zou stoppen



86

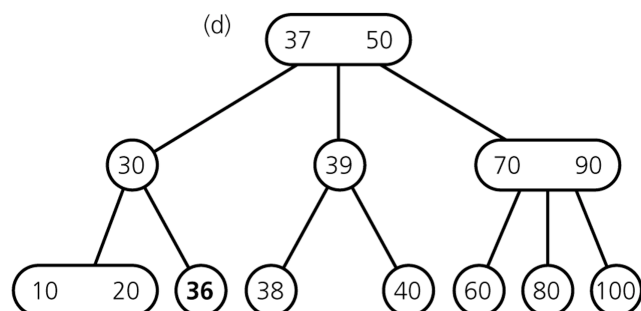
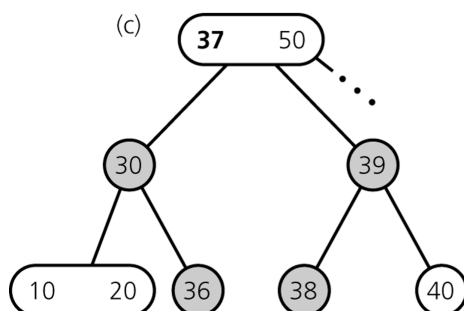
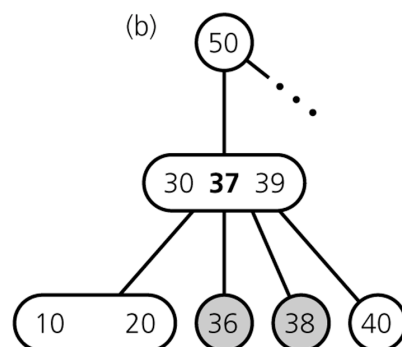
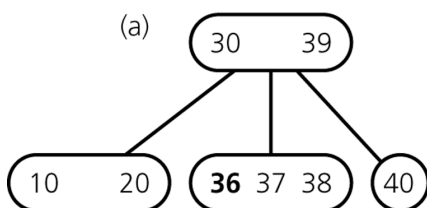
Insert 36

- (a) voeg 36 toe in het blad waar het zoeken naar 36 zou stoppen
 → blad met 3 items! → geen 2-3 boom!
- (b) splits het overvolle blad: het middelste item gaat naar de ouder, de andere 2 elk in een nieuw blad.
 → interne knoop met 3 items! → geen 2-3 boom!
- (c) splits de overvolle knoop: het middelste item (37) gaat naar de ouder, de andere 2 elk in een nieuwe knoop – de kinderen van de overvolle knoop worden nu kinderen van de nieuwe knopen



87

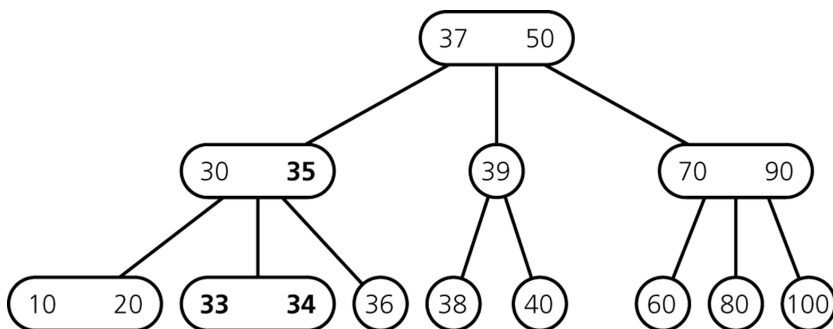
Insert 36 (vervolg)



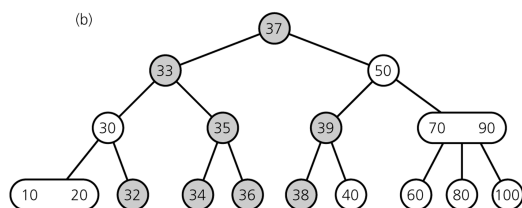
88

Insert 35, 34, 33 en 32

Oefening!



Dit is het juiste resultaat:

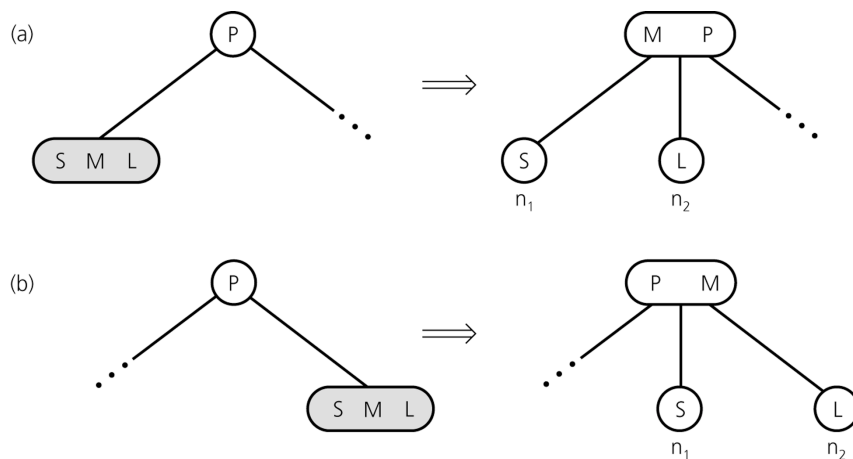


89

Het insert algoritme

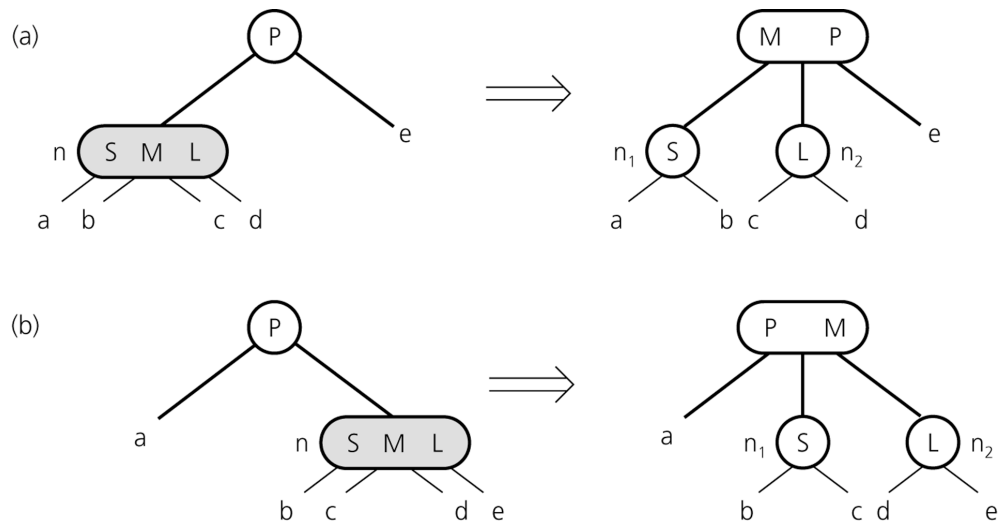
Voeg het nieuwe element altijd toe aan het blad waarin het zoeken naar het nieuwe element stopt.

- Als het blad dan 3 items bevat wordt het gesplitst in 2 bladeren: het linkeritem in het nieuwe linkerblad, het rechteritem in het nieuwe rechterblad en het middelste item in de ouder.



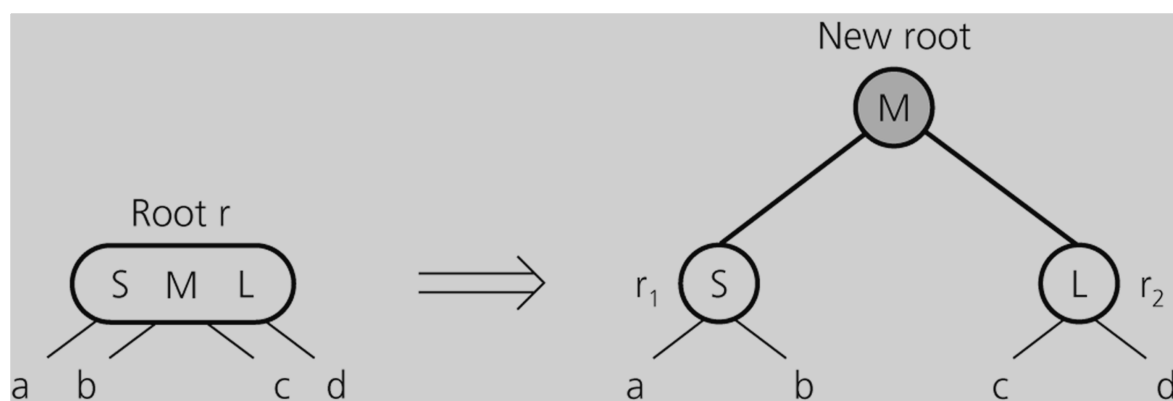
90

2. Als een ouder (of interne knoop) dan 3 items bevat wordt er opnieuw gesplitst: het linkeritem in de nieuwe linkerknoop, het rechteritem in de nieuwe rechterknoop en het middelste item in de ouder. De 2 meest linkse resp. rechtse kinderen van de overvolle knoop worden nu kinderen van de nieuwe linker- resp. rechterknoop.



91

3. Als de wortel 3 items bevat wordt er weer gesplitst en ontstaat er op een analoge manier een nieuwe wortel (ouder) die het middelste item bevat en die als kinderen de nieuwe interne knopen heeft.



Een 2-3 boom kan enkel op deze manier in hoogte toenemen!

92

```

insertItem(in tree:TwoThreeTree, in newItem: TreeltemType)
// Voegt newItem toe in 2-3 boom tree. Alle zoeksleutels in tree zijn
// uniek en verschillen ook van de zoeksleutel van newItem.

leafNode := het blad waarin het newItem volgens z'n zoeksleutel hoort
voeg newItem toe aan leafNode

if (leafNode heeft nu 3 items)
    split(leafNode)

split(inout n: TreeNode)
// Splitst de knoop n die 3 items bevat. Als n geen blad is, dan heeft n
// 4 kinderen.

if (n is de wortel)
    p := een nieuwe knoop
else
    p := de ouder van n

```

93

```

splits de knoop n op in 2 knopen  $n_1$  en  $n_2$  met ouder p

// herverdeel de items van n
plaats in  $n_1$  het item uit n met de kleinste zoeksleutel
plaats in  $n_2$  het item uit n met de grootste zoeksleutel
plaats het item uit n met de middelste zoeksleutel in p

if (n is een interne knoop)          // herverdeel de deelbomen van n
{  $n_1$  wordt ouder van de 2 meest linkse kinderen van n
   $n_2$  wordt ouder van de 2 meest rechtse kinderen van n
}

if (p bevat 3 items)
    split(p)

```

94

Een element verwijderen uit een 2-3 boom

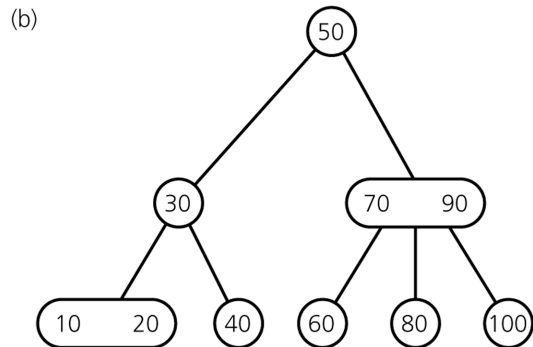
Bij het **toevoegen** van een element **splitsen** we knopen die te vol zijn.

Bij het **verwijderen** van een element **voegen** we knopen **samen** als er 1 leeg wordt.

Het verwijderen start altijd in een blad!

→ als het te verwijderen item in een interne knoop zit, verwisselen we het eerst met z'n 'inorder successor' – de inorder successor van een item in een interne knoop zit altijd in een blad.

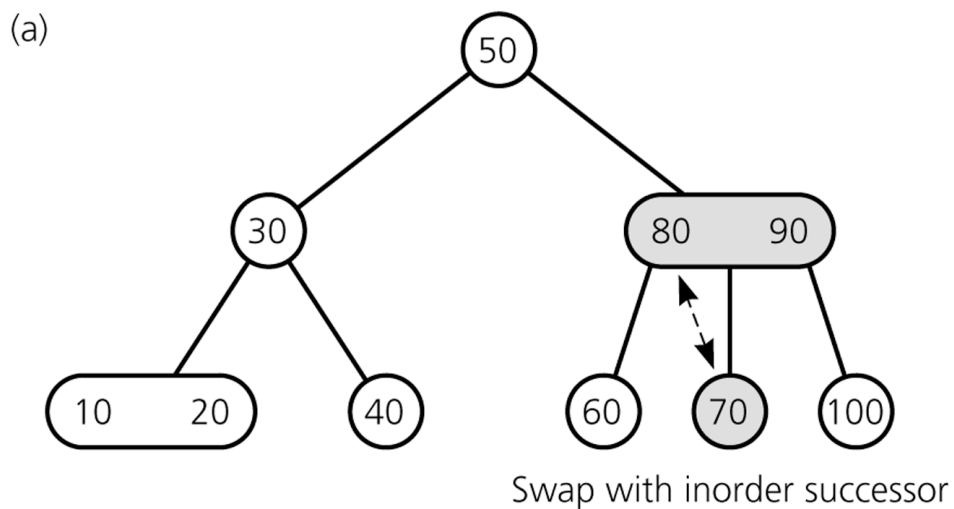
Voorbeeld: verwijder 70, 100, 80 achtereenvolgens uit



95

Voorbeeld 1: verwijder 70

70 zit in een interne knoop en wordt verwisseld met 80, z'n inorder successor.



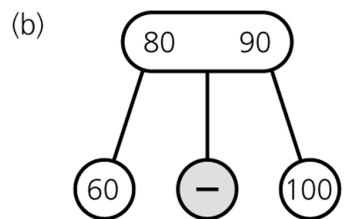
96

fig.(a) Nu 70 in een blad zit kan 70 verwijderd worden.

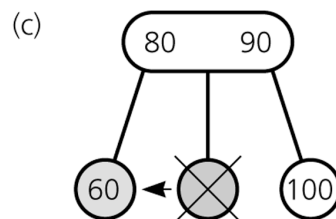
fig.(b) Leeg blad → nog geen 2-3 boom!

fig.(c) Verwijder het blad → interne knoop met 2 items en 2 kinderen
→ nog geen 2-3 boom!

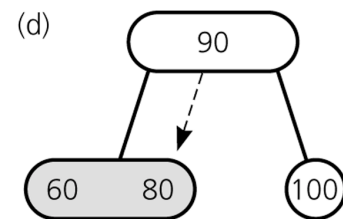
fig.(d) Plaats een item uit de ouder in een sibling van het verwijderde blad
→ nieuwe 2-3 boom!



Delete value from leaf



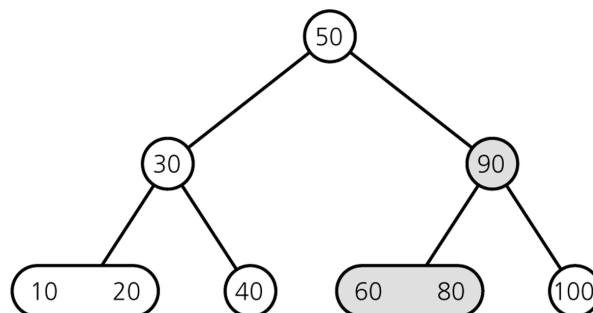
Merge nodes by deleting empty leaf and moving 80 down



(c)+(d) is een **merge** = samenvoegen van items uit ouder en sibling van de verwijderde knoop, in sibling.

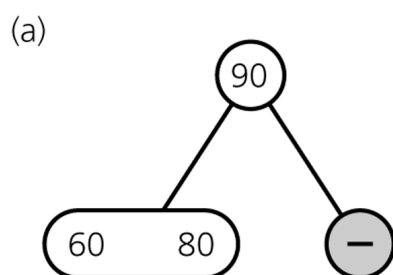
97

Voorbeeld 2: verwijder 100 uit

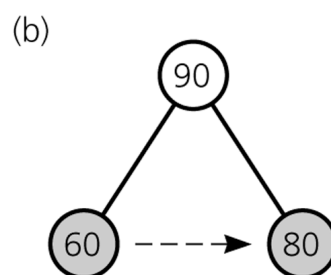


(a) Verwijder 100 uit het blad → leeg blad → nog geen 2-3 boom!

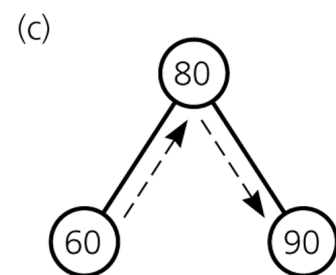
(c) Maar sibling kan een item schenken → **herverdeel** tussen lege knoop, sibling en ouder.



Delete value from leaf



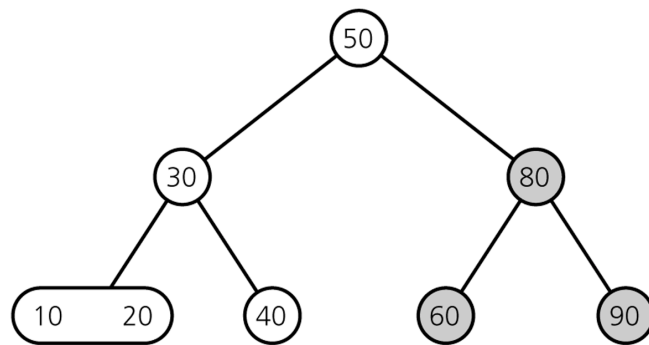
Doesn't work



Redistribute

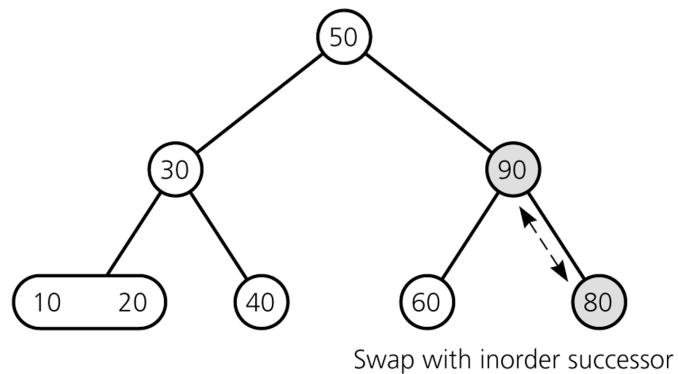
98

Voorbeeld 3: verwijder 80 uit



(a) 80 zit in een interne knoop en wordt verwisseld met 90, z'n inorder successor.

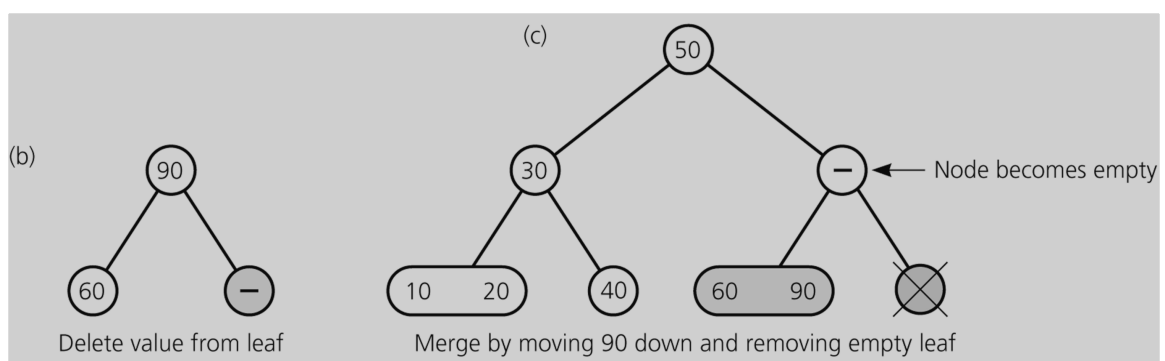
(a)



99

(b) Nu 80 in een blad zit kan het verwijderd worden → leeg blad
→ nog geen 2-3 boom!

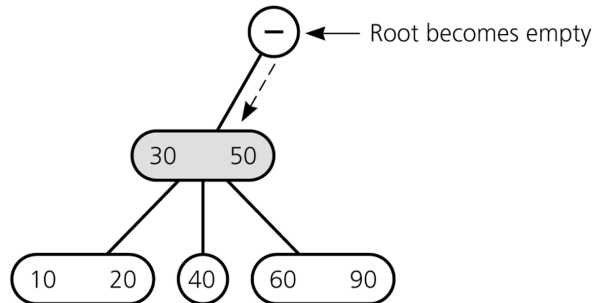
(c) Herverdelen is niet mogelijk (sibling heeft 1 item). Merge zoals in voorbeeld 1 (verwijder de lege knoop en plaats een item uit de ouder in een sibling) → lege interne knoop → nog geen 2-3 boom!



100

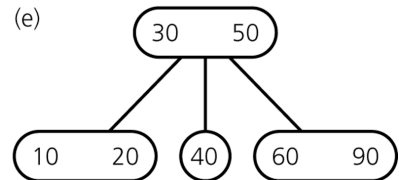
- (d) Sibling van lege interne knoop heeft 1 item dus herverdelen kan niet. Merge: verwijder de lege knoop en plaats een item uit de ouder in sibling
→ lege wortel → nog geen 2-3 boom!
- (e) Lege wortel mag verwijderd worden – het enige kind van de verwijderde wortel wordt de nieuwe wortel. De hoogte is met 1 verminderd.

(d)



Merge: move 50 down, adopt empty leaf's child, remove empty node

(e)



Remove empty root

Het verwijderen uit de rechterdeelboom van een 2-3 boom kan de linkerdeelboom wijzigen. Bij een binaire zoekboom is dit niet het geval.

101

Het delete algoritme

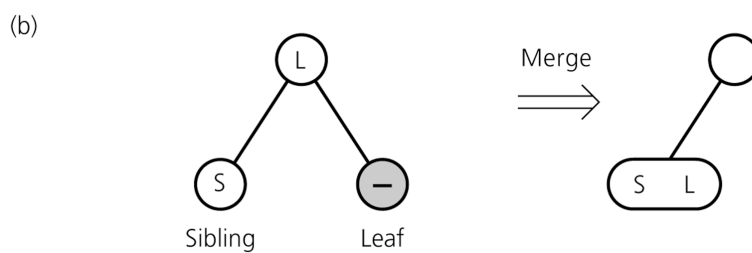
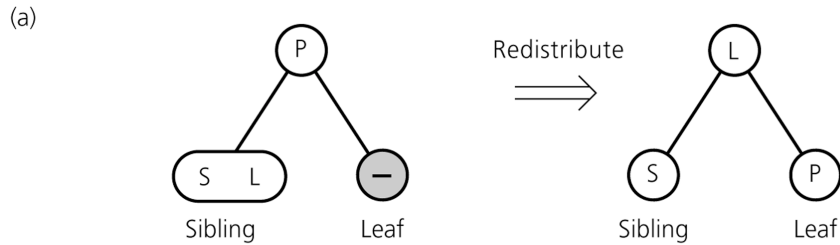
1. Het verwijderen start altijd in een blad.

Om een item uit een interne knoop in een blad te krijgen wordt het verwisseld met z'n 'inorder successor'.

102

2. Het verwijderen van een item uit een blad kan als gevolg hebben dat het blad leeg wordt → geen 2-3 boom:

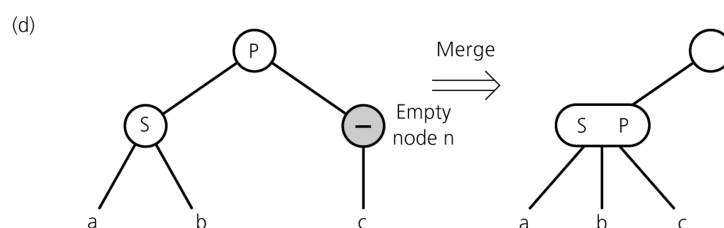
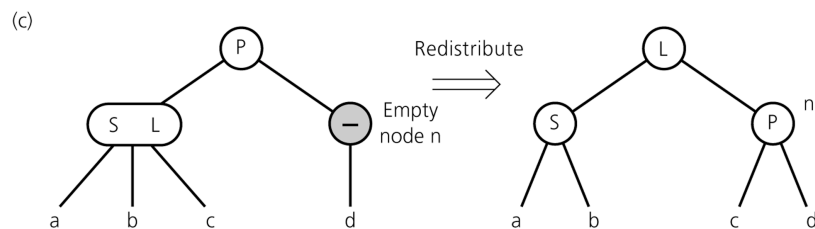
- (a) het lege blad heeft een aangrenzende sibling met 2 elementen → **herverdelen**.
- (b) het lege blad heeft enkel aangrenzende siblings met 1 element → **samenvoegen (merge)**: item uit ouder in sibling.



103

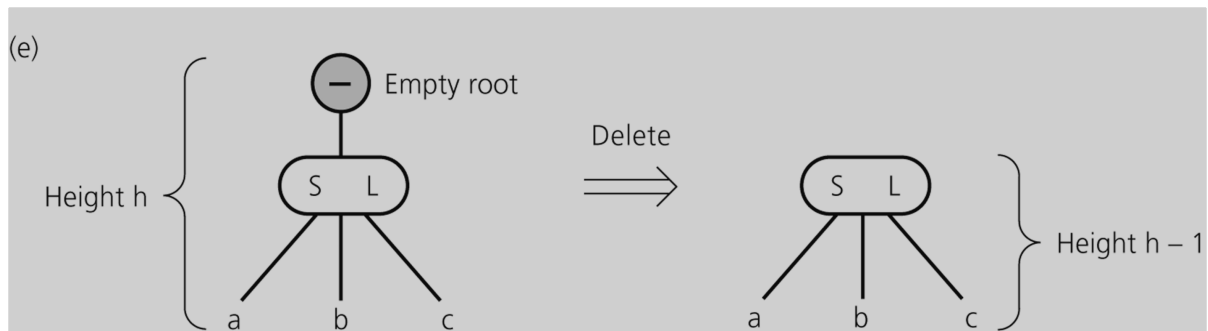
3. Het samenvoegen kan als gevolg hebben dat de ouder leeg wordt → lege interne knoop → geen 2-3 boom:

- (c) de lege knoop heeft een aangrenzende sibling met 2 elementen → **herverdelen**, ook het kind van de sibling, dat het dichtst bij de lege knoop zit.
- (d) de lege knoop heeft enkel aangrenzende siblings met 1 element → **samenvoegen**: item uit ouder in sibling. Het kind van de lege knoop wordt kind van deze sibling.



104

- (e) de lege knoop is de wortel → verwijder de wortel → de hoogte van de boom neemt af.



105

deleteltem(in tree: TwoThreeTree, in searchKey: KeyType, out success: boolean)

// Verwijdert uit de 2-3 boom tree het item met zoek sleutel searchKey.

// success is false als er in tree geen item is met deze zoek sleutel.

node := de knoop die het element item met zoek sleutel searchKey bevat

```

if (node is gevonden)      // er is een item met searchKey
{
    if (node is geen blad)
        verwissel item met z'n inorder successor in het blad leafNode
    // het verwijderen start in een blad
    verwijder item uit leafNode

    if (leafNode heeft geen items)
        fix(leafNode)
    success := true
}
else
    success := false

```

106

```

fix(in n: TreeNode)
// het verwijderen wordt afgerond door het verwijderen van de wortel, het
// herverdelen van items of het samenvoegen van knopen. Indien n geen blad is,
// heeft n 1 kind.

if (n is de wortel)
    verwijder de wortel
else
{
    parent := de ouder van n
    if (een aangrenzende sibling van n heeft 2 items)
    {
        herverdeel de items van sibling en parent over n, sibling en parent
        if (n is een interne knoop)
            verplaats het juiste kind van sibling naar n
    }
    else // samenvoegen van knopen
    {
        kies een aangrenzende sibling van n
        verplaats het juiste item van parent naar sibling
        if (n is een interne knoop)
            verplaats het kind van n naar sibling
        verwijder n
        if (parent is nu leeg)
            fix(parent)
    } // end if
} // end if

```

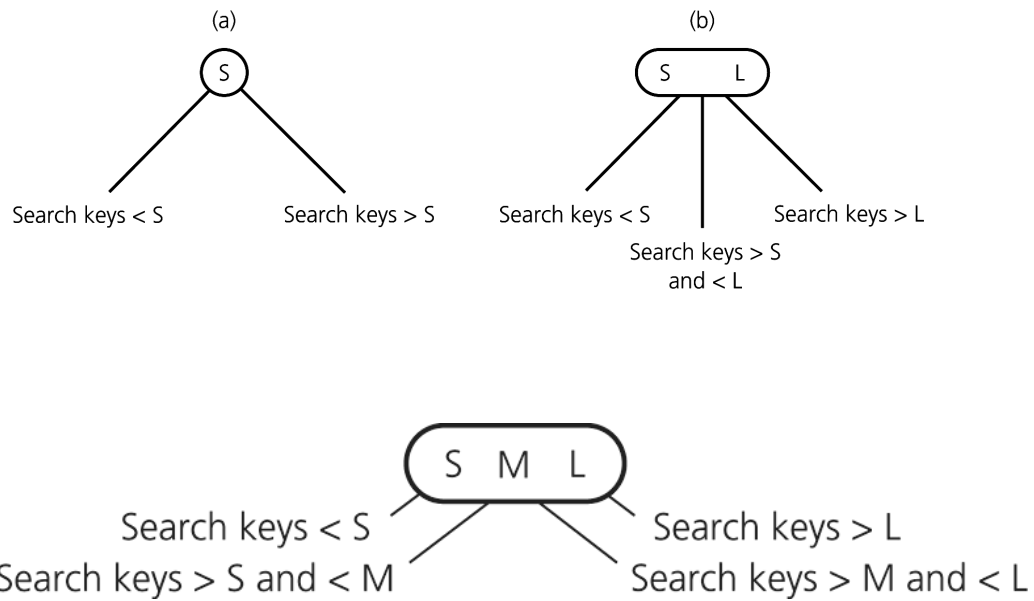
107

2-3-4 bomen

- hebben 2-knopen, 3-knopen en 4-knopen
 - een 2-knoop heeft 1 item en 2 kinderen
 - een 3-knoop heeft 2 items en 3 kinderen
 - een 4-knoop heeft 3 items en 4 kinderen
- zijn algemene bomen, geen binaire bomen
- zijn nooit hoger dan een 2-3 boom
- zoek en traverse algoritmen zijn eenvoudige uitbreidingen van de overeenkomstige algoritmen voor 2-3 bomen

108

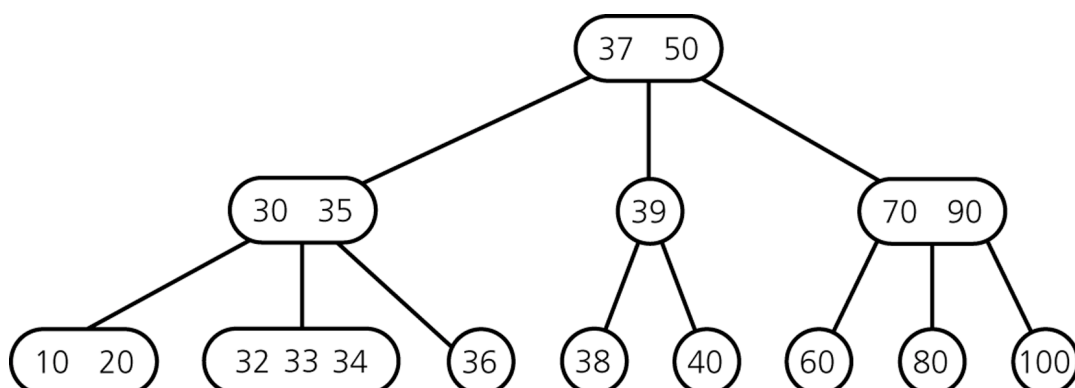
Knopen in een 2-3-4 boom



109

Knopen in een 2-3-4 boom

Een blad kan 1, 2 of 3 items bevatten.



110

Insert in een 2-3-4 boom, beter voorkomen dan genezen

Insert in een 2-3 boom:

- doorloop pad van de wortel naar een blad
- voeg item toe in blad
- splits overvolle knopen op de terugweg van blad richting wortel

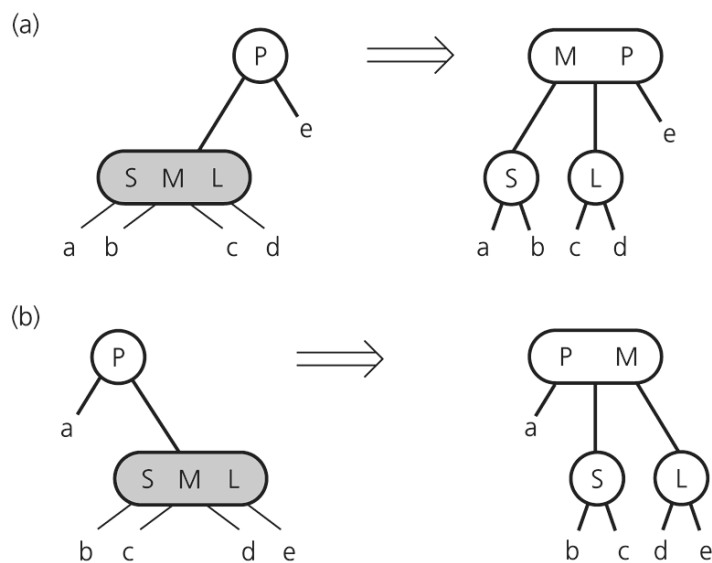
Insert in een 2-3-4 boom:

- doorloop pad van de wortel naar een blad en:
 - **splits elke 4-knoop (3 items, 4 kinderen) die je tegenkomt op dit pad!**
- voeg item toe in blad

111

Insert in een 2-3-4 boom: splitsen van een 4-knoop

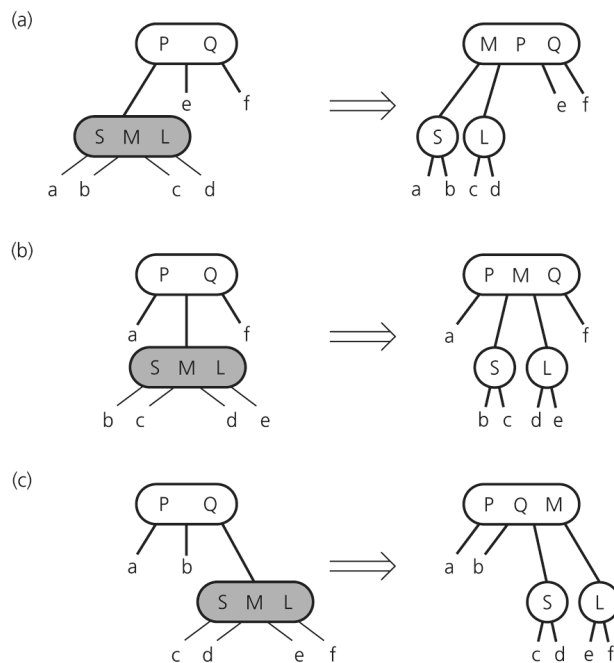
- ouder van de te splitsen knoop is een 2-knoop



112

Insert in een 2-3-4 boom: splitsen van een 4-knoop

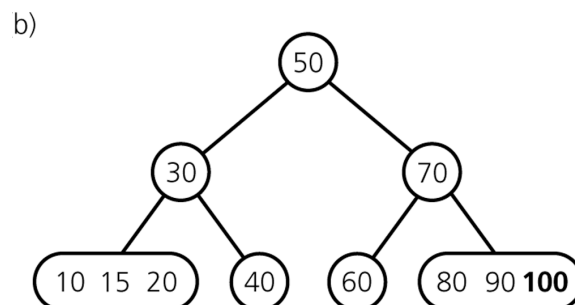
- ouder van de te splitsen knoop is een 3-knoop



113

Oefening

Voeg volgende items achtereenvolgens toe aan een lege 2-3-4 boom: 60, 30, 10, 20, 50, 40, 70, 80, 15, 90, 100. Teken alle tussenstappen. Bekom je uiteindelijk volgende 2-3-4 boom?



114

Delete uit een 2-3-4 boom, beter voorkomen dan genezen

- zoek de knoop n die het te verwijderen item bevat en:
 - **vorm elke 2-knoop (behalve de wortel) op dit pad om tot een 3-knoop of een 4-knoop**
- zoek de inorder successor van dit item en:
 - **vorm elke 2-knoop (behalve de wortel) op dit pad om tot een 3-knoop of een 4-knoop**
- het blad met de inorder successor is nu altijd een 3-knoop of een 4-knoop!
- swap item en inorder successor
- delete het item uit het blad – dit blad wordt niet leeg!
- de boom hoeft verder niet geherstructureerd te worden zoals bij de delete in een 2-3 boom

115

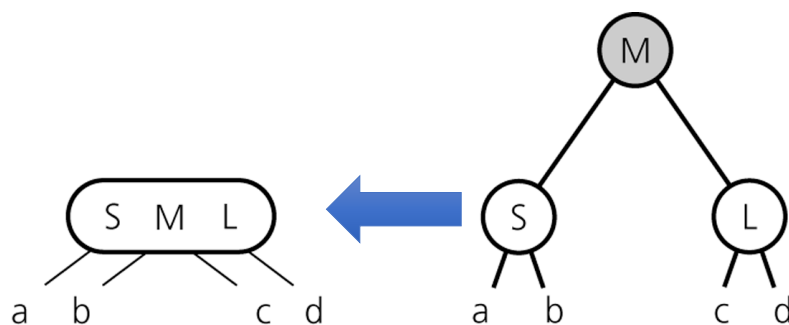
Omvormen van een 2-knoop tot een 3-knoop of een 4-knoop

Gebruik voor de delete

- ofwel herverdelen indien een gepaste sibling een item kan missen
- ofwel samenvoegen (merge): het omgekeerde van het splitsen in de insert

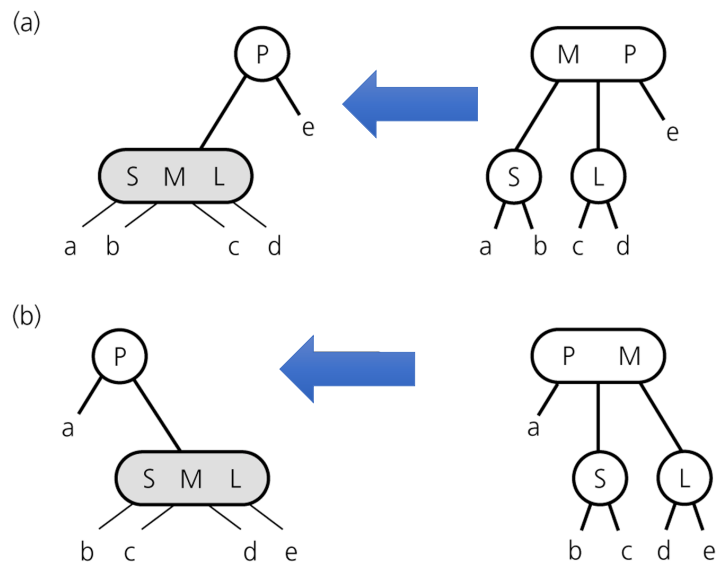
Samenvoegen, verschillende mogelijkheden

1. Als de ouder en sibling van de om te vormen knoop 2-knopen zijn:



116

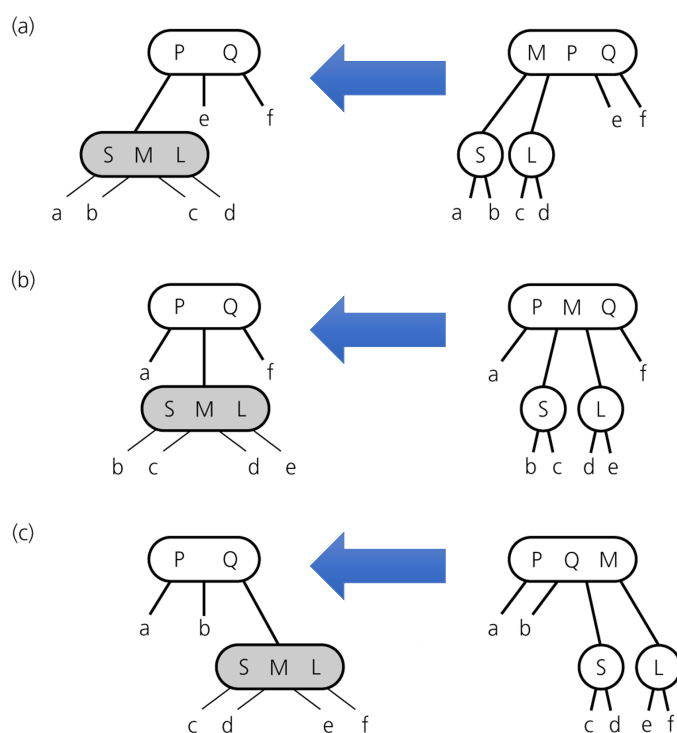
2. Als de ouder van de om te vormen knoop een 3-knoop is:



Let op: enkel sibling-buurtjes kunnen samengevoegd worden!

117

3. Als de ouder van de om te vormen knoop een 4-knoop is:



Let op: enkel sibling-buurtjes kunnen samengevoegd worden!

118

2-3-4 bomen: conclusies

- insert en delete algoritmen voor een 2-3-4 boom vragen minder stappen dan voor een 2-3 boom:
 - slechts 1 pad volgen tussen wortel en blad
- een 2-3-4 boom is altijd gebalanceerd
- een 2-3-4 boom gebruikt meer geheugen dan een binaire zoekboom
- zijn 2-3-4-5 bomen dan nog beter? Neen: knopen toelaten met meer items en kinderen geeft geen betere resultaten, behalve bij externe opslag (zie thema 23)

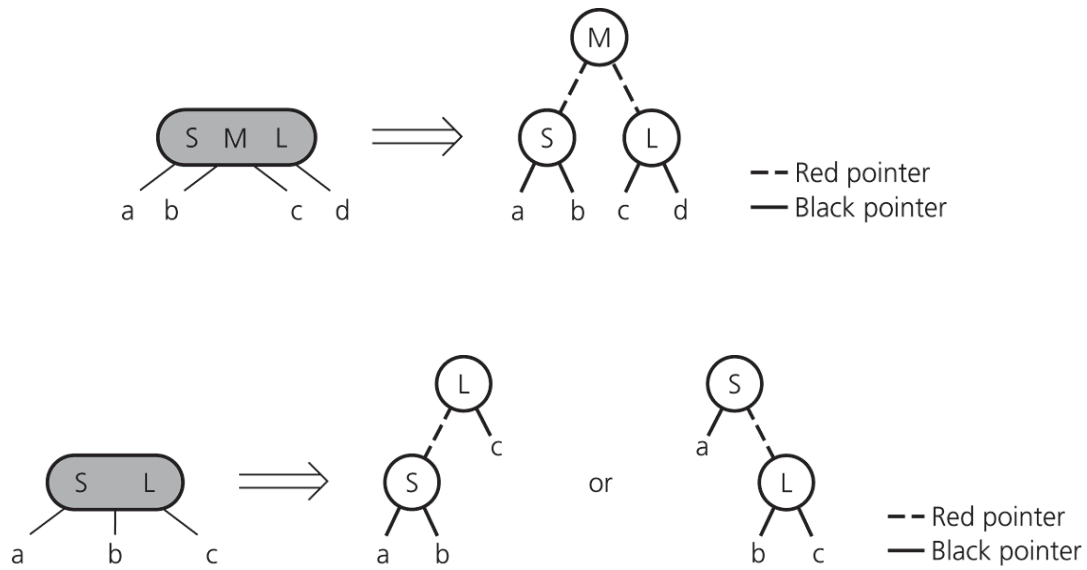
119

Rood-zwart bomen

- speciale binaire zoekbomen
- stellen 2-3-4 bomen voor
- dezelfde voordelen als 2-3-4 bomen *met minder geheugen!*
- stelt elke 3-knoop en 4-knoop in een 2-3-4 boom voor als een equivalente binaire boom

120

Rood-zwart bomen



121

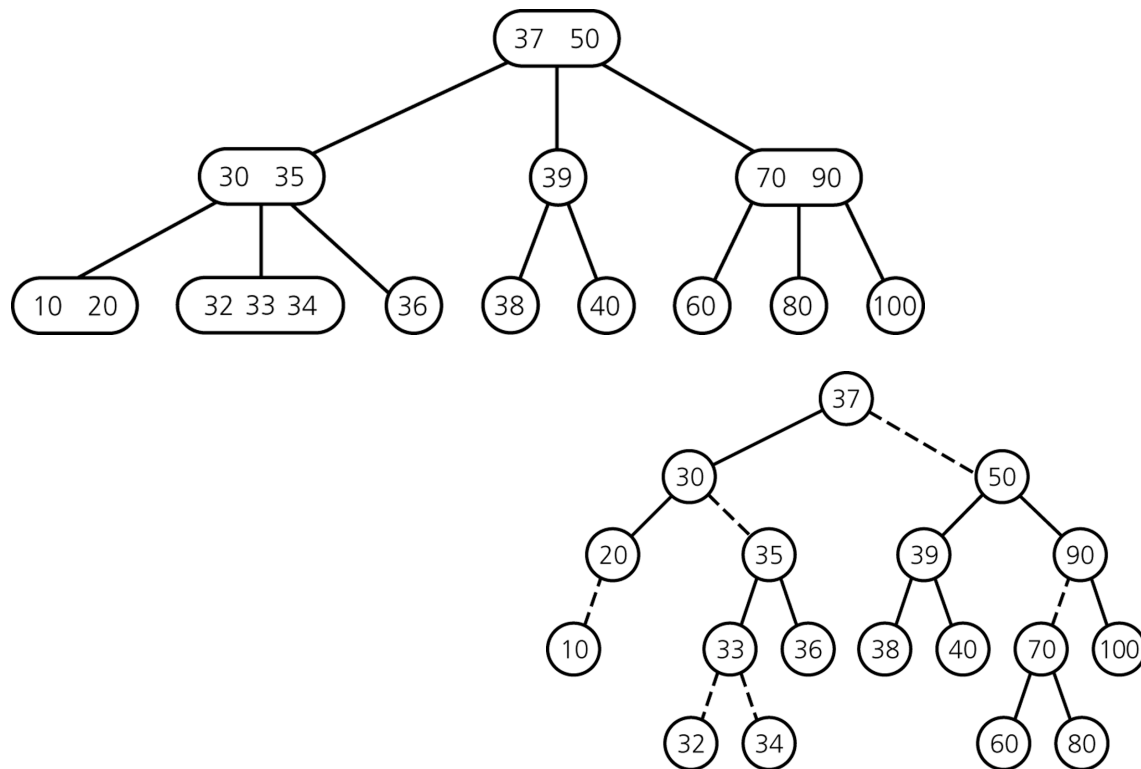
Rood-zwart bomen

Rode en zwarte pointers naar de kinderen

- de pointers die in de originele 2-3-4 boom ook aanwezig zijn
→ zwarte 'kinderpointers'
- de pointers die in de originele 2-3-4 boom niet aanwezig zijn, verbinden items die in de originele 2-3-4 boom in dezelfde knoop zitten
→ rode 'itempointers'

122

Voorbeeld. Een 2-3-4 boom en equivalente rood-zwart boom



123

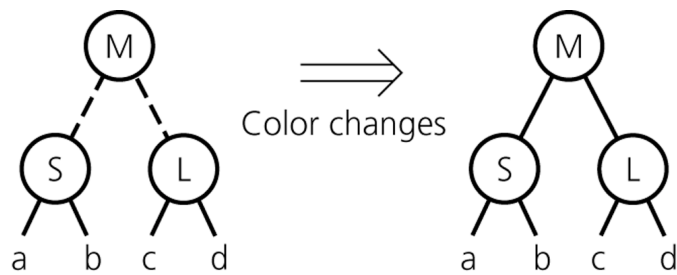
Rood-zwart bomen – bewerkingen

- zoeken en traverse in een rood-zwart boom
 - zelfde algoritme als voor een binaire zoekboom
- insert en delete
 - pas 2-3-4 algoritmen aan
 - bijvoorbeeld: splitsen van het equivalent van een 4-knoop in een rood-zwart boom
 - ofwel door eenvoudige kleurwijzigingen van de pointers
 - ofwel via andere aanpassingen aan de pointers (rotaties) die dan resulteren in een kleinere hoogte

124

Rood-zwart bomen

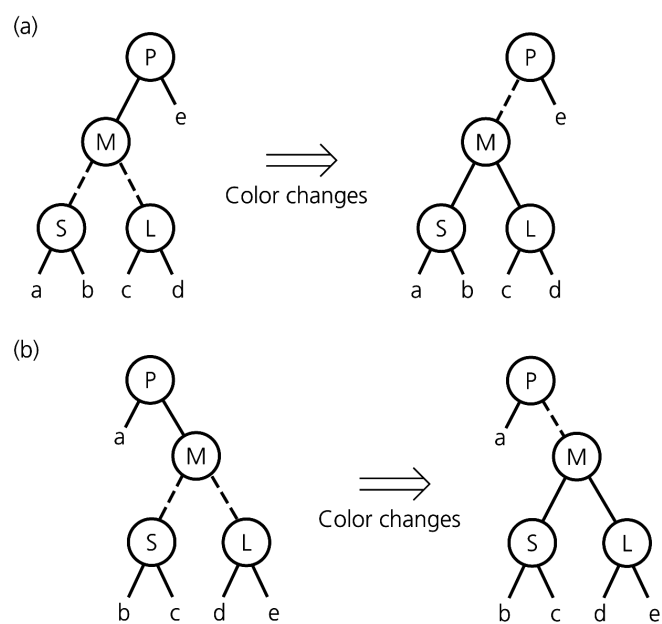
Het splitsen van een 4-knoop-wortel.



125

Rood-zwart bomen

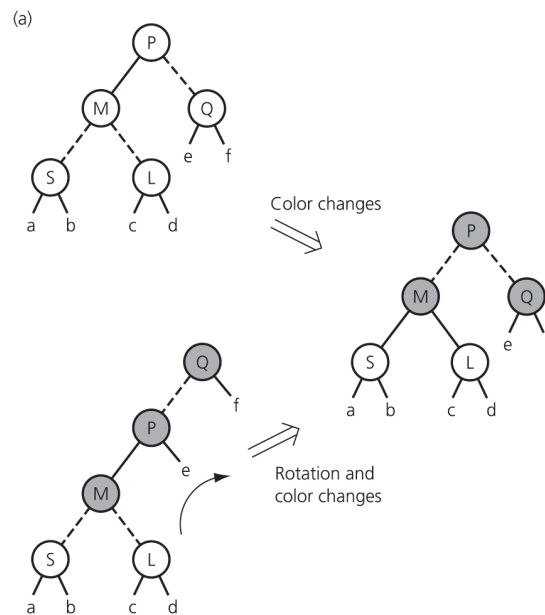
Het splitsen van een 4-knoop met een 2-knoop-ouder.



126

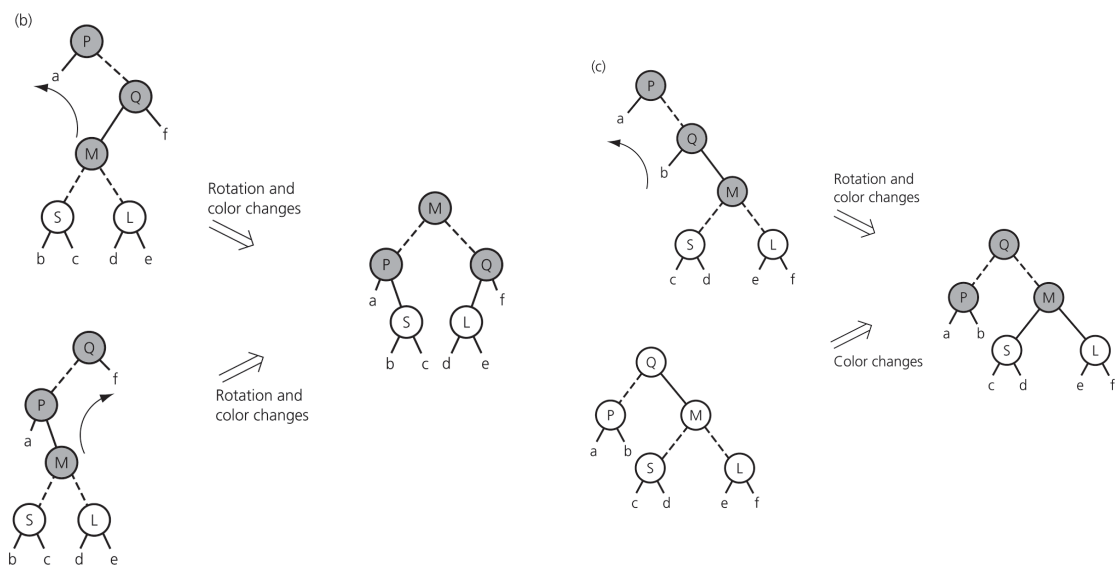
Rood-zwart bomen

Het splitsen van een 4-knoop met een 3-knoop-ouder.



127

Het splitsen van een 4-knoop met een 3-knoop-ouder (vervolg).



128

Gegevensabstractie en -structuren

Thema 8 Heaps en heapsort

129

Definitie: Een **heap** is een complete binaire boom T zodat

- T leeg is
of
- de zoeksleutel van de root r van T is groter dan of gelijk aan de zoeksleutels van de kinderen van r , *en*
- de deelbomen van r zijn heaps.

dus:

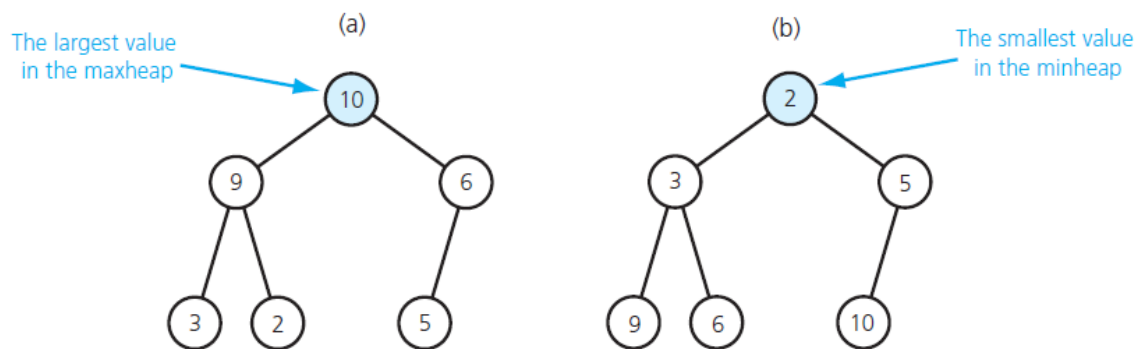
- In een heap heeft elke knoop een zoeksleutel groter dan of gelijk aan de zoeksleutels van de kinderen van de knoop.
- De root van de heap bevat het element met de grootste zoeksleutel.
- Er hoeft geen verband te zijn tussen de zoeksleutels van siblings.

Definitie: Een **semiheap** is een complete binaire boom T zodat

- T leeg is
of
- de deelbomen van de root r van T zijn heaps.

130

Maxheaps en minheaps



131

Specificaties van de bewerkingen van het ADT heap

// HeapItemType is het type van de elementen in de heap.

+createHeap()

+destroyHeap()

+heapIsEmpty():boolean {query}

+heapInsert(in newItem: HeapItemType, out success: boolean)

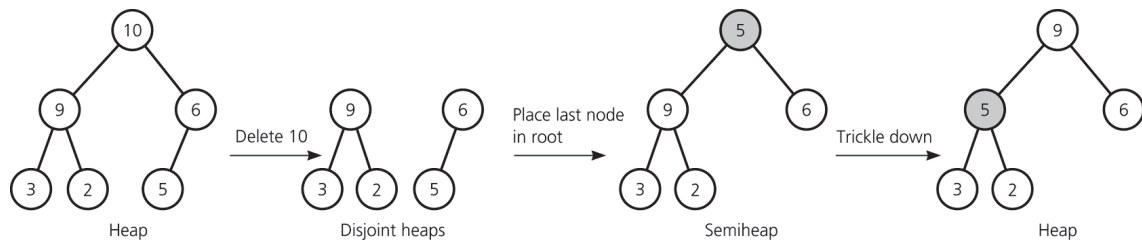
+heapDelete(out rootItem: HeapItemType, out success: boolean)

// Stopt het item uit de root in rootItem en verwijdert het dan uit de heap.

// Dit item heeft de grootste zoek sleutelwaarde. Success wordt false als de heap
// leeg is.

132

heapDelete

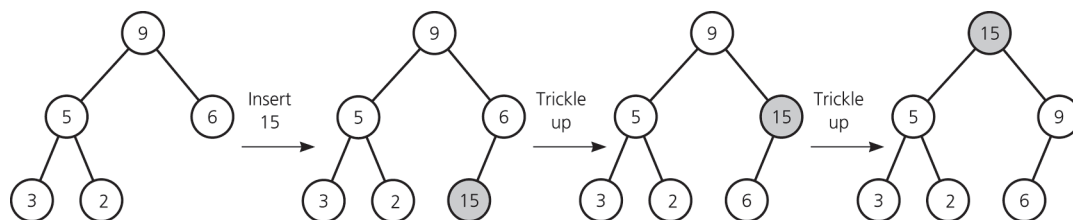


efficiëntie:

items worden verwisseld langs een pad dat in het slechtste geval van de root naar een blad gaat (trickle down) - het aantal verwisselingen (swaps) is dus hoogstens de hoogte van de boom - 1.

133

heapInsert

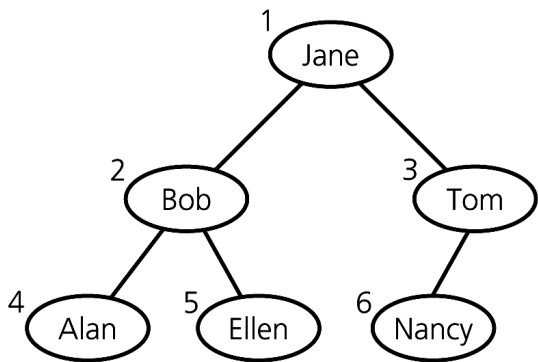


efficiëntie:

items worden verwisseld langs een pad dat in het slechtste geval van een blad naar de root gaat (trickle up) - het aantal verwisselingen (swaps) is dus hoogstens de hoogte van de boom - 1.

134

Array implementatie voor complete binaire bomen



0	Jane
1	Bob
2	Tom
3	Alan
4	Ellen
5	Nancy
6	
7	

array[i*2+1] = het linkerkind van de knoop in array[i]
array[i*2+2] = het rechterkind van de knoop in array[i]

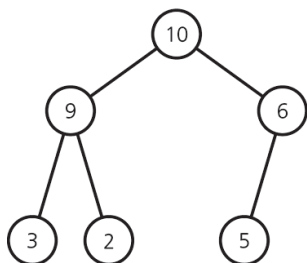
array[⌊ (i-1)/2 ⌋] = de ouder van de knoop in array[i]

Array implementatie voor heaps

Als de maximum grootte van de heap gekend is, kan gekozen worden voor de array implementatie van complete binaire bomen.

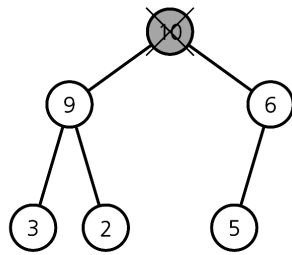
de heap wordt voorgesteld door:

- een array met de heap items, en
- een teller die het aantal elementen van de heap bijhoudt.



0	10
1	9
2	6
3	3
4	2
5	5

De delete bewerking voor de array implementatie van een heap

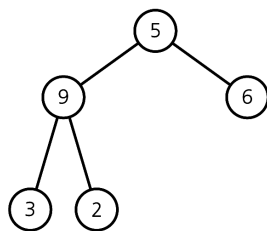


(a)

0	10
1	9
2	6
3	3
4	2
5	5

137

Delete (vervolg)



(b)

0	5
1	9
2	6
3	3
4	2

Hoe kunnen we van deze semiheap een heap maken?

→ heapRebuild

138

heapDelete (out rootItem: HeapItemType, out success: boolean)

```
// geef het item in de root weer  
rootItem := items[0];
```

```
// copieër het item van de laatste node naar de root  
items[0] := items[size - 1];
```

```
// verwijder de laatste node  
size := size - 1;
```

```
// maak van de semiheap terug een heap  
heapRebuild (items, 0, size)
```

139

heapRebuild (inout items: ArrayType, in root: integer, in size: integer)

// Bouwt een semiheap met wortel in index root om tot een heap.

```
// Haal recursief het item in index root naar beneden (= "trickle down") tot op zijn  
// juiste positie, door het te verwisselen met zijn grootste kind, als het kind  
// groter is dan het item. Als het item zich in een blad bevindt, gebeurt er niets.
```

```
if (de wortel is geen blad)
```

```
{ // wortel moet een linkerkind hebben  
  child := 2 * root + 1; // linkerkind index
```

```
  if (de wortel heeft een rechterkind)
```

```
  { rightChild := child + 1; // rechterkind index  
    if (items[rightChild].getKey() > items[child].getKey())  
      child := rightChild // index van het grootste kind  
  }
```

140

```

// als het item in de wortel een kleinere zoek sleutel waarde heeft dan die van
// het grootste kind, worden de items verwisseld.
if (items[root].getKey() < items[child].getKey())
{
    verwissel items[root] en items[child];

    // Bouw een semiheap met wortel child om tot een heap.
    heapRebuild(items, child, size)
}
}

```

141

```

heapInsert (in newItem: HeapItemType, out success: boolean)

// voeg newItem toe als laatste element van de heap
items[size] := newItem;

// haal newItem naar boven ("trickle up") tot op de juiste plaats in de boom
place := size;

// index van de parent van het item op index place
parent := (place - 1)/2;

while ((parent >= 0) and (items[place].getKey() > items[parent].getKey()))
{
    verwissel items[place] en items[parent];
    place := parent;
    parent := (place - 1)/2
}

size := size + 1

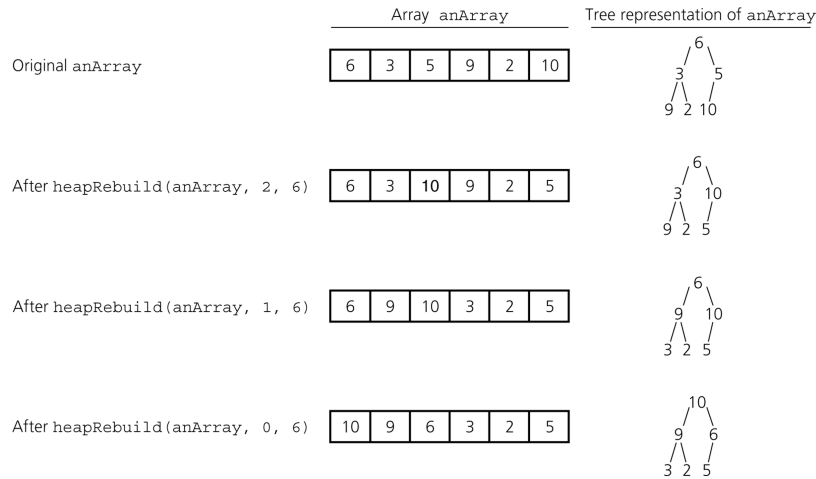
```

142

HeapSort

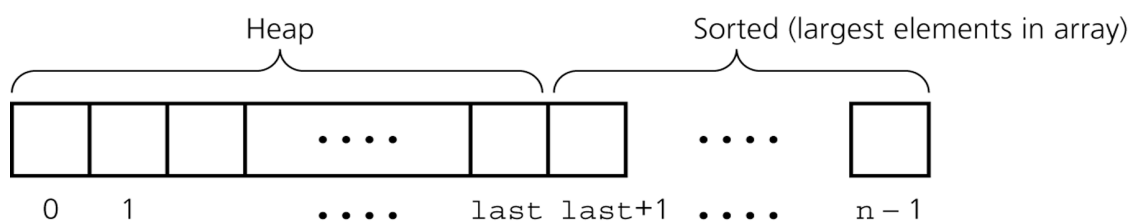
Sorteeralgoritme dat gebruik maakt van een heap om een array van items te sorteren.

1. Transformeer array in heap, gebruik makend van *heapRebuild*, waarmee semiheaps worden omgevormd tot heaps.



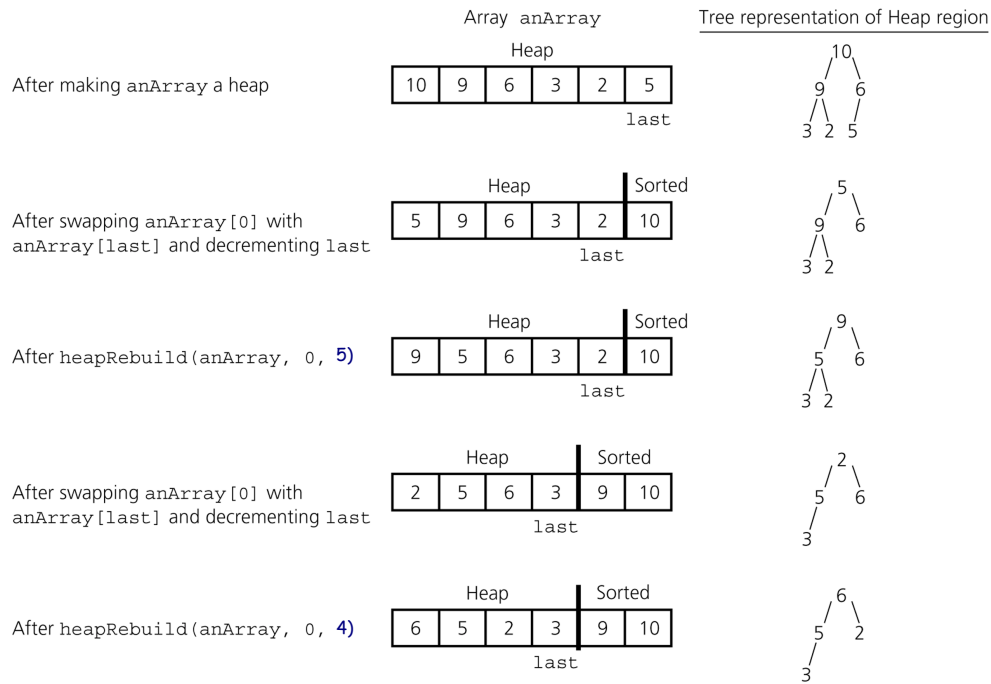
143

2. Verdeel array in twee gebieden:
 - i. het **heap gebied**, en
 - ii. het **gesorteerde gebied**.

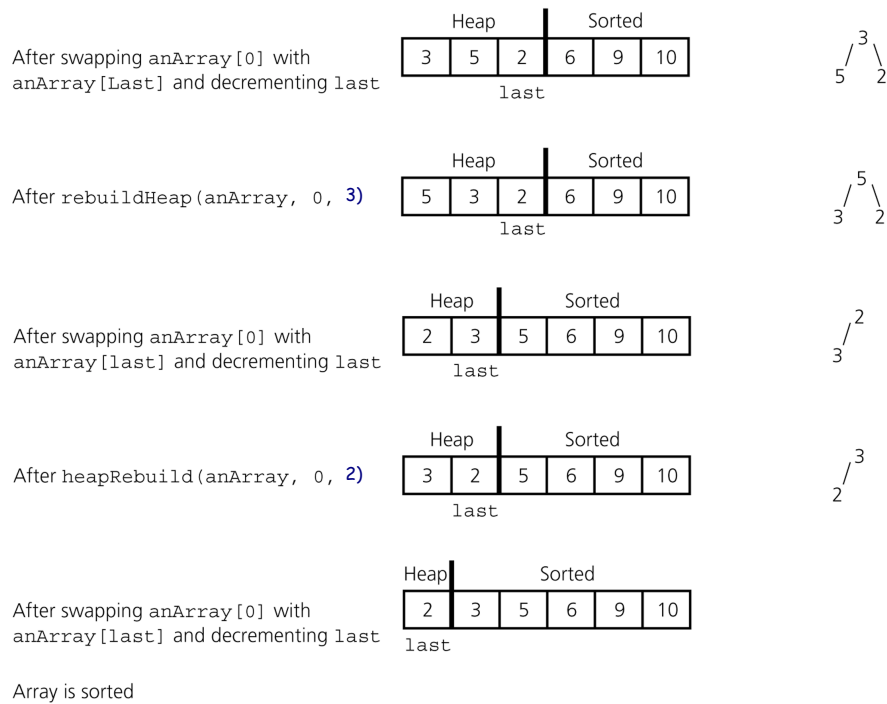


3. Bij elke stap:
 - i. verplaats een item van het heap gebied naar het gesorteerde gebied, nl.
 - ii. verwissel de root van de heap (het grootste element in het heap gebied) met het laatste element in de heap
 - iii. transformeer de verkregen semiheap terug tot een heap

144



145



146

De invariant hierbij is:

- Na stap k bevat het gesorteerde gebied de k grootste waarden. Het laatste element van de array is het grootste, het voorlaatste is het tweede grootste, ...
- De elementen in het heap gebied vormen een heap.

heapSort (inout anArray: ArrayType, in n: integer)

// sorteert anArray[0..n-1]

// vorm de array om tot een heap

for (index = n-1 down to 0)

{ // Invariant: boom met wortel index is een semiheap

heapRebuild(anArray, index, n)

// Assertion: boom met wortel index is een heap

}

// Assertion: anArray[0] is het grootste element in heap anArray[0 .. n-1]

147

// Initialiseer de gebieden

last := $n - 1$

// Invariant: anArray[0..last] is een heap, anArray[last+1..n-1] is gesorteerd en
// bevat de grootste elementen van anArray.

for (step = 1 to n)

{ // verplaats het grootste item uit het heap gebied naar het begin van het
// gesorteerde gebied.

verwissel anArray[0] en anArray[last];

// breid het gesorteerde gebied uit, verklein het heap gebied

last := last - 1;

// maak van het heap gebied opnieuw een heap

heapRebuild(anArray, 0, last+1)

}

148

Gegevensabstractie en -structuren

Thema 9 Hashing

149

Hashing

Gebalanceerde bomen geven een indrukwekkende efficiëntie: een item zoeken in een boom van 10.000 items gebeurt in $\log_2 10.000 \approx 13$ stappen!

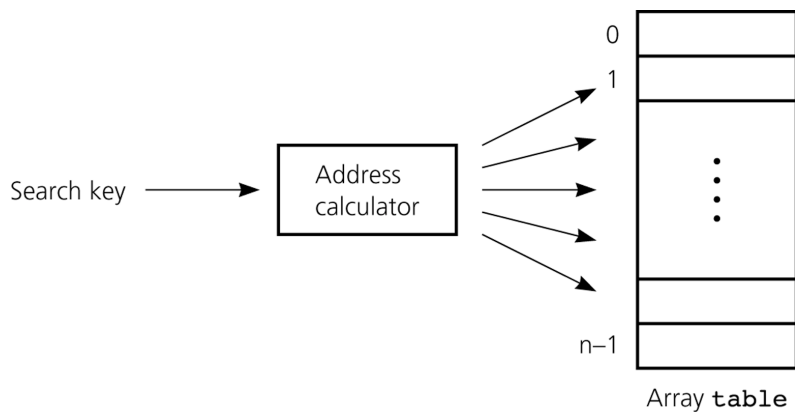
Maar ... sommige toepassingen vereisen een nog betere efficiëntie

- hulpdiensten: zoeken van adres via telefoonnummer
- luchtverkeersleider: zoeken van vluchtinformatie via vluchtnummer

→ bepaal onmiddellijk (zonder zoeken) de juiste plaats van een item m.b.v. een 'address calculator'.

150

Hashing



Een **hashfunctie** (address calculator) bepaalt de plaats van een item in een **hashtabel**.

→ hashtabelbewerkingen `tableInsert`, `tableRetrieve` en `tableDelete` zonder zoeken:

151

tableInsert (in `newItem:TableItemType`)

```
index := hash(searchKey)    // de index in de array, door de 'adres calculator'  
                                // berekend uit de zoeksleutel van newItem  
hashTable[index] := newItem
```

tableRetrieve (in `searchKey: KeyType`, out `tableItem:TableItemType`, out `success: boolean`)

```
index := hash(searchKey)    // de index in de array, door de 'adres calculator'  
                                // berekend uit searchKey  
  
if (hashTable[index].getKey() ≠ searchKey)  
    success := false  
else  
{   tableItem := hashTable [index]  
    success := true  
}
```

152

tableDelete(in searchKey: KeyType,
out success: boolean)

```
index := hash(searchKey)    // de index in de array, door de 'adres calculator'  
                             // berekend uit searchKey
```

```
if (hashTable[index].getKey() ≠ searchKey)  
    success := false  
else  
{   verwijder het item uit hashTable[index]  
    success := true  
}
```

Een perfecte hashfunctie beeldt elke zoeksleutel af op een *unieke* locatie in de hashtable.

Een perfecte hashfunctie is mogelijk indien alle zoeksleutels gekend zijn.

Bij de meeste hashfuncties treden er **botsingen** op: meerdere (>1) zoeksleutels worden op *dezelfde* locatie in de hashtable afgebeeld.

153

Voorbeeld:

hashtabel table[0..100] en **hashfunctie** h met

$$h(45567) = h(54321) = 22$$

→ volgens de hashfunctie dienen de items met zoeksleutels 45567 en 54321 beide op positie 22 van table terecht te komen (zelfs als de hashtable slechts enkele items bevat).

154

Hashfuncties

Een goede hashfunctie

- is snel en makkelijk te berekenen
- zorgt voor een gelijke verdeling van items over de hele hashtabel

hashfuncties voor integer-zoeksleutels volstaan - andere zoeksleutels kunnen omgezet worden:

- optellen van ASCII waarden
- optellen van letterwaarden (vb. 1 tot 26 voor a tot z)
→ niet uniek, vb. noot en toon
- concateneer de binaire letterwaarden (vb. 1 tot 11010 voor a tot z):
string 'toon' → binaire letterwaarden 10100.01111.01111.01110 → decimaal
getal 671214
- Horner's regel geeft zelfde resultaat via vereenvoudigde berekening:
string 'toon' → letterwaarden 20.15.15.14 → $((20 \cdot 32 + 15) \cdot 32 + 15) \cdot 32 + 14 =$
decimaal getal 671214

probleem: grote getallen kunnen overflow veroorzaken

→ combineer Horner's regel met modulo berekening na elke optelling

155

Modulo berekening

een eenvoudige en doeltreffende hashfunctie h voor een hashtabel met 'tableSize' posities:

$$h(x) := x \bmod \text{tableSize}$$

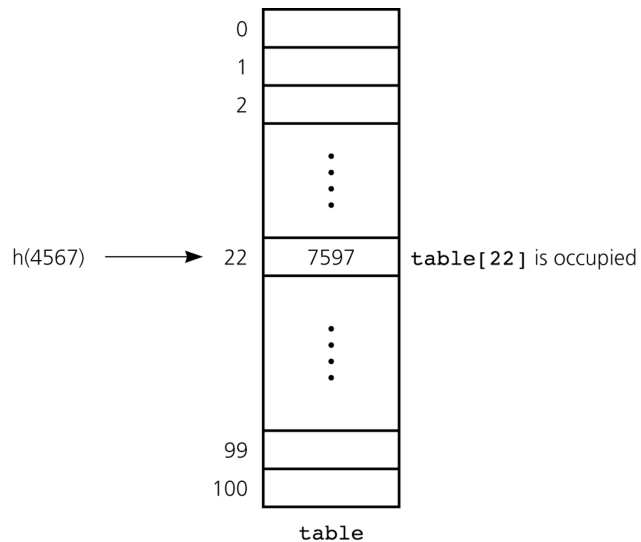
kies voor tableSize een **priemgetal** (vb. 101 ipv 100)

- de items worden dan gelijk verdeeld over de hele tabel
- het aantal botsingen wordt erdoor beperkt

156

Botsingen

Waar item 4567 plaatsen?



157

Er zijn 2 benaderingen voor de behandeling van botsingen:

1. **Open addressing:** neem een andere positie in de hashtabel
 - a. Linear probing
 - b. Quadratic probing
 - c. Double hashing
2. **Restructuring the hash table:** wijzig de structuur van de hashtabel zodat elke locatie $table[i]$ meer dan 1 item kan bevatten
 - a. Buckets
 - b. Separate chaining

158

Benadering 1: open addressing

Linear probing

Begin bij de locatie gegeven door de hashfunctie en zoek **sequentieel** naar een lege positie. De zoekreeks is de opeenvolging van locaties in de hashtabel die achtereenvolgens bezocht worden. Bij linear probing is dit:

table[h(searchKey)],
table[h(searchKey)+1],
table[h(searchKey)+2],
table[h(searchKey)+3], ...

Begin indien nodig terug vooraan in de hashtabel.

Bijv. de items op posities 22 tot 25 worden door de hashfunctie allen gehashed op 22.

	⋮	
22	7597	$h = 7597 \bmod 101 = 22$
23	4567	$h+1$
24	0628	$h+2$
25	3658	$h+3$
	⋮	

table

159

tableRetrieve volgt in de hashTabel dezelfde weg als tableInsert en stopt als het gewenste item of een lege positie gevonden is of als de hele tabel doorzocht is.

- tableDelete mag een positie niet zomaar leeg maken
- gebruik 3 toestanden: bezet, leeg en verwijderd

Probleem: items hebben de neiging zich te groeperen – er worden **clusters** gevormd in de hashtabel wat het zoeken inefficiënt maakt

160

Quadratic probing

Variatie op linear probing die de clustervorming vertraagt. De zoekreeks is

table [h(searchKey)],
table [h(searchKey)+1²],
table [h(searchKey)+2²],
table [h(searchKey)+3²], ...

	⋮	
22	7597	$h = 7597 \bmod 101 = 22$
23	4567	$h+1^2$
24		
25		
26	0628	$h+2^2$
	⋮	
31	3658	$h+3^2$
	⋮	

table

de items op posities 22, 23, 26 en 31 worden door de hashfunctie allen gehashed op 22.

Merk op dat voor alle items precies dezelfde locaties worden geprobeerd: de zoekreeks is onafhankelijk van de zoeksleutel.

161

Double hashing

Beperkt clustervorming drastisch door **zoeksleutelafhankelijke** zoekreeksen te gebruiken.

→ 2 hashfuncties bepalen samen de zoekreeks

- $h_1(\text{key})$ bepaalt de startpositie
- $h_2(\text{key})$ bepaalt de stapgrootte en heeft volgende eigenschappen:
 - $h_2(\text{key}) \neq 0$
 - $h_2 \neq h_1$ om clustervorming te voorkomen

alle zoekreeksen bezoeken *elke locatie* in de hashtabel als de tabelgrootte en de stapgrootte relatief priem (ggd=1) zijn

→ ok als tabelgrootte een priemgetal is want elk priemgetal is relatief priem tot alle kleinere getallen.

162

Voorbeeld double hashing

De grootte van de hashtabel is 11 en de primaire en secundaire hashfuncties zijn als volgt gedefinieerd:

$$h_1(\text{key}) = \text{key} \bmod 11$$

$$h_2(\text{key}) = 7 - (\text{key} \bmod 7)$$

Oefening

Bereken de zoekreeks voor 58, 14 en 91. Waar worden deze elementen achtereenvolgens toegevoegd in een lege tabel?

163

$$h_1(58) = 58 \bmod 11 = 3$$

$$h_2(58) = 7 - (58 \bmod 7) = 5$$

→ de zoekreeks voor 58 is

table[3], table[8], table[2], table[7],
table[1], table[6], table[0], table[5],
table[10], table[4], table[9]

$$h_1(14) = 14 \bmod 11 = 3$$

$$h_2(14) = 7 - (14 \bmod 7) = 7$$

→ de zoekreeks voor 14 is

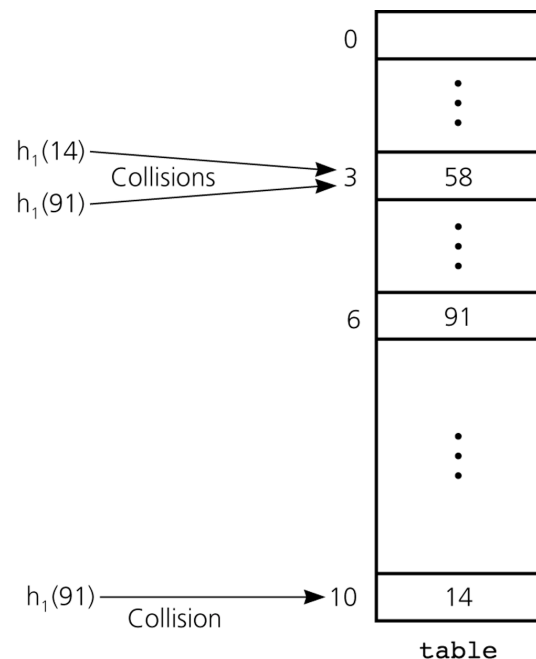
table[3], table[10], table[6], table[2],
table[9], table[5], table[1], table[8],
table[4], table[0], table[7]

$$h_1(91) = 91 \bmod 11 = 3$$

$$h_2(91) = 7 - (91 \bmod 7) = 7$$

→ de zoekreeks voor 91 is

table[3], table[10], table[6], table[2],
table[9], table[5], table[1], table[8],
table[4], table[0], table[7]



164

Benadering 2: restructuring the hash table

Elke locatie in de hashtable kan meerdere items bevatten

- buckets
- separate chaining

Buckets ~ arrays

Een bucket is een element uit de hashtable dat zelf een array is.

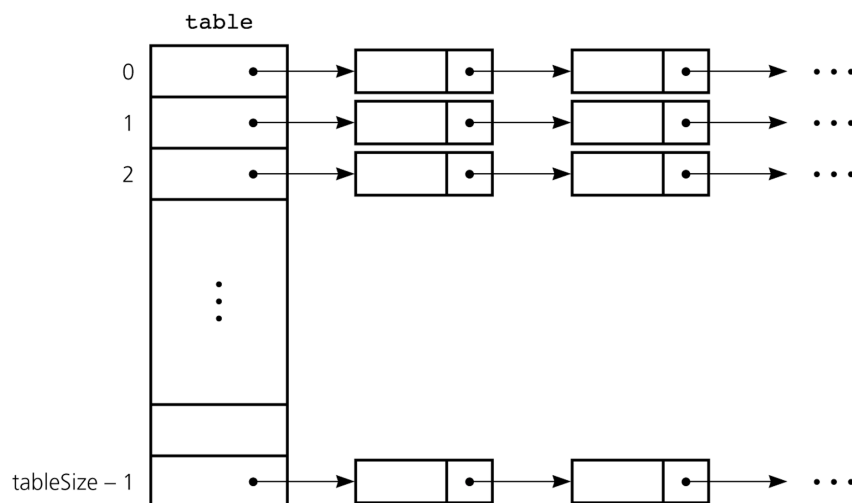
Alle items met hashwaarde i kunnen dan in de array $table[i]$ geplaatst worden.

Probleem: Welke bucket-grootte kiezen?

165

Separate chaining ~ kettingen

Elke locatie $table[i]$ in de hashtable is een pointer naar een gelinkte ketting die alle items met hashwaarde i bevat.



Each location of the hash table contains a pointer to a linked list

Met separate chaining is de grootte van het ADT tabel dynamisch (ook niet beperkt tot de grootte van de hashtable) want elke gelinkte ketting kan zo lang als nodig worden.

166

De efficiëntie van hashing

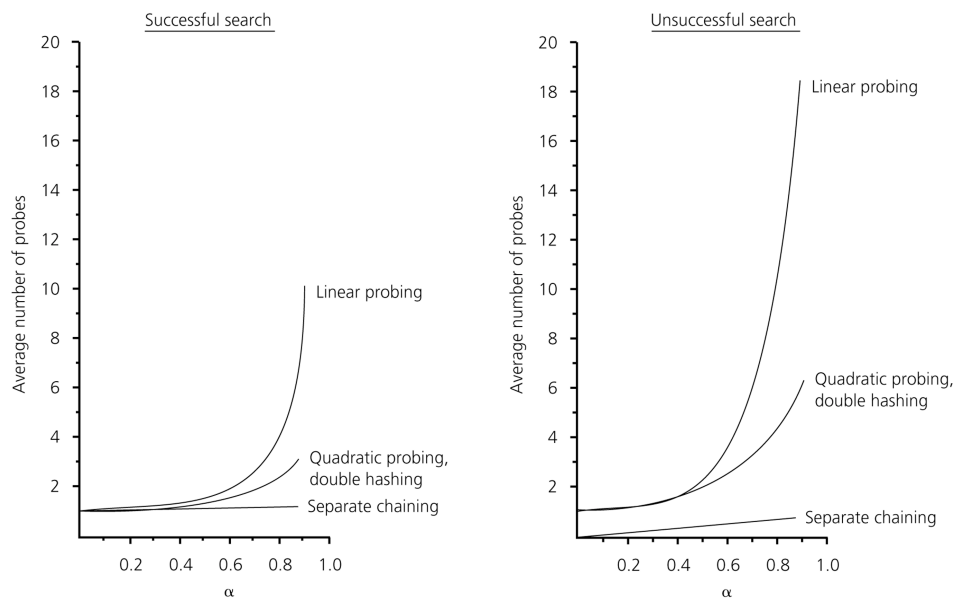
De 'load factor' meet hoe vol een hashtabel is:

$$\alpha = \text{aantal items in de tabel} / \text{tabelgrootte}$$

Hoe groter α , hoe kleiner de efficiëntie – α mag niet groter worden dan $2/3$.

167

Zoeken zonder succes neemt meestal meer tijd in beslag dan succesvol zoeken.



De relatieve efficiëntie van verschillende methodes voor de behandeling van botsingen:

- als de hashtabel half vol is ($\alpha = 0.5$) dan zijn de verschillende methodes ongeveer gelijkwaardig
- als de hashtabel voller wordt en α 1 nadert, is separate chaining het meest efficiënt

168

Opgelet: voor sommige toepassingen kan hashing in het **slechtste** geval veel trager zijn dan andere implementaties.

Voor veel toepassingen geeft hashing de meest efficiënte implementatie, maar ... **niet voor gesorteerd doorlopen** (vb. alfabetische volgorde):

er bestaat geen orderrelatie tussen de items in `table[i]` en in `table[i+1]`!

- om de tabel gesorteerd te doorlopen moeten de items eerst gesorteerd worden
- als sorteren vaak nodig is, is het veel beter een zoekboom te gebruiken

Ook volgende problemen zijn efficiënt op te lossen met zoekbomen en niet met hashing:

- zoek item met kleinste/grootste zoeksleutel
- range queries: vind alle items met zoeksleutel tussen 2 gegeven waarden

Gegevensabstractie en -structuren

Thema 10 Het ADT (gesorteerde) lijst intro UML

Het ADT lijst

Bewerkingencontract (zie ook Thema 4)

Specificatie van pré- en postcondities van enkele bewerkingen van het ADT lijst.

+isEmpty(): boolean {query}

// Bepaalt of een lijst leeg is.

// **Précondities:** geen.

// **Postcondities:** geeft true als de lijst leeg is en false als de lijst niet leeg is.

+getLength(): integer {query}

// Bepaalt de lengte (het aantal elementen) van de lijst.

// **Précondities:** geen.

// **Postcondities:** geeft het aantal elementen in de lijst.

171

+insert(in positie:integer, in newItem:ListItemType, out success:boolean)

// Voegt een element toe op een bepaalde positie in de lijst.

// **Précondities:** 'positie' duidt de plaats aan waar het element 'newItem' moet

// worden toegevoegd.

// **Postcondities:** Als het toevoegen lukte, dan zit 'newItem' op 'positie' in

// de lijst en is 'success' true. Als in de oorspronkelijke lijst

// positie <= getLength(), dan zijn de elementen als volgt hernummerd: het

// element op 'positie' naar 'positie+1', het element op 'positie+1' naar 'positie+2', enz..

// 'success' is false als het toevoegen niet lukte, bvb. als positie < 1 of

// positie > getLength()+1.

+delete(in positie:integer, out success:boolean)

// Verwijdert het element op een bepaalde positie uit de lijst.

// **Précondities:** 'positie' duidt de plaats aan van het element dat moet verwijderd

// worden.

// **Postcondities:** Als in de oorspronkelijke lijst 1 <= positie <= getLength(),

// dan is het element op 'positie' uit de lijst verwijderd en

// als positie < getLength(), dan zijn de elementen als volgt hernummerd: het

// element op 'positie+1' naar 'positie', het element op 'positie+2' naar 'positie+1', enz..

// 'success' is true als het verwijderen lukte en false als het niet lukte.

172

**+retrieve(in positie:integer, out dataltem:ListItemType, out success:boolean)
{query}**

// Vraagt het element op een bepaalde positie in de lijst op.

// **Précondities:** 'positie' duidt de plaats aan van het op te vragen element.

// **Postcondities:** Als in de oorspronkelijke lijst $1 \leq \text{positie} \leq \text{getLength}()$, dan

// bevat 'dataltem' het element van de lijst op 'positie'.

// De lijst blijft onveranderd bij deze bewerking.

// 'Success' is true als het opvragen lukte en false als het niet lukte.

173

UML (unified modeling language) syntax voor bewerkingen in een bewerkingencontract (meer info Appendix handboek)

visibility name(**parameter-list**): return-type {property-list}

met

- **parameter-list** bevat parameters, door komma's gescheiden, met als syntax

direction name: type = defaultValue

met

- o *direction* is **in** (input), **out** (output) of **inout** (input en output)
- o *name* is de naam van de parameter
- o *type* is het type van de parameter
- o *defaultValue* is een waarde die voor de parameter zal gebruikt worden indien deze niet gegeven is

vb: +insert(in index:integer, in newItem:ListItemType, out success:boolean)

174

visibility name(parameter-list): return-type {property-list}

- *visibility* is + (public), – (private) of # (protected)
- *name* is de naam van de bewerking
- *return-type* is het type van het resultaat van de bewerking (enkel vermeld indien de bewerking een waarde teruggeeft)
- *property-list* geeft eigenschappen van de bewerking weer, bvb. *query* duidt aan dat de bewerking geen gegevens wijzigt
- enkel *name* moet vermeld worden, de rest is optioneel

vb: **+getLength(): integer {query}**

175

Het ADT gesorteerde lijst

- houdt de elementen gesorteerd
- voegt elementen toe en verwijdert elementen volgens hun **waarde**, niet volgens hun positie

Specificatie (bewerkingencontract) van de ADT bewerkingen van het ADT gesorteerde lijst

// ListItemType is het type van de elementen in de lijst.

+sortedCreateList()

// Creëert een lege gesorteerde lijst.

+sortedDestroyList()

// Wist een gesorteerde lijst.

+sortedIsEmpty(): boolean {query}

// Bepaalt of een gesorteerde lijst leeg is.

+sortedGetLength(): integer {query}

// Geeft het aantal elementen in de gesorteerde lijst.

176

+sortedInsert(in newItem:ListItemType, out success:boolean)

// Voegt het element 'newItem' toe op de juiste plaats in een gesorteerde lijst.
// 'Success' duidt aan of het toevoegen gelukt is.

+sortedRemove(in anItem: ListItemType, out success:boolean)

// Verwijdert het element 'anItem' uit een gesorteerde lijst. 'Success' duidt
// aan of het verwijderen gelukt is.

+sortedRetrieve(in index:integer,out dataItem:ListItemType, out success:boolean) {query}

// Plaatst het element op positie 'index' van een gesorteerde lijst in 'dataItem', indien
// $1 \leq \text{index} \leq \text{sortedGetLength}()$. De lijst blijft onveranderd.
// 'Success' duidt aan of het opvragen gelukt is.

+locatePosition(in anItem:ListItemType, out isPresent:boolean): integer {query}

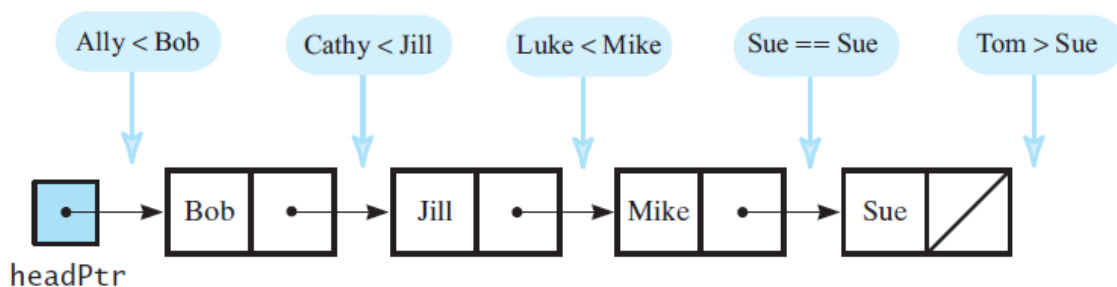
// Geeft de positie waar 'anItem' zit of zou zitten in een gesorteerde lijst.
// 'isPresent' duidt aan of 'anItem' aanwezig is in de gesorteerde lijst.
// 'anItem' en de lijst blijven onveranderd.

177

Implementaties van het ADT gesorteerde lijst

- Een array implementatie – vaste grootte
- Een ketting implementatie – dynamische grootte
- Een implementatie mbv het ADT Lijst - de gebruiker beheert de volgorde in de lijst!
hergebruik van reeds geïmplementeerde klasse bespaart tijd

Een ketting implementatie van het ADT gesorteerde lijst



178

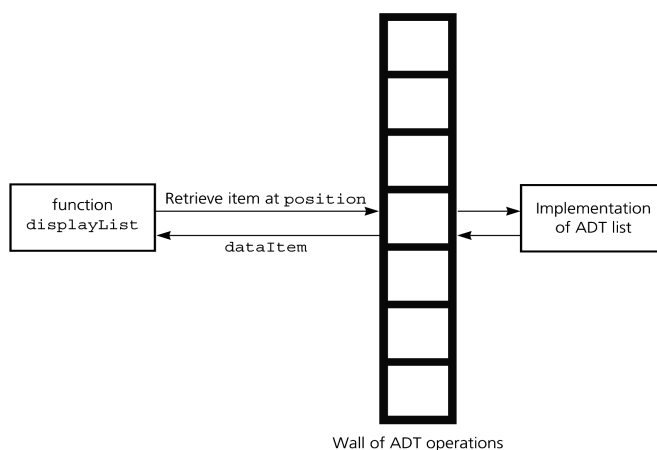
Gegevensabstractie en -structuren

Thema 11 Implementatie van ADT's

179

ADT bewerkingen verzorgen de interface met de gegevensstructuren

```
displayList(in aList:List)
// Toont de elementen uit de lijst aList.
for (position = 1 to aList.getLength())
{   aList.retrieve(position, dataItem, success)
    display dataItem
}
```

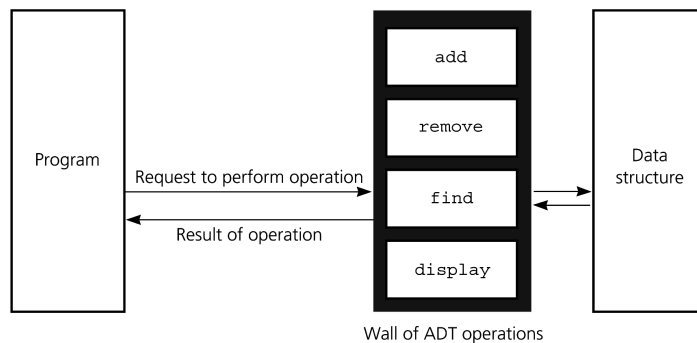


180

ADTs implementeren – object-georiënteerde talen, behalve Python (Python is dus een echt buitenbeentje!)

De gegevensstructuren worden verstopt achter een ‘muur’ van ADT bewerkingen, het programma dat het ADT gebruikt ziet

- enkel een ‘muur’ van beschikbare ADT bewerkingen
- geen gegevensstructuren en implementatie van algoritmen



Object-georiënteerde talen

- dwingen om de ADT-muur te respecteren
- verhinderen elke toegang tot gegevensstructuren zonder gebruik van de ADT bewerkingen

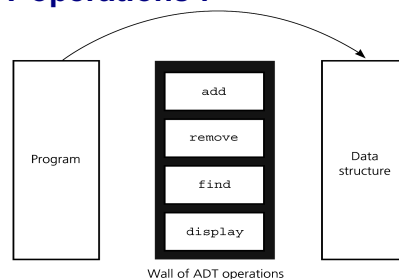
181

ADTs implementeren – niet-object-georiënteerde talen en Python!

- bij niet-object-georiënteerde talen kan je wél over de ADT-muur kijken
- er is **programmeerdiscipline** nodig om de ADT-muur te respecteren:
 - gebruik enkel de ADT-bewerkingen om de gegevensstructuren te bereiken en bewerken!
 - vb: gebruik `aList.retrieve(1, firstItem, succes)` en NIET `firstItem = items[0]`
 - verhoogt de leesbaarheid van de implementatie en
 - voorkomt veel fouten en aanpassingen als de implementatie van het ADT verandert
 - vb. bij verandering datastructuur: aanpassingen blijven overzichtelijk en vooral lokaal, niet het hele programma dient aangepast te worden

- **zware overtreding - ‘Violating the wall of ADT operations’:**

```
displayList2(L)  
current = L.head  
while (current <> null) do  
{ display current->data  
  current = current->next  
}
```



182

Gegevensabstractie en -structuren

Thema 12 ADT stack

183

Stacks - Stapels

De specificaties van een ADT ontstaan vaak tijdens het ontwerp van een oplossing

voorbeeld: intypen van tekst, met correctie via backspace (←)

abcc ← ddde ← ← ← ef ← fg geeft input abcdefg

Hoe kunnen we de originele lijn lezen en de juiste input bepalen?

Hoe de inputlijn opslaan?

Volgens de ADT benadering dienen we deze keuze uit te stellen tot we weten welke bewerkingen we op de gegevens nodig hebben!

184

Het read-and-correct algoritme

Volgende oplossing plaatst de correcte input in *theADT*, die de inputlijn voorstelt.

```
// read the line, correcting mistakes along the way
while (not end of line)
{   read a new character ch
    if (ch is not ' ← ')
        add ch to theADT
    else if (theADT is not empty)
        remove from theADT the item that was added most recently
    else
        ignore the ' ← '
} // end while
```

- ➔ 3 ADT bewerkingen:
- voeg een nieuw element toe
 - verwijder het laatst toegevoegde element
 - bepaal of het leeg is

185

Afdrukken in omgekeerde volgorde

```
// write the line in reversed order
while (theADT is not empty)
{   retrieve from theADT the item ch that was added most recently
    write ch
    remove from theADT the item that was added most recently
} // end while
```

- ➔ nog een 4^{de} ADT bewerking
- vraag het laatst toegevoegde element op

186

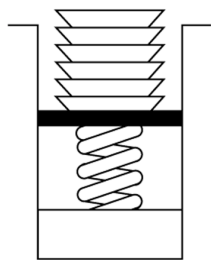
Het ADT stack

De ADT gegevens

- niet noodzakelijk verschillend
- van hetzelfde type
- geordend volgens moment van toevoegen

187

Het ADT stack



LIFO
Last In First Out

Stack
<i>top</i> <i>items</i>
<i>createStack()</i> <i>destroyStack()</i> <i>isEmpty()</i> <i>push()</i> <i>pop()</i> <i>getTop()</i>

De ADT bewerkingen:

1. **createStack** creëert een lege stack
2. **destroyStack** wist een stack
3. **isEmpty** bepaalt of een stack leeg is
4. **push** voegt een element toe op de top van de stack
5. **pop** verwijdert het laatst toegevoegde element uit de stack (de top van de stack)
6. **getTop** vraagt het laatst toegevoegde element uit de stack op (de top van de stack)

Het laatst toegevoegde element zal als eerste opgevraagd of verwijderd worden.

188

Bewerkingencontract van het ADT stack

// StackItemType is het type van de elementen in de stack.

+createStack()

// Creëert een lege stack.

+destroyStack()

// Wist een stack.

+isEmpty(): boolean {query}

// Bepaalt of een stack leeg is.

189

+push(in newItem:StackItemType, out success:boolean)

// Voegt het element 'newItem' toe op de top van een stack.

// 'success' duidt aan of het toevoegen gelukt is.

+pop(out success:boolean)

// Verwijdert de top van een stack, d.i. het laatst toegevoegde element.

// 'success' duidt aan of het verwijderen gelukt is.

+pop(out stackTop:StackItemType, out success:boolean)

// Plaatst de top van een stack (d.i. het laatst toegevoegde element) in

// 'stackTop' en verwijdert dan deze top.

// 'success' duidt aan of het verwijderen gelukt is.

+getTop(out stackTop:StackItemType, out success:boolean) {query}

// Plaatst de top van een stack (d.i. het laatst toegevoegde element) in

// 'stackTop' en laat de stack ongewijzigd. 'success' duidt aan of het gelukt is.

190

Dit ADT contract volstaat om het ADT te kunnen gebruiken – we hoeven de implementatie niet te kennen:

```
+readAndCorrect(out aStack:Stack)  
// Leest de input. Plaatst elk symbool verschillend van ' ← ' in de stack 'aStack'.  
// Bij het lezen van ' ← ' wordt de top van 'aStack' verwijderd.
```

```
aStack.createStack()  
read newChar  
while (newchar is not the end-of-line symbol)  
  { if (newchar is not ' ← ')  
    aStack.push(newChar)  
    else if (not aStack.isEmpty())  
      aStack.pop()  
  
    read newChar  
  } // end while
```

191

```
+displayBackward(in aStack:Stack)  
// Schrijft de inhoud van de stack 'aStack' in omgekeerde volgorde.
```

```
while (not aStack.isEmpty())  
  { aStack.pop(newChar)  
    write newChar  
  } // end while
```

Het volgende schrijft dan de input in omgekeerde volgorde:

```
ReadAndCorrect(inputStack)  
DisplayBackward(inputStack)
```

192

Oefening:

Bovenstaande specificaties laten een aantal vragen onbeantwoord, bvb. wat gebeurt er indien je pop en getTop op een lege stack laat werken? Werk de specificaties van de ADT Stack bewerkingen in detail uit, met pré- en postcondities.

193

Een eenvoudige toepassing: het checken van haakjes

abc{defgh{ijk}{l{mn}}op}qr heeft gebalanceerde haakjes

abc{def}{ghij{kl}mnopqr heeft geen gebalanceerde haakjes

Voorwaarden voor gebalanceerde haakjes:

- 1 Telkens je een '}' leest, komt dit overeen met een eerder gelezen '{'.
- 2 Als de hele string gelezen is, heb je voor elk '{' een '}' gelezen.

Pseudocode:

```
while (not end of the string)
{   if (the next character is a '{')
    aStack.push('{')
    else if (the character is a '}')
    aStack.pop()
}   // end while
```

194

Input string	Stack as algorithm executes			
	1.	2.	3.	4.
{a{b}c}	{	{ {	}	
	1. push "{ " 2. push "{ " 3. pop 4. pop Stack empty $\Rightarrow$ balanced			
{a{bc}	{	{ {	}	
	1. push "{ " 2. push "{ " 3. pop Stack not empty $\Rightarrow$ not balanced			
{ab}c}	{			
	1. push "{ " 2. pop Stack empty when last "}" encountered $\Rightarrow$ not balanced			

195

Gedetailleerde pseudocode:

```

aStack.createStack()
balancedSoFar = true
i = 1
while (balancedSoFar and i ≤ length of aString)
{
  ch = character at position i in aString
  i = i+1
  if (ch is '{')    // push an open brace
    aStack.push('{')
  else if (ch is '}') // close brace
    if (not aStack.isEmpty())
      aStack.pop()    // pop a matching open brace
    else
      balancedSoFar = false // no matching open brace

  // else: ignore all characters other than braces
}
// end while

if (balancedSoFar and aStack.isEmpty())
  aString has balanced braces
else
  aString does not have balanced braces

```

196

Het herkennen van strings in een taal

$L = \{ w\$w' : w \text{ is een string van karakters verschillend van } \$ \text{ en } w' = \text{reverse}(w) \}$

Een oplossing met stacks:

- doorloop het eerste deel w van de string en push elk karakter op de stack
- lees $\$$
- doorloop de tweede helft w' van de string en vergelijk elk karakter met het gepopte karakter van de stack

197

Toepassing: algebraïsche uitdrukkingen

Infix uitdrukkingen: de binaire bewerking staat tussen de operanden

$a + b$
 $a + b * c$
 $(a + b) * c$

Prefix uitdrukkingen: de bewerking staat voor de operanden

$+ a b$
 $+ a * b c$
 $* + a b c$

Postfix uitdrukkingen: de bewerking staat na de operanden

$a b +$
 $a b c * +$
 $a b + c *$

Prefix en postfix uitdrukkingen hebben geen volgorde van bewerkingen nodig en ook geen haakjes!

198

Om een infix uitdrukking uit te rekenen:

1. zet de infix uitdrukking om in postfix vorm
2. reken de postfix uitdrukking uit

We nemen 4 binaire bewerkingen: $*$, $/$, $+$ en $-$

Stap 2: een postfix uitdrukking uitrekenen

lees de uitdrukking van links naar rechts

voor elke operand: push de operand

voor elke bewerking: pas de bewerking toe op de 2 bovenste stackelementen:

- 2 keer pop en
- push van het resultaat van de bewerking

```
for (each character ch in the string)
{   if (ch is an operand)
    push value that operand ch represents onto stack
    else    // ch is an operator named op
    {   // evaluate and push the result
        operand2 = top of stack
        pop the stack
        operand1 = top of stack
        pop the stack
        result = operand1 op operand2
        push result onto stack
    }   // end if
}   // end for
```


Voorbeeld: $2 * (3 + 4)$ infix
 2 3 4 + * postfix

Key entered	Calculator action	After stack operation: Stack (bottom to top)
2	push 2	2
3	push 3	2 3
4	push 4	2 3 4
+	operand2 = pop stack (4)	2 3
	operand1 = pop stack (3)	2
	result = operand1 + operand2 (7)	2
	push result	2 7
*	operand2 = pop stack (7)	2
	operand1 = pop stack (2)	
	result = operand1 * operand2 (14)	
	push result	14

201

Stap 1: van infix naar postfix vorm

- de bewerkingen verschuiven naar rechts t.o.v. de operanden
- alle haakjes verdwijnen

for (each character ch in the infix expression)

```
{  switch (ch)
    {  case operand:  // append operand to end of PE
        postfixExp = postfixExp . ch
      case '(':       // save '(' on stack
        aStack.push(ch)
      case ')':       // pop stack until matching '('
        while (top of stack is not '(')
        {  postfixExp = postfixExp . (top of aStack)
            aStack.pop()
        }  // end while
        aStack.pop()  // remove '('
```

202

```

    case operator: // process stack operators of greater precedence
        while ( not aStack.isEmpty() and
                top of aStack is not '(' and
                precedence(ch) <= precedence(top of aStack))
        { postfixExp = postfixExp . (top of aStack)
          aStack.pop()
        } // end while
        aStack.push(ch) // save new operator
    } // end switch
} // end for
// append to postfixExp the operators remaining in the stack
while (not aStack.isEmpty())
{ postfixExp = postfixExp . (top of aStack)
  aStack.pop()
} // end while

```

203

Voorbeeld: zet $a - (b + c * d) / e$ om naar postfix-vorm

<u>ch</u>	<u>Stack (bottom to top)</u>	<u>postfixExp</u>	
a		a	
-	-	a	
(	-(	a	
b	-(	ab	
+	-(+	ab	
c	-(+	abc	
*	-(+ *	abc	
d	-(+ *	abcd	
)	-(+	abcd*	Move operators from stack to postfixExp until "("
	-(	abcd*+	
	-	abcd*+	
/	-/	abcd*+	
e	-/	abcd*+e	Copy operators from stack to postfixExp
		abcd*+e/-	

Oefening: zet $a / (b * c + d) - e$ om naar postfix-vorm

204

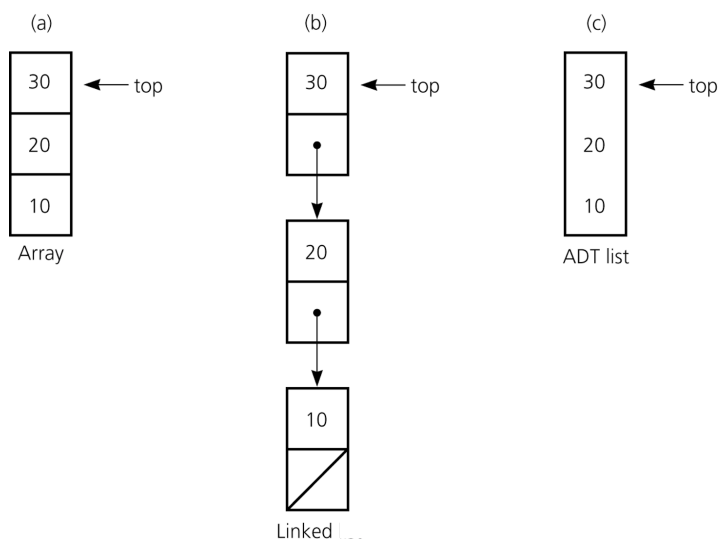
Gegevensabstractie en -structuren

Thema 13 Stackimplementaties

205

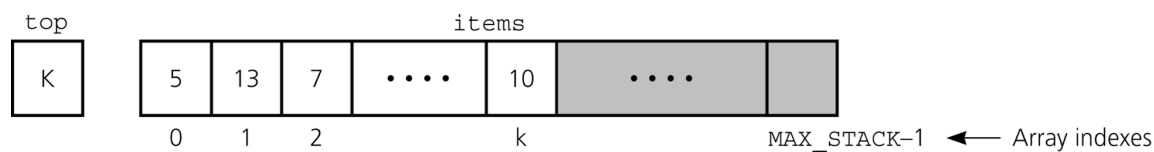
Implementaties van het ADT Stack

- a. Een array implementatie, vaste grootte
- b. Een ketting implementatie, dynamische grootte
- c. Een implementatie mbv het ADT Lijst, **hergebruik van een reeds geïmplementeerde klasse bespaart tijd**

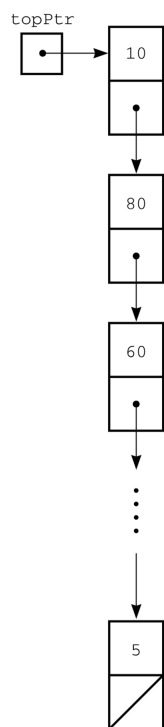


206

a. Een array implementatie van het ADT Stack



b. Een ketting implementatie van het ADT Stack



c. Een implementatie van het ADT Stack mbv het ADT Lijst

```
Class Stack {public: ...;    private: List aList;// list of stack items }
```

```
+isEmpty(): boolean
```

```
    return aList.isEmpty()
```

```
+push(in newItem:StackItemType, out success:boolean)
```

```
    aList.insert(1, newItem, success)    // ADT lijst
```

```
+pop(out success:boolean)
```

```
    aList.remove(1, success)    // ADT lijst
```

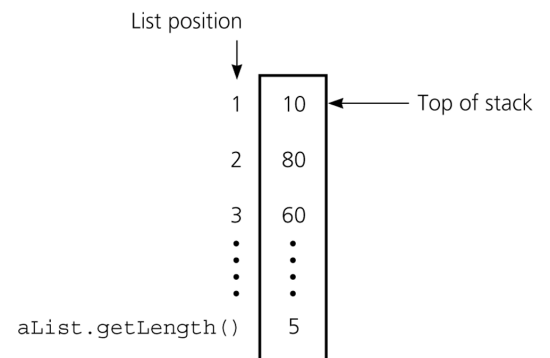
```
+pop(out stackTop:StackItemType, out success:boolean)
```

```
    aList.retrieve(1, stackTop, success)    // ADT lijst
```

```
    aList.remove(1, success)    // ADT lijst
```

```
+getTop(out stackTop:StackItemType, out success:boolean)
```

```
    aList.retrieve(1, stackTop, success)    // ADT lijst
```



209

Gegevensabstractie en -structuren

Thema 14 Boomimplementaties

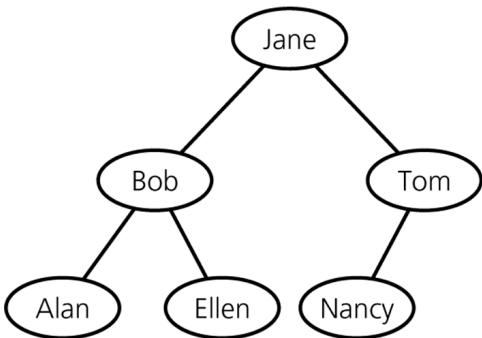
210

Mogelijke implementaties van een binaire boom

- Een array implementatie voor binaire bomen
- Een ketting implementatie voor binaire bomen
- Een array implementatie voor **complete** binaire bomen (zie heaps)

Een array implementatie voor binaire bomen

(a)

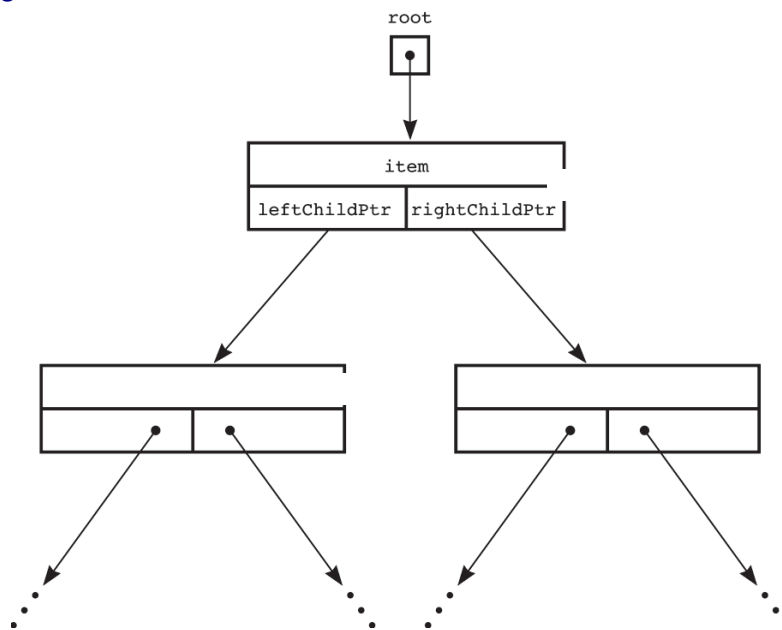


(b)

tree				root
	item	leftChild	rightChild	
0	Jane	1	2	0
1	Bob	3	4	free
2	Tom	5	-1	
3	Alan	-1	-1	6
4	Ellen	-1	-1	
5	Nancy	-1	-1	
6	?	-1	7	Free list
7	?	-1	8	
8	?	-1	9	
•	•	•	•	
•	•	•	•	
•	•	•	•	

Een ketting implementatie van binaire bomen

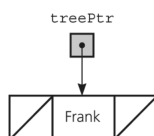
Algemeen



213

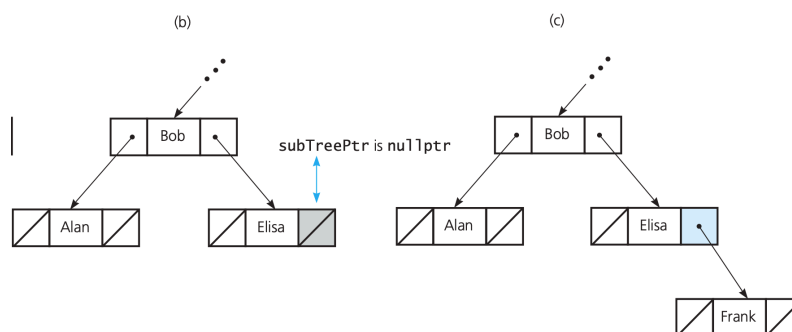
Voorbeeld

- insert Frank in een lege boom (a)



(a)

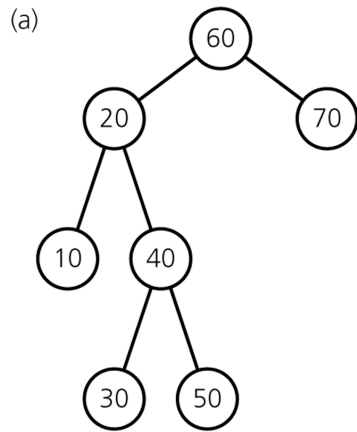
- insert Frank in een niet-lege boom
 - zoeken naar Frank (b)
 - insert in nieuw blad (c)



214

Opslag van een binaire zoekboom in een file

1. opslaan en terug inlezen in z'n **oorspronkelijke vorm**
 - gebruik preorder traversal om de boom weg te schrijven
 - gebruik `searchTreeInsert` om de items achtereenvolgens toe te voegen

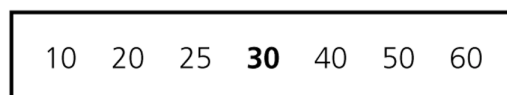
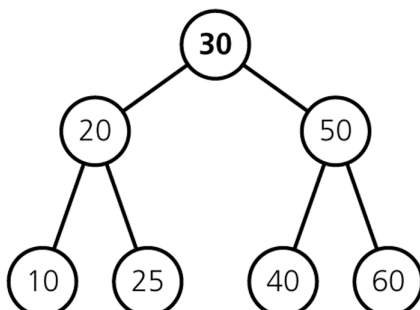


(b)

```
bst.searchTreeInsert(60);  
bst.searchTreeInsert(20);  
bst.searchTreeInsert(10);  
bst.searchTreeInsert(40);  
bst.searchTreeInsert(30);  
bst.searchTreeInsert(50);  
bst.searchTreeInsert(70);
```

215

2. opslaan en als **gebalanceerde** binaire zoekboom inlezen
 - gebruik inorder traversal om de boom weg te schrijven
 - gebruik het aantal knopen om de gebalanceerde boom in te lezen
 - vb: een ingelezen volle binaire zoekboom:



File

216

- een ingelezen niet-volle binaire zoekboom, hoeft niet compleet te zijn

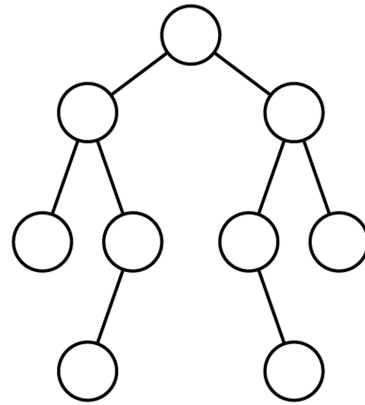
```

• readTree(out tree: bst, in n: integer)
  // lees n knopen in in de deelboom tree
  if (n > 0)
  { readTree(left subtree of the root, n/2)

    read item from file into the root

    readTree(right subtree of the root, (n-1)/2)
  }

```



217

Algemene bomen - n-ary bomen

- een n-ary boom is een algemene boom waarbij de knopen niet meer dan n kinderen hebben
- een 2-ary boom is een binaire boom

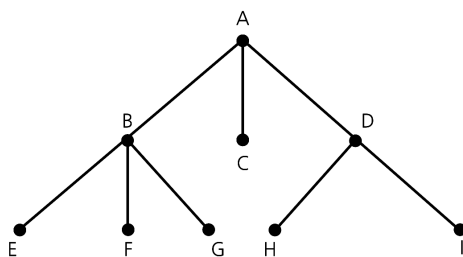


Figure 10-38 A general tree

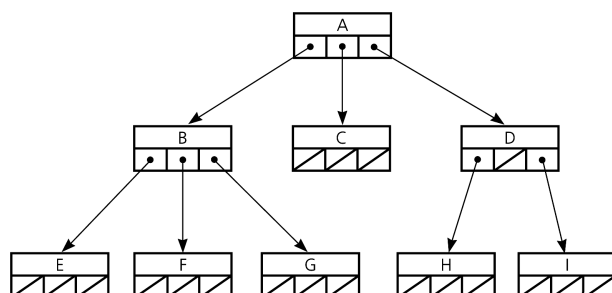


Figure 10-41

An implementation of the n -ary tree in Figure 10-38

218

n-ary bomen

- een binaire boom kan een n-ary boom voorstellen

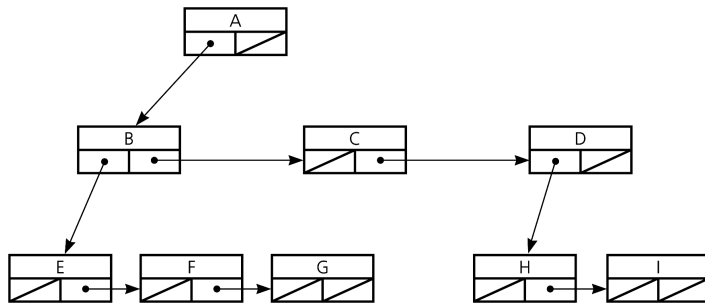


Figure 10-39 Another implementation of the tree in Figure 10-38

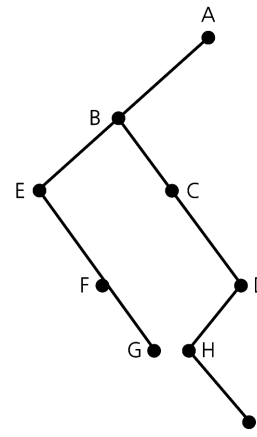


Figure 10-40 The binary tree that Figure 10-39 represents

Gegevensabstractie en -structuren

Thema 15 ADT queue en priority queue

Een queue of wachtrij

- achteraan elementen toevoegen & vooraan elementen verwijderen
- alledaags: files, rij aan de kassa, ...
- toepassing in de informatica: printerqueue
- simulaties: studie (vaak m.b.v. queues) over hoe de wachttijd te verminderen

FIFO

- **First In First Out**

221

- Het ADT queue

FIFO
First In First Out

De bewerkingen:

1. **createQueue** creëert een lege queue.
2. **destroyQueue** wist een queue.
3. **isEmpty** bepaalt of een queue leeg is.
4. **enqueue** voegt een element toe aan de queue.
5. **dequeue** verwijdert het eerst toegevoegde element uit de queue.
6. **getFront** vraagt het eerst toegevoegde element uit de queue op.

Queue
<i>front</i>
<i>back</i>
<i>items</i>
<i>createQueue()</i>
<i>destroyQueue()</i>
<i>isEmpty()</i>
<i>enqueue()</i>
<i>dequeue()</i>
<i>getFront()</i>

Het eerst toegevoegde element zal als eerste opgevraagd of verwijderd worden.

222

Bewerkingencontract van het ADT queue

// QueueItemType is het type van de elementen in de queue

+createQueue()

// Creëert een lege queue.

+destroyQueue()

// Wist een queue.

+isEmpty(): boolean {query}

// Bepaalt of een queue leeg is.

223

+enqueue(in newItem:QueueItemType, out success:boolean)

// Voegt het element 'newItem' toe aan het eind (de staart) van de queue.

// 'success' duidt aan of het toevoegen gelukt is.

+dequeue(out success:boolean)

// Verwijdert de kop van de queue, d.i. het eerst toegevoegde element.

// 'success' duidt aan of het verwijderen gelukt is.

+dequeue(out queueFront:QueueItemType, out success:boolean)

// Plaatst de kop van een queue (d.i. het eerst toegevoegde element) in

// 'queueFront' en verwijdert dan deze kop.

// 'success' duidt aan of het verwijderen gelukt is.

+getFront(out queueFront:QueueItemType, out success:boolean) {query}

// Plaatst de kop van een queue (d.i. het eerst toegevoegde element) in

// 'queueFront' en laat de queue ongewijzigd. 'success' duidt aan of het gelukt is.

224

Dit ADT contract volstaat om het ADT te kunnen gebruiken – we hoeven de implementatie niet te kennen:

Voorbeeld 1: een queue van integers

Operation

```
aQueue.createQueue()  
aQueue.enqueue(5)  
aQueue.enqueue(2)  
aQueue.enqueue(7)  
aQueue.getFront(queueFront)  
aQueue.dequeue(queueFront)  
aQueue.dequeue(queueFront)
```

Queue after operation

front
↓
5
5 2
5 2 7
5 2 7 (queueFront is 5)
2 7 (queueFront is 5)
7 (queueFront is 2)

225

Voorbeeld 2: een string inlezen

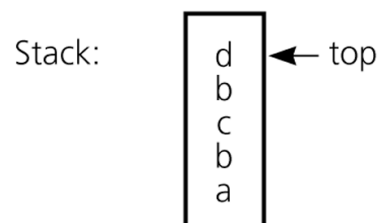
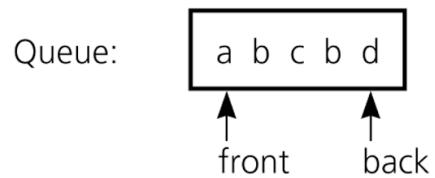
// read a string of characters from a single line of input into a queue

```
aQueue.createQueue()  
while (not end of line)  
{   read a new character ch  
    aQueue.enqueue(ch)  
}   // end while
```

226

Voorbeeld 3: gebruik stacks en queues samen om palindromen te herkennen

String: abcbd



227

isPal(in str:string): boolean

// determines whether str is a palindrome

// create an empty queue and an empty stack

aQueue.createQueue()

aStack.createStack()

// insert each character of the string into both the queue and the stack

length = length of str

for (i=1 to length)

{ nextChar = ith character of str

aQueue.enqueue(nextChar)

aStack.push(nextChar)

} // end for

228

```

// compare the queue characters with the stack characters
charactersAreEqual = true
while (aQueue is not empty and charactersAreEqual)
{
    aQueue.getFront(queueFront)
    aStack.getTop(stackTop)
    if (queueFront == stackTop)
    {
        aQueue.dequeue()
        aStack.pop()
    }
    else
        charactersAreEqual = false
} // end while
return charactersAreEqual

```

229

Het ADT Priority Queue

- vb. spoedafdeling ziekenhuis, de staf
 - plaatst elke patiënt in een database
 - beslist wanneer welke patiënt geholpen wordt
- welk ADT?
 - ADT gesorteerde lijst? Alfabetische volgorde op naam? Volgorde ID-nummer?
 - ADT queue? In volgorde van binnenkomen?
 - Volgens de mate van urgentie!
 - Volgens een **prioriteitswaarde**, de eerstvolgende beschikbare dokter dient de patiënt met de hoogste prioriteit te behandelen.
- ADT priority queue
 - Voeg een prioriteitswaarde toe aan de gegevens
 - Insert een item, d.i. gegevens en prioriteitswaarde, in de priority queue
 - Haal het item met de hoogste prioriteitswaarde uit de priority queue

230

Het ADT Priority Queue

De bewerkingen:

1. Creëer een lege priority queue.
2. Wis een priority queue.
3. Bepaal of een priority queue leeg is.
4. Voeg een nieuw item toe aan een priority queue.
5. Geef het item met de hoogste prioriteit weer, en verwijder het uit de priority queue.

PriorityQueue
<i>items</i>
<i>createPriorityQueue()</i>
<i>destroyPriorityQueue()</i>
<i>pqlsEmpty()</i>
<i>pqInsert()</i>
<i>pqDelete()</i>

231

Bewerkingencontract van het ADT priority queue

// PQItemType is het type van de elementen in de priority queue.

+createPQueue()

+destroyPQueue()

+pqlsEmpty(): boolean {query}

+pqInsert(in newItem: PQItemType, out success: boolean)

+pqDelete(out priorityItem: PQItemType, out success: boolean)

// Stopt het item met de hoogste prioriteit in priorityItem, en verwijdert het uit
// de priority queue. Success is false als de priority queue leeg is.

232

Positie-georiënteerde en waarde-georiënteerde ADTs

- **Positie**-georiënteerde ADTs: bewerkingen die
 - gegevens toevoegen op de i^{de} positie
 - gegevens verwijderen van de i^{de} positie
 - informatie opvragen over gegevens op de i^{de} positie
 - bijvoorbeeld: lijst, stack, queue
- **Waarde**-georiënteerde ADTs: bewerkingen die
 - gegevens toevoegen volgens hun waarde (bv. zoeksleutel, prioriteitswaarde, ...)
 - gegevens verwijderen volgens hun waarde
 - informatie opvragen over gegevens op basis van hun waarde
 - bijvoorbeeld: gesorteerde lijst, priority queue

233

Gegevensabstractie en -structuren

Thema 16 Queue en priority queue implementaties

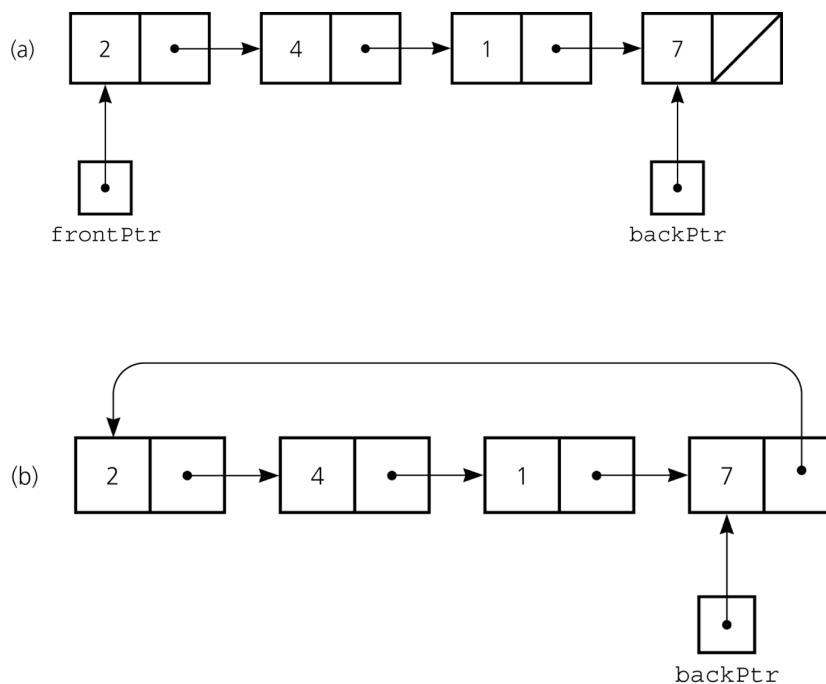
234

Implementaties van het ADT Queue

- a. Een ketting implementatie – dynamische grootte
- b. Een array implementatie – vaste grootte
- c. Een implementatie mbv het ADT Lijst – **hergebruik van reeds geïmplementeerde klasse bespaart tijd**

235

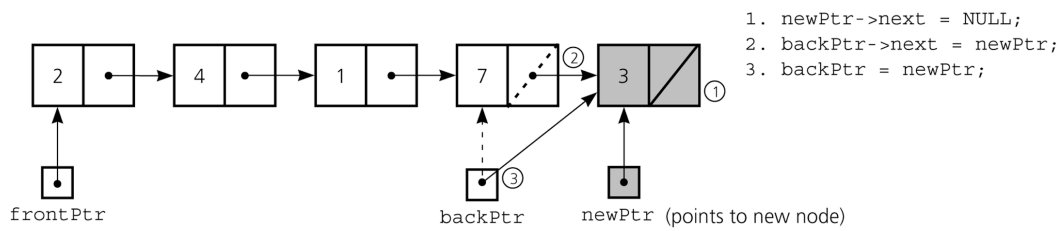
a. Een ketting implementatie van het ADT queue



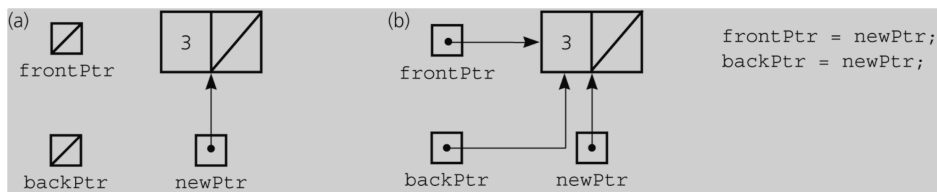
236

Een item toevoegen aan een

1. niet-lege queue

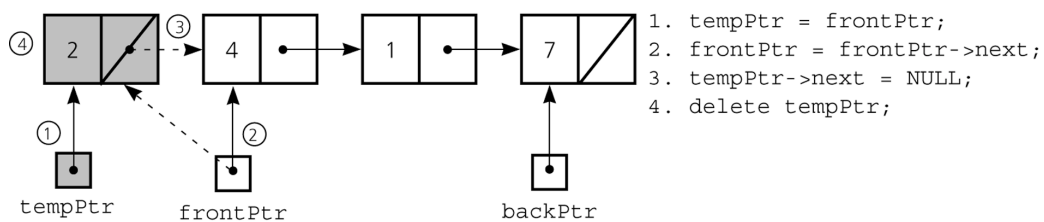


2. lege queue



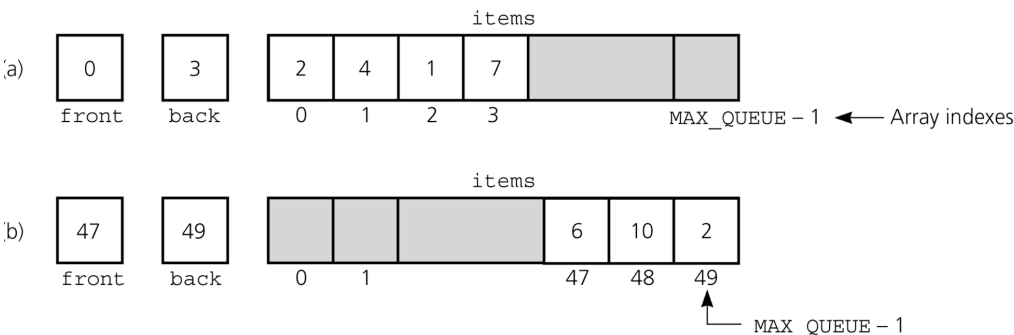
237

Een item verwijderen uit een queue

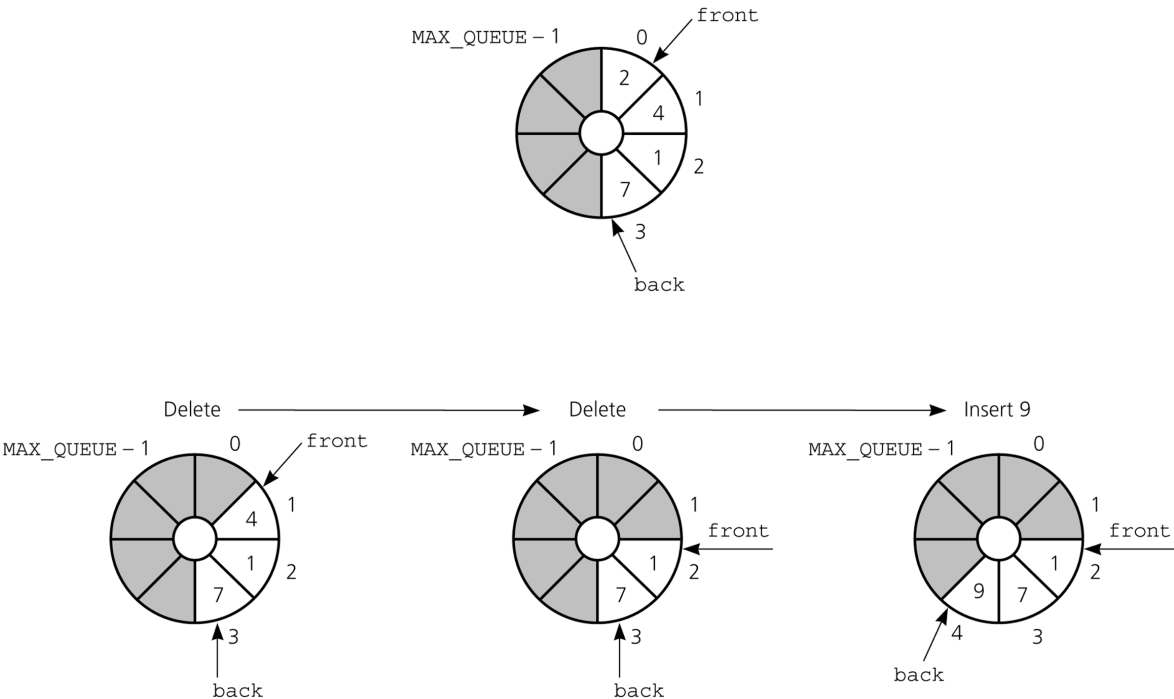


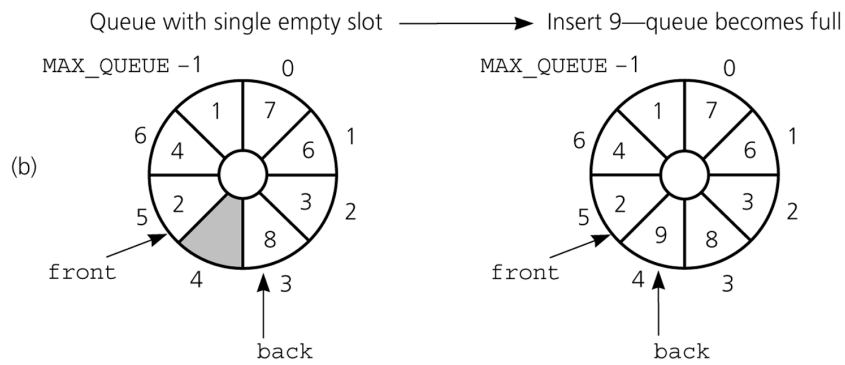
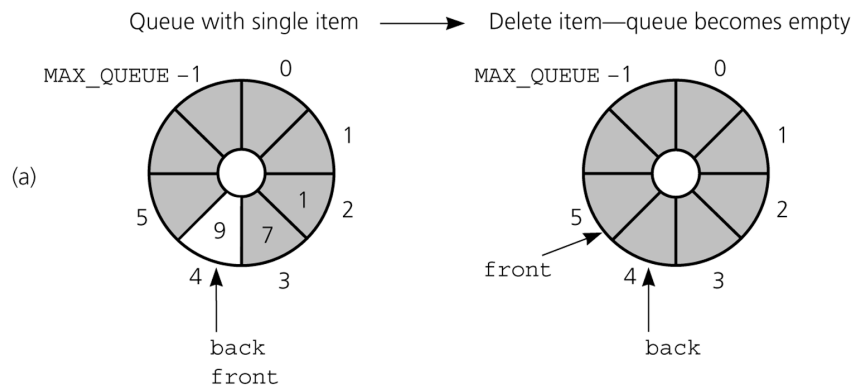
238

b. Een array implementatie van het ADT queue



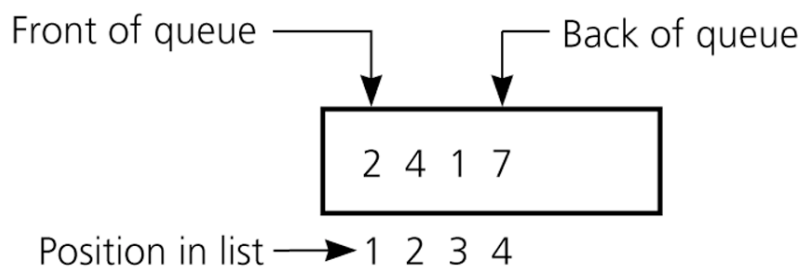
Een circulaire array-implementatie van een queue





241

c. Een implementatie van het ADT Queue mbv het ADT Lijst



242

```

Class Queue {public: ...;   private: List aList;// list of queue items }

+isEmpty(): boolean
    return aList.isEmpty()

+enqueue(in newItem: QueueItemType, out success:boolean)
    aList.insert(aList.getLength()+1, newItem, success)

+dequeue(out success:boolean)
    aList.remove(1, success)

+dequeue(out queueFront: QueueItemType, out success:boolean)
    aList.retrieve(1, queueFront, success)
    aList.remove(1, success)

+getFront(out queueFront: QueueItemType, out success:boolean)    {query}
    aList.retrieve(1, queueFront, success)

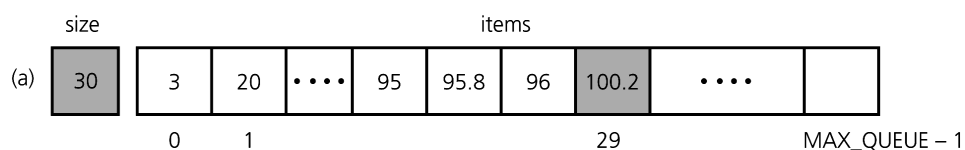
```

243

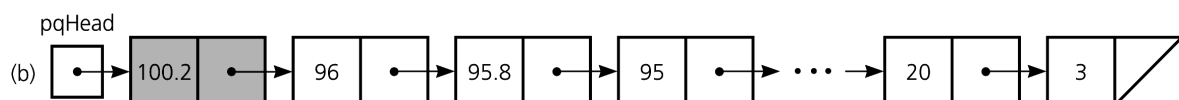
Implementaties van het ADT priority queue

1. Lineaire implementaties

- array implementatie, gesorteerd volgens stijgende prioriteitswaarde.



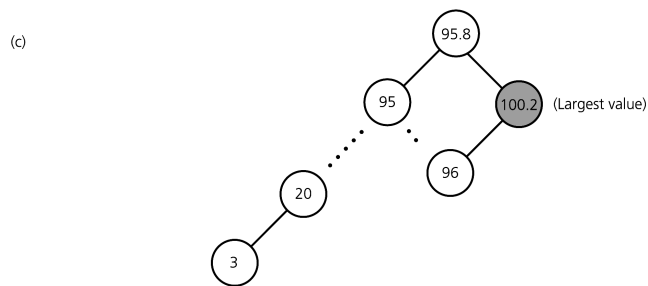
- ketting implementatie, gesorteerd volgens dalende prioriteitswaarde.



244

2. Niet-lineaire implementaties

- Binaire zoekboom implementatie; waar zit het item met de hoogste prioriteit?



- Heap implementatie

245

Een implementatie van het ADT priority queue mbv het ADT heap

De prioriteitswaarde in de priority queue komt overeen met de zoeksleutel in de heap.

```
class Priority Queue
{public:...; private: Heap h};

+pqInsert(in newItem: PQItemType, out success: boolean)
    h.heapInsert(newItem, success)

+ pqDelete(out priorityItem: PQItemType, out success: boolean)
    h.heapDelete(priorityItem, success)
```

Nadeel: kennis van het max aantal items in de priority queue is noodzakelijk (array)

Voordeel: een heap is altijd gebalanceerd → efficiënt

246

Gegevensabstractie en -structuren

Thema 17 Efficiëntie van algoritmen

247

Wat is een goede oplossing?

- criterium: een oplossing is goed als de **totale** kost van
 - probleemanalyse
 - ontwerp
 - implementatie
 - debuggen
 - testen
 - gebruik
 - onderhoud
 - aanpassen
 - uitbreidenminimaal is!
- efficiëntie is vaak een bepalende factor voor het al dan niet bruikbaar zijn van een oplossing
- maar, efficiëntie is slechts één aspect van de kost van een oplossing
- houd het multidimensionale kostplaatje in gedachten

248

De efficiëntie van algoritmen meten

Analyse van algoritmen

- vergelijkt de efficiëntie van verschillende algoritmen
 - tijdsefficiëntie en ruimte-efficiëntie
- bestudeert enkel significante verschillen in efficiëntie
- geen vergelijking van programma's!
- geen gepruts met slimme codeertrukjes ten koste van de leesbaarheid (implementatie- en onderhoudskost stijgt)

249

Hoe de efficiëntie van 2 verschillende algoritmen vergelijken?

1. Implementeren en uitvoeren – d.i. het vergelijken van programma's

drie fundamentele problemen:

- de kwaliteit van de **implementatie** van het programma speelt mee
- de snelheid van de bewerkingen is **computer**afhankelijk:
 - 1 computer kan voor sommige bewerkingen sneller en voor andere trager zijn
- de **gegevens** beïnvloeden de efficiëntie

2. Gebruik wiskundige technieken

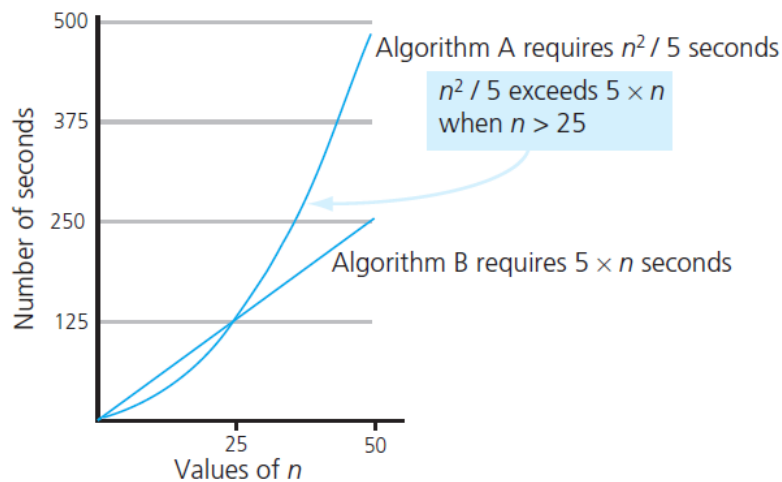
Analyse van algoritme-efficiëntie moet onafhankelijk zijn van:

- implementaties
- computers
- gegevens

250

De groeigraad van een algoritme

Meet de tijdsvereisten in functie van de probleemgrootte n



251

Grootte-orde analyse en Big O notatie

Big O notatie

- **$O(f(n))$** is de **notatie** voor: 'van de (grootte-)orde $f(n)$ '
- **n** is de **probleemgrootte**
- **$f(n)$** is de **groeigraadfunctie** (growth-rate function):
 - een wiskundige functie die de orde van een algoritme weergeeft in functie van n

Definitie van de orde van een algoritme:

- **Algoritme A is $O(f(n))$** als er constanten k en n_0 bestaan zodat A niet meer dan $k \cdot f(n)$ tijdseenheden nodig heeft om een probleem van grootte $n \geq n_0$ op te lossen.

$n > n_0$ betekent: voor voldoende grote problemen

252

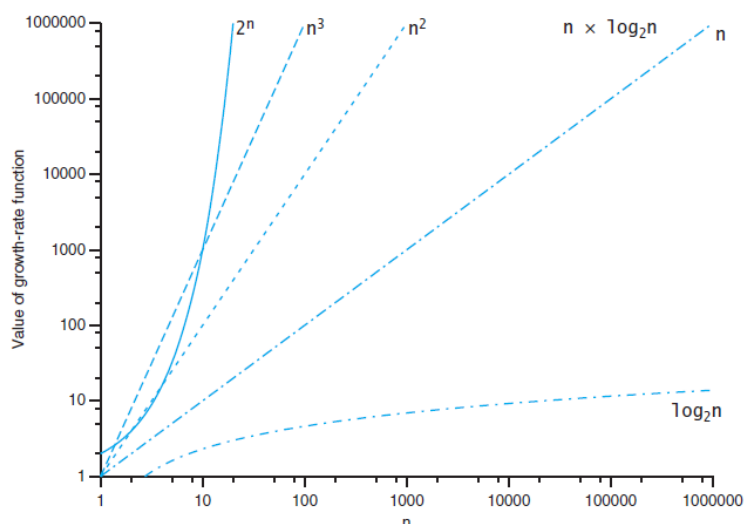
Grootte-orde analyse en Big O notatie

$O(1) < O(\log_2 n) < O(n) < O(n \cdot \log_2 n) < O(n^2) < O(n^3) < O(2^n)$

Function	n					
	10	100	1,000	10,000	100,000	1,000,000
1	1	1	1	1	1	1
$\log_2 n$	3	6	9	13	16	19
n	10	10^2	10^3	10^4	10^5	10^6
$n \times \log_2 n$	30	664	9,965	10^5	10^6	10^7
n^2	10^2	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

253

Grootte-orde analyse en Big O notatie



254

Grootte-orde analyse en Big O notatie

Intuïtieve betekenis van de groeigraadfunctie f in $O(f(n))$:

- 1 constante tijd en dus onafhankelijk van n
- $\log_2 n$ logaritmische tijd – langzame toename in tijd wanneer n toeneemt: n^2 geeft slechts een verdubbeling van tijd, bijv. binair zoeken en andere problemen die worden opgelost door een kleiner constant deel van het probleem op te lossen
- n lineaire tijd – gelijke toename in tijd met n : n^2 geeft tijd²
- $n \cdot \log_2 n$ de tijd neemt sneller toe dan n – problemen die worden opgelost door ze in kleinere problemen te verdelen die elk afzonderlijk worden opgelost
- n^2 kwadratische tijd – sterke toename in tijd wanneer n toeneemt – oplossingen die twee geneste lussen gebruiken – deze zijn enkel bruikbaar voor kleine n , bijv. bubble sort
- n^3 kubieke tijd – nog sterkere toename in tijd wanneer n toeneemt, dan bij kwadratische tijd – oplossingen die drie geneste lussen gebruiken – enkel bruikbaar voor kleine n

255

- 2^n exponentiële tijd – de tijd neemt te snel toe bij toename van n om bruikbaar te zijn voor grote n – bijv. towers of Hanoi

Eigenschappen van groeigraadfuncties:

1. laat lagere graadstermen weg, vb. $O(n^3 + 3n)$ is $O(n^3)$
2. laat constante factoren in de hoogstegraadsterm weg, vb. $O(5 f(n)) = O(f(n))$
3. $O(f(n)) + O(g(n)) = O(f(n) + g(n))$

256

Worst case versus average case analyses

Eenzelfde algoritme A kan verschillende tijdsvereisten hebben om verschillende problemen van dezelfde grootte op te lossen.

bv. algoritme voor sequentieel zoeken

- vergelijk item per item uit de array met het te zoeken item
- stop als het item gevonden is of als alle items bezocht zijn

Worst-case analyse van A

- bepaalt de maximum hoeveelheid tijd die A nodig heeft om een probleem op te lossen
- bv. sequentieel zoeken: worst case $O(n)$

Average-case analyse van A

- bepaalt de gemiddelde hoeveelheid tijd die A nodig heeft om een probleem op te lossen
- bv. sequentieel zoeken: average case $O(n)$

Best-case analyse van A

- bepaalt de minimum hoeveelheid tijd die A nodig heeft om een probleem op te lossen
- bv. sequentieel zoeken: best case $O(1)$

257

Keeping your perspective

Enkel **significante verschillen** in efficiëntie zijn interessant.

Frequentie van bewerkingen

- als je een algoritme kiest, ga dan na hoe frequent het zal worden toegepast
- kritieke bewerkingen moeten sowieso efficiënt zijn, ook als ze zelden gebruikt worden.

Als het probleem altijd klein is, is een efficiëntie-analyse meestal niet nodig.

Soms is het nodig de tijds- en de ruimte-efficiëntie af te wegen.

Vergelijk algoritmen naar stijl en naar efficiëntie.

258

De efficiëntie van algoritmen vergelijken

= grootte-ordes vergelijken = groeigraadfuncties vergelijken

Bepaal de significante bewerkingen

Criterium: bereken het aantal significante bewerkingen → groeigraadfunctie →
standaard groeigraadfunctie (vereenvoudigen mbv eigenschappen)

De efficiëntie van zoekalgoritmen

Significante bewerking: vergelijken

Criterium: het aantal vergelijkingen

Sequentieel zoeken van een array met n items

- Algoritme
- Efficiëntie
 - worst case: $O(n)$
 - average case: $O(n)$
 - best case: $O(1)$

259

De efficiëntie van zoekalgoritmen

Binair zoeken in een gesorteerde array met n items

- Algoritme
 - halveer de array
 - bepaal welke helft het item kan bevatten
 - herhaal deze stappen
- Efficiëntie
 - worst case: $O(\log_2 n)$
 - voor grote arrays is binair zoeken veel beter dan sequentieel zoeken: $\log_2 n < 20$ voor $n = 1$ miljoen!

260

Binair zoeken in een gesorteerde array met n items

We bewijzen dat de worst case efficiëntie $O(\log_2 n)$ is.

na 1^e vergelijking en splitsing: zoeken in 1 van max 2^1 delen

na 2^e vergelijking en splitsing: zoeken in 1 van max 2^2 delen

na 3^e vergelijking en splitsing: zoeken in 1 van max 2^3 delen

...

na de laatste, k^{de} , vergelijking en splitsing: zoeken in 1 van max 2^k delen en elk deel bevat 1 element

→ er zijn in totaal maximum k vergelijkingen. Waaraan is k gelijk bij n elementen?

1. stel n is een macht van 2, dan $n = 2^k \Leftrightarrow k = \log_2 n$

2. stel n is geen macht van 2,

dan $2^{k-1} < n < 2^k$

$\Leftrightarrow k-1 < \log_2 n < k$

$\Leftrightarrow k < \log_2 n + 1 < k + 1$

$\Leftrightarrow k = \text{het gehele deel van } (\log_2 n + 1) = \lfloor \log_2 n + 1 \rfloor$

Dus in beide gevallen vinden we $O(\log_2 n)$

□

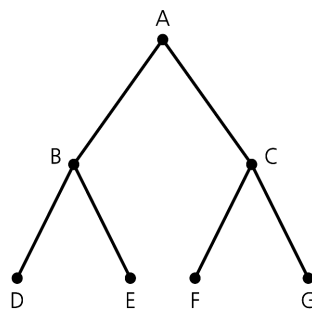
261

Gegevensabstractie en -structuren

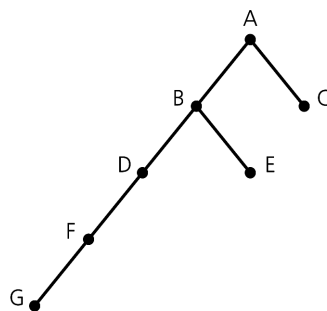
Thema 18 Binaire bomen, theorema's en efficiëntie

262

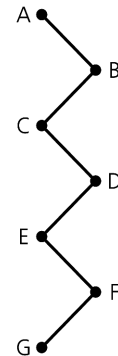
De maximum en minimum hoogte van een binaire boom



(a)



(b)



(c)

263

Gegeven de hoogte van een binaire boom, wat is het aantal knopen?

- **Theorema 15.1**

Een volle binaire boom van hoogte $h \geq 0$ heeft $2^h - 1$ knopen.

- **Theorema 15.2**

Een binaire boom van hoogte h heeft ten hoogste $2^h - 1$ knopen.

Gegeven het aantal knopen in een binaire boom, wat is de hoogte?

- De hoogte van een binaire boom met n knopen is ten hoogste n . (1)

- **Theorema 15.3**

De hoogte van een binaire boom met n knopen is minstens $\lceil \log_2(n+1) \rceil$. (2)

- **(1)+(2) $\rightarrow \lceil \log_2(n+1) \rceil \leq \text{hoogte binaire boom met } n \text{ knopen} \leq n$**

264

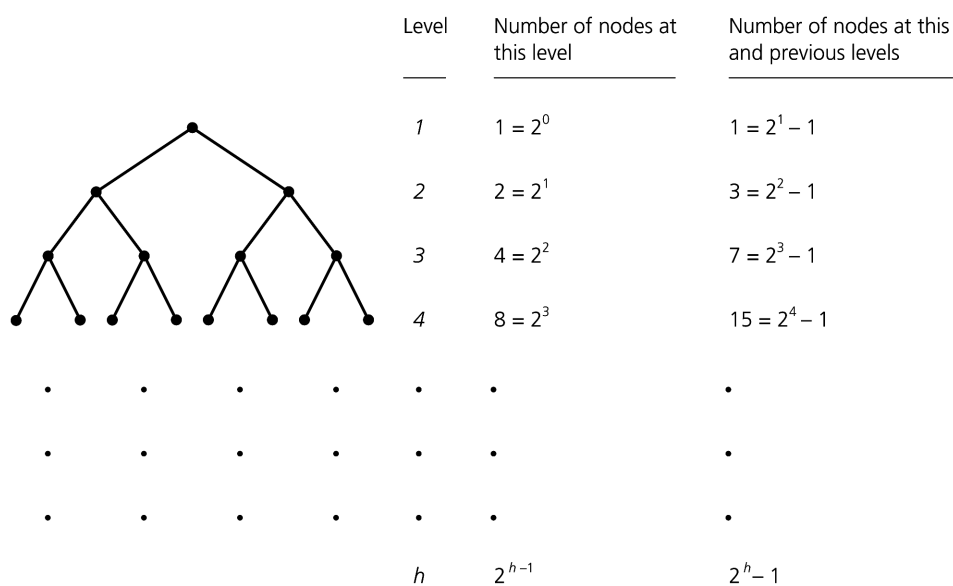


Figure 10-32 Counting the nodes in a full binary tree of height h

De bewijzen

Theorema 15.1

Een **volle** binaire boom van hoogte $h \geq 0$ heeft $2^h - 1$ knopen.

Bewijs:

Per inductie op de hoogte van de **volle** binaire boom.

Basis: hoogte = 0, dus 0 knopen. $2^0 - 1 = 0$.

Inductiehypothese IH: stel dat het theorema klopt voor alle hoogten $\leq h$.

Inductie: bewijs het theorema voor hoogte $h+1$.

De 2 deelbomen L en R van de wortel zijn vol en hebben hoogte h .

Volgens de **IH** hebben ze elk $2^h - 1$ knopen.

Samen met de wortel is dit in totaal $2(2^h - 1) + 1 = 2^{h+1} - 2 + 1 = 2^{h+1} - 1$

□

Theorema 15.2

Een binaire boom van hoogte h heeft ten hoogste $2^h - 1$ knopen.

Bewijs:

aantal knopen van een binaire boom van hoogte $h \leq$

aantal knopen van een **volle** binaire boom van hoogte $h = 2^h - 1$ (theorema 15.1)

□

Lemma

Stel n een strikt positief natuurlijk getal, dan bestaat er een strikt positief natuurlijk getal h zodat

$$2^{h-1} - 1 < n \leq 2^h - 1$$

Bewijs:

Per inductie op n .

Basis: $n=1$

Neem $h=1$, dan wordt de ongelijkheid $2^0 - 1 < 1 \leq 2^1 - 1$.

Inductiehypothese IH: voor alle waarden $\leq n$ klopt het lemma.

Inductie: bewijs het lemma voor $n+1$.

Volgens de IH: $2^{h-1} - 1 < n \leq 2^h - 1$.

Dan is

(1) $n < 2^h - 1$ of

(2) $n = 2^h - 1$.

Als (1) geldt, dan ook $2^{h-1} - 1 < n+1 \leq 2^h - 1$.

Als (2) geldt, dan ook $2^h - 1 < n+1 \leq 2^{h+1} - 1$ want $n+1 = 2^h \leq 2^h + 2^h - 1$ (h is strikt positief)

□

Theorema 15.3

De hoogte van een binaire boom met n knopen is minstens $\lceil \log_2(n+1) \rceil$.

Bewijs:

Gegeven: het aantal knopen van een binaire boom is n .

Gevraagd: de minimum hoogte h van de binaire boom.

Uit het lemma volgt dat er een strikt positief natuurlijk getal h is zodat

$$2^{h-1} - 1 < n \leq 2^h - 1 \quad (\star)$$

Volgens theorema 15.1 is $2^{h-1} - 1$ resp. $2^h - 1$ het aantal knopen van een **volle** binaire boom met hoogte $h-1$ resp. h .

Dus uit $(\star)$ volgt

- dat er een **complete** binaire boom bestaat met n knopen en hoogte h , en
- dat elke binaire boom met hoogte $h-1$ minder dan n knopen heeft (theorema 15.2)

M.a.w. h is de minimum hoogte voor een binaire boom met n knopen.

Waarom is h gelijk? Uit $(\star)$ volgt dat

$$2^{h-1} < n + 1 \leq 2^h$$

of

$$h-1 < \log_2(n+1) \leq h$$

Bijgevolg is $\log_2(n+1) = h$ of $\lceil \log_2(n+1) \rceil = h$. □

269

Traversals van een binaire zoekboom

- Zoals traversals voor gewone binaire bomen.

- **Theorema**

Inorder traversal van binaire zoekboom bezoekt de knopen in gesorteerde zoeksleutel volgorde.

Bewijs:

Per inductie op de hoogte van de binaire zoekboom.

Basis: hoogte = 1, dus 1 knoop. Triviaal.

Inductiehypothese IH: stel dat het theorema klopt voor alle hoogten $\leq h$.

Inductie: bewijs het theorema voor $h+1$.

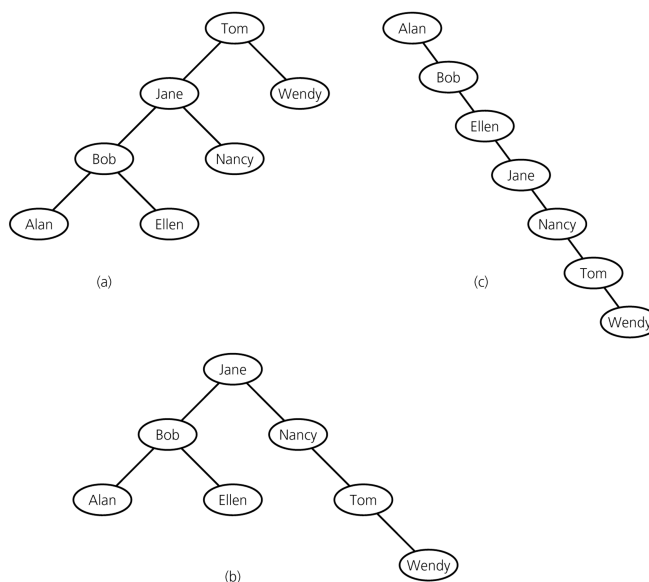
De 2 deelbomen L en R van de wortel hebben een hoogte $\leq h$. Volgens de IH worden L en R door een inorder traversal doorlopen in gesorteerde zoeksleutel volgorde. Uit het algoritme van inorder traversal volgt dat de wortel bezocht wordt na L en voor R . Vermits het een zoekboom is, is de zoeksleutel van de wortel groter dan elke zoeksleutel in L en kleiner dan elke zoeksleutel in R . Dus de volgorde klopt: $L < \text{wortel} < R$. □

270

Efficiëntie van de binaire zoekboom bewerkingen

Bij elke retrieve, insert en delete bewerking gebeuren een aantal vergelijkingen van zoek sleutelwaarden, volgens een pad startend bij de wortel en eindigend bij de gezochte knoop of een lege deelboom.

DUS: hoogte van binaire zoekboom bepaalt maximum aantal vergelijkingen.



271

Binaire zoekboom met n nodes en **maximale hoogte**

- $\approx$ sequentieel zoeken
- in het slechtste geval moet heel deze ketting doorlopen worden

Binaire zoekboom met n nodes en **minimale hoogte**

- elk niveau van de boom moet zo volledig mogelijk gevuld zijn, zoals bij complete binaire bomen
- deze minimale hoogte is $\lceil \log_2(n+1) \rceil$
- tijd nodig om een waarde te zoeken in een binaire zoekboom met minimale hoogte $\approx$ tijd nodig voor binair zoeken in een gesorteerde array

MAAR: insert en delete bewerkingen beïnvloeden de hoogte van een binaire zoekboom; we kunnen niet garanderen dat de hoogte minimaal zal blijven.

- insert in volgorde van de zoek sleutels geeft een binaire zoekboom met maximale hoogte
- insert in random volgorde geeft een binaire zoekboom die de minimale hoogte benadert

272

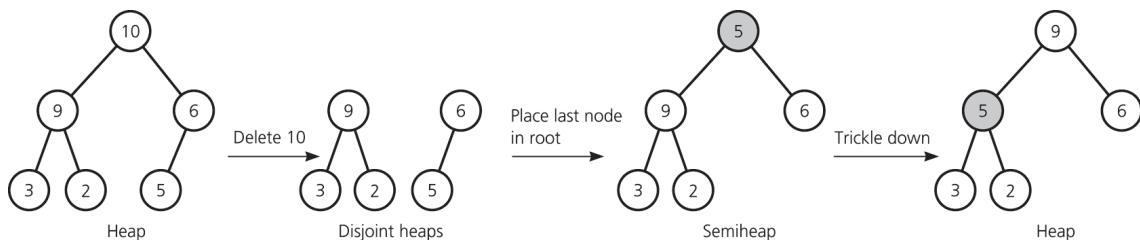
Efficiëntie overzicht

The Big O for the retrieval, insertion, removal, and traversal operations of the ADT binary search tree

Operation	Average case	Worst case
Retrieval	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Removal	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$

273

Efficiëntie heapDelete en heapRebuild



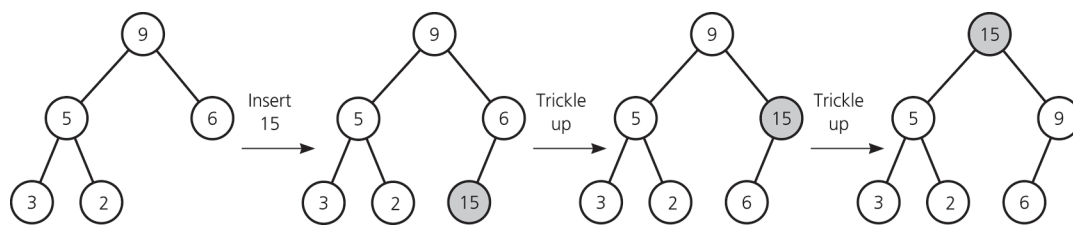
efficiëntie:

items worden verwisseld langs een pad dat in het slechtste geval van de root naar een blad gaat (trickle down) - het aantal verwisselingen (swaps) is dus hoogstens de hoogte van de boom.

→ $O(\log n)$

274

Efficiëntie heapInsert



efficiëntie:

items worden verwisseld langs een pad dat in het slechtste geval van een blad naar de root gaat (trickle up) - het aantal verwisselingen (swaps) is dus hoogstens de hoogte van de boom.

→ $O(\log n)$

275

Gegevensabstractie en -structuren

Thema 19
ADT tabel,
tabelimplementaties
en hun efficiëntie

276

Tabellen (dictionaries, maps)

Het ADT tabel (of ADT dictionary)

- gebruikt een zoek sleutel om de items te identificeren
- de items zijn objecten die meerdere gegevens bevatten

Aanname: alle items in de tabel hebben een verschillende zoek sleutelwaarde.

<u>City</u>	<u>Country</u>	<u>Population</u>
Athens	Greece	2,500,000
Barcelona	Spain	1,800,000
Cairo	Egypt	9,500,000
London	England	9,400,000
New York	U.S.A.	7,300,000
Paris	France	2,200,000
Rome	Italy	2,800,000
Toronto	Canada	3,200,000
Venice	Italy	300,000

277

Het ADT Tabel

Mogelijke ADT bewerkingen:

1. Creëer een lege tabel.
2. Wis een tabel.
3. Bepaal of de tabel leeg is.
4. Bepaal het aantal items in een tabel.
5. Voeg een nieuw item aan de tabel toe.
6. Verwijder het item met een bepaalde zoek sleutelwaarde uit de tabel.
7. Vraag het item met een bepaalde zoek sleutelwaarde op.
8. Doorloop de items van de tabel, gesorteerd volgens zoek sleutel.

Table
<i>items</i>
<i>createTable()</i> <i>destroyTable()</i> <i>tableIsEmpty()</i> <i>tableLength()</i> <i>tableInsert()</i> <i>tableDelete()</i> <i>tableRetrieve()</i> <i>traverseTable()</i>

278

Specificaties van de bewerkingen van het ADT tabel.

// TableItem type is het type van de elementen in de tabel.
// Elk element heeft een zoek sleutelveld van het type KeyType.

+createTable()

+destroyTable()

+tableIsEmpty(): boolean {query}

+tableLength(): integer {query}

279

+tableInsert(in newItem: TableItem type, out success: boolean)
// Voegt newItem toe aan een table met items met unieke zoek sleutelwaarden,
// verschillend van de zoek sleutel van newItem. Success geeft weer
// of het toevoegen gelukt is.

+tableDelete(in searchKey: KeyType, out success: boolean)

+tableRetrieve(in searchKey: KeyType, out tableItem: TableItem type,
out success: boolean)

+traverseTable(in visit: FunctionType)

280

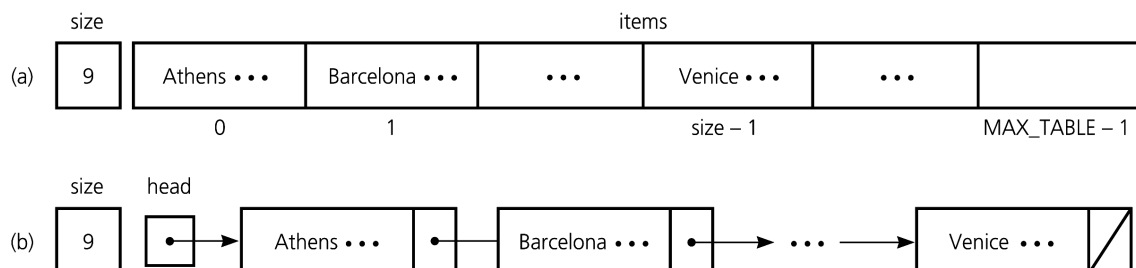
ADT tabel bewerkingen

- d.i. slechts één mogelijke verzameling tabel bewerkingen
- ook andere verzamelingen van tabel bewerkingen zijn mogelijk
- afhankelijk van de toepassing kan een andere verzameling tabel bewerkingen nodig zijn:
 - een deelverzameling van de beschreven bewerkingen
 - andere bewerkingen
 - variaties op de bewerkingen (vb. als zoek sleutels niet uniek moeten zijn).
 - voor traversetable zijn er verschillende definities mogelijk
 - elk item in de tabel bezoeken
 - elk item in de tabel bezoeken in een welbepaalde volgorde

281

Enkele mogelijke implementaties van het ADT tabel

1. Lineaire implementaties, waarbij het aantal elementen bijgehouden wordt:
 - niet-gesorteerd, array.
 - niet-gesorteerd, ketting.
 - gesorteerd (volgens zoek sleutel), array.
 - gesorteerd (volgens zoek sleutel), ketting.

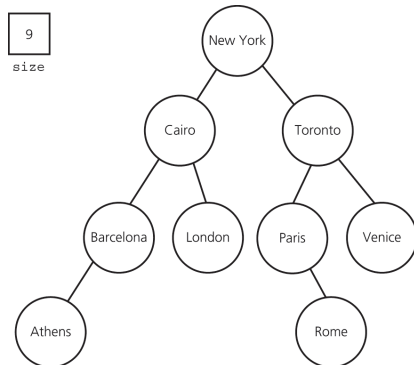


282

2. Niet-lineaire implementatie

- Binaire zoekboom implementatie

- combineert – indien gebalanceerd - de voordelen van lineaire implementaties:
 - array: efficiënt zoeken via binair zoeken
 - ketting: dynamisch en efficiënte insert en delete



- Gevorderde gebalanceerde zoekbomen

- 2-3 bomen
- 2-3-4 bomen
- rood-zwart bomen

3. Hashing

283

Een implementatie kiezen voor het ADT tabel

- Niet alle implementaties voorzien alle bewerkingen even efficiënt.
- Vooraleer implementatie te kiezen:
 - welke bewerkingen zijn er echt **nodig** voor de toepassing?
 - hoe **frequent** zal elke bewerking uitgevoerd worden?
 - zijn er bewerkingen die **zeer belangrijk** zijn en die dus zo snel mogelijk moeten worden uitgevoerd?

→ de implementatie moet gekozen worden op basis van de toepassing.

- Efficiëntie is belangrijk, maar is niet het enige wat telt!

284

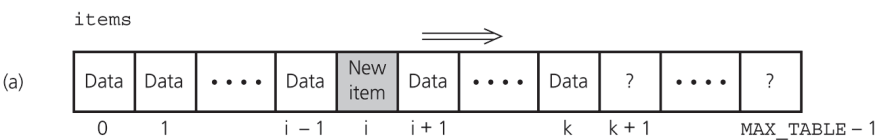
Een vergelijking van lineaire implementaties voor het ADT tabel

1. niet-gesorteerd, array
- insert
 - delete
 - retrieve
 - traversal



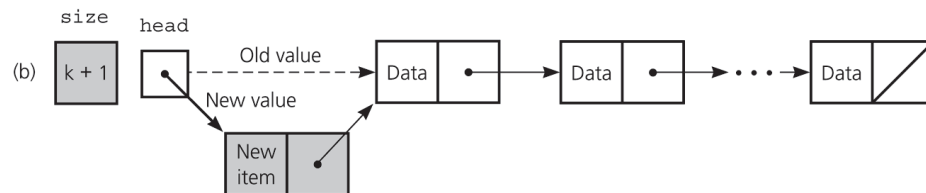
2. gesorteerd (volgens zoeksleutel), array

- insert en delete
- retrieve
- traversal



3. niet-gesorteerd, ketting

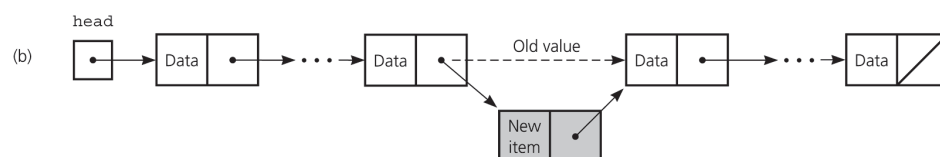
- geen shifts
- insert
- delete
- retrieve
- traversal



287

4. gesorteerd (volgens zoeksleutel), ketting

- geen shifts
- insert, delete en retrieve
- traversal



288

The average-case order of the ADT dictionary operations for various implementations

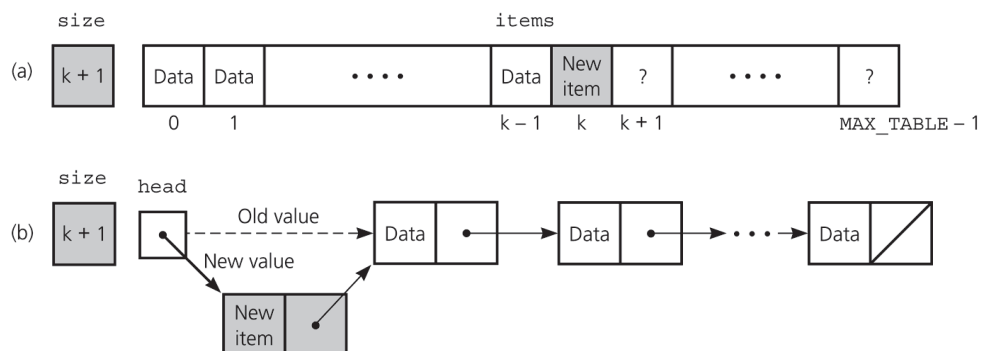
	<u>Insertion</u>	<u>Removal</u>	<u>Retrieval</u>	<u>Traversal</u>
Unsorted array-based	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Unsorted link-based	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Sorted array-based	$O(n)$	$O(n)$	$O(\log n)$	$O(n)$
Sorted link-based	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Binary search tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Balanced				

289

Enkele toepassingsvoorbeelden

Toepassing 1: frequente insert en soms traverse waarbij volgorde niet van belang is.

- sorteren is niet nodig.
- een niet-gesorteerde lineaire implementatie maakt tableInsert zeer efficiënt.



- binaire zoekboom: insert minder efficiënt.

290

Toepassing 2: frequente retrieve

- zoeken op zoek sleutelwaarde moet vlot gaan.
- aantal elementen gekend
 - gesorteerde array implementatie, met binary search.
- geen lineaire ketting implementatie.
- aantal elementen niet gekend
 - binaire zoekboom implementatie, waarbij de boom ongeveer in minimale hoogte wordt gehouden, maakt zoeken ook efficiënt.

291

Toepassing 3: frequente insert, delete, retrieve en traverse in gesorteerde volgorde.

- voor insert en delete moeten er twee stappen uitgevoerd worden.
 1. de juiste locatie in de tabel zoeken (beter bij gesorteerde array implementatie).
 2. insert of delete op deze positie (beter bij gesorteerde ketting implementatie).
- binaire zoekboom implementatie combineert de voordelen van beide implementaties (indien gebalanceerd).

292

Samengevat ... een implementatie kiezen voor het ADT tabel

Lineaire implementaties

- eenvoudig
- ok voor kleine tabellen
- ok voor niet-gesorteerde tabellen waaruit weinig verwijderd en opgevraagd wordt

Niet-lineaire implementaties

- meestal is een binaire zoekboom implementatie een betere keuze dan een lineaire implementatie
- een *gebalanceerde* binaire zoekboom
 - betere efficiëntie voor de verzameling van ADT tabel bewerkingen
- idem voor gevorderde gebalanceerde zoekbomen
 - 2-3 bomen
 - 2-3-4 bomen
 - rood-zwart bomen

Hashing

- de loadfactor α bepaalt de efficiëntie

293

Een implementatie van het ADT tabel mbv het ADT binaire zoekboom

```
class Table
{ public: ...; private: BinarySearchTree bst; int size};

+tableInsert (in newItem: TableItemType, out success: boolean)
    bst.searchTreeInsert(newItem, success);
    IF success THEN
        size++
    END

+tableDelete (in searchKey: KeyType, out success: boolean)
    bst.searchTreeDelete(searchKey, success);
    IF success THEN
        size--
    END

+tableRetrieve(in searchKey: KeyType, out tableItem: TableItemType,
               out success: boolean)
    bst.searchTreeRetrieve(searchKey, tableItem, success)

+traverseTable(in visit: FunctionType)
    bst.inorderTraverse(visit)
```

294

Gegevensabstractie en -structuren

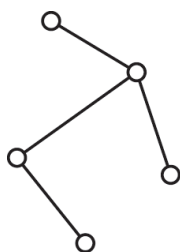
Thema 20 ADT graaf en implementaties

295

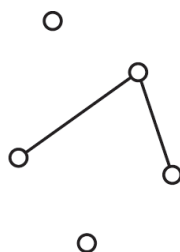
Terminologie

Een graaf bestaat uit twee verzamelingen, $G = (V, E)$

- een verzameling V van *vertices* – hoekpunten - of *nodes* - knopen
- een verzameling E van *edges* - randen



(a)



(b)



(c)

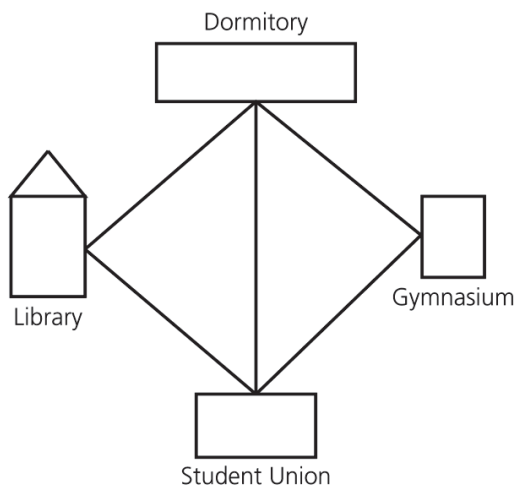
Een deelgraaf $G' = (V', E')$ van G is een graaf die bestaat uit

- een deelverzameling V' van V en
- een deelverzameling E' van E .

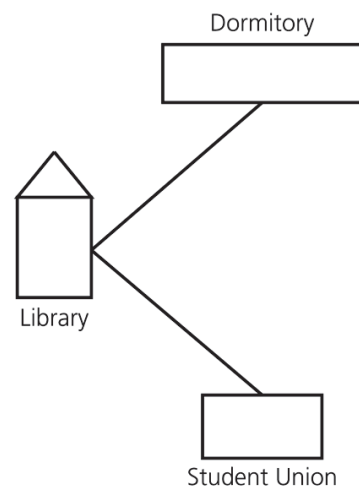
296

Een campusplan als een graaf (a) en een deelgraaf (b)

(a)



(b)



297

Terminologie

Naburige knopen

- 2 knopen verbonden door een edge

Een **pad** tussen 2 knopen

- een opeenvolging van edges beginnend bij de ene knoop en eindigend bij de andere knoop
- mag meerdere keren langs dezelfde knoop gaan

Een **simpel pad**

- een pad dat slechts 1 maal door elke knoop op het pad gaat

Een **cycle**

- een pad dat begint en eindigt bij dezelfde knoop

Een **simpele cycle**

- een cycle die slechts 1 maal door elke knoop op de cycle gaat

298

Een **samenhangende graaf** (connected) (a)

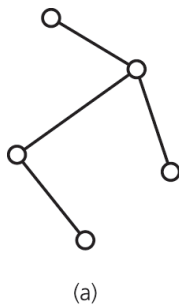
- een graaf die tussen elke 2 verschillende knopen een pad heeft

Een **niet-samenhangende graaf** (unconnected) (b)

- een graaf met minstens 1 paar knopen waartussen geen pad is

Een **complete graaf** (c)

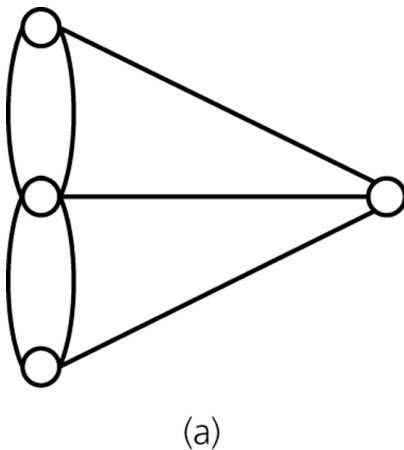
- een graaf met een edge tussen elk paar verschillende knopen



299

Een **multigraaf** (a)

- is geen graaf
- laat meerdere edges toe tussen knopen



Een lus als edge (self edge or loop) is niet toegelaten in een graaf (b)

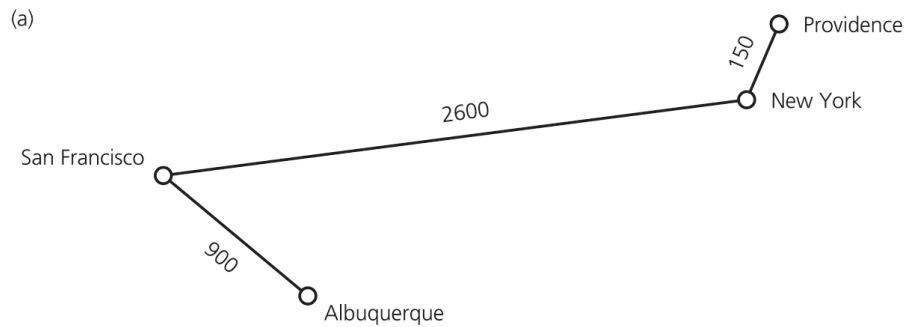
300

Een **gewogen graaf**

- de edges hebben getallen/gewichten als labels

Een **niet-gerichte graaf**

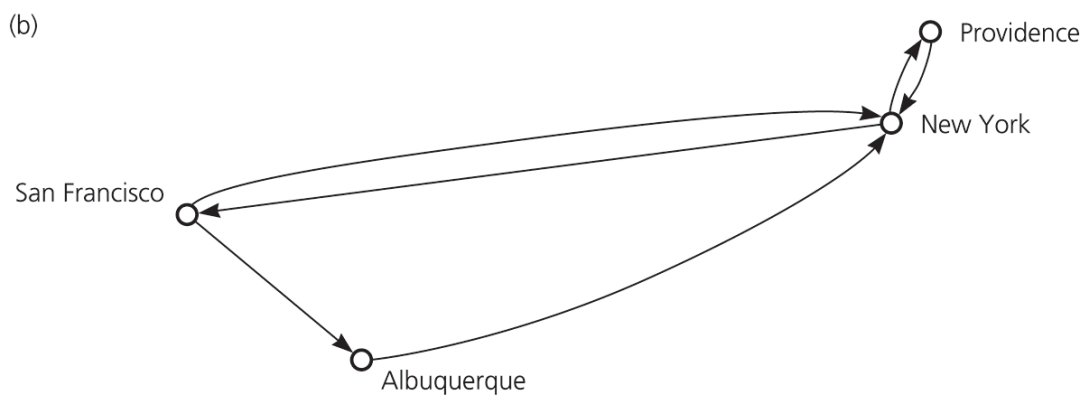
- de edges duiden geen richting aan



301

Een **gerichte graaf**

- pijlen geven een richting aan elke edge: **gerichte edges**
- kan 2 edges hebben tussen een paar knopen, elk met een andere richting
- een gericht pad is een opeenvolging van gerichte edges tussen 2 knopen
- knoop y is naburig aan knoop x als er een gerichte edge is van x naar y



A directed, unweighted graph

302

Het ADT graaf

ADT bewerkingen:

- `create(G)`
- `isEmpty(G)`
- `insertVertex(G, v)`
- `insertEdge(G, v1, v2)`
- `deleteVertex(G, v)`
- `deleteEdge(G, v1, v2)`
- `retrieve(G, searchkey, v)`
- `isEdge(G, v1, v2)`
- ...

303

Het ADT graaf

- de knopen kunnen al dan niet gegevens bevatten
 - veel toepassingen hebben geen gegevens in de knopen nodig
 - de edges tussen de knopen zijn van belang
- de edges kunnen al dan niet gericht zijn
- de edges kunnen al dan niet labels bevatten

Insert en delete bewerkingen voor knopen en voor edges

Grafen kunnen ook doorlopen worden (traverse)

304

Implementatie van ADT graaf

Meest gebruikte implementaties:

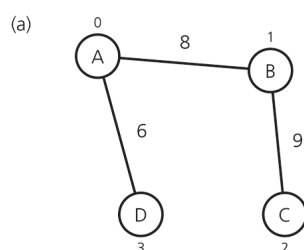
- adjacency matrix
- adjacency list

305

Adjacency matrix

voor een graaf met n knopen genummerd van 0 tot $n-1$ is dit een $n \times n$ matrix met

- voor een niet-gewogen graaf:
 - $\text{matrix}[i][j] = 1$ (of true) als er een edge is tussen knoop i en knoop j
 - $\text{matrix}[i][j] = 0$ (of false) als er geen edge is tussen knoop i en knoop j
- voor een gewogen graaf:
 - $\text{matrix}[i][j] =$ het gewicht van de edge tussen knoop i en knoop j
 - $\text{matrix}[i][j] = \infty$ als er geen edge is tussen knoop i en knoop j

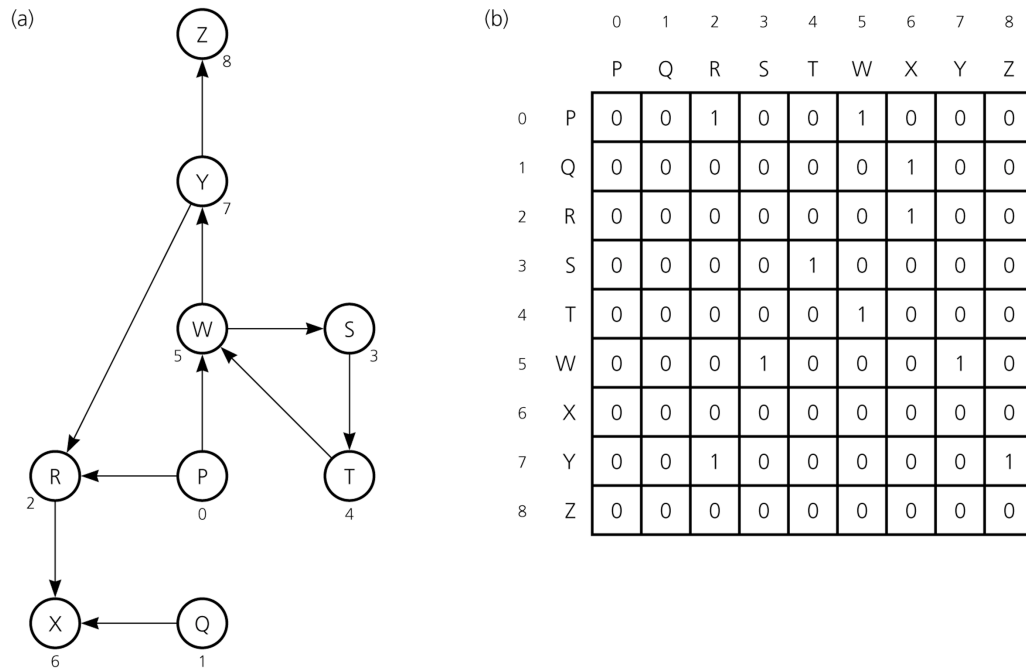


(b)

	0	1	2	3
	A	B	C	D
0 A	∞	8	∞	6
1 B	8	∞	9	∞
2 C	∞	9	∞	∞
3 D	6	∞	∞	∞

306

Adjacency matrix

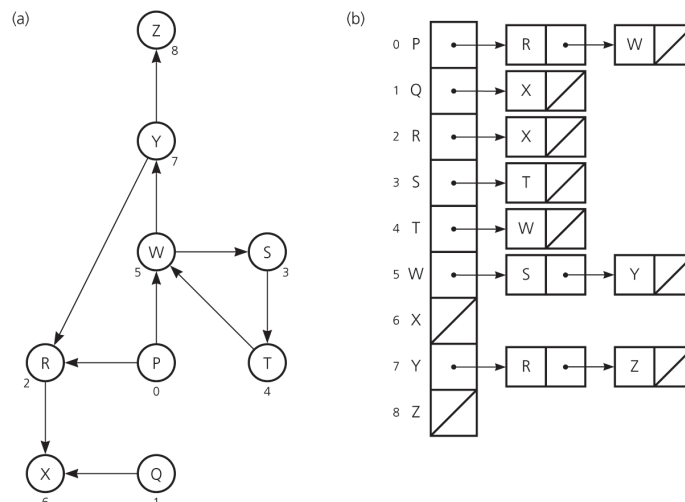


307

Adjacency list

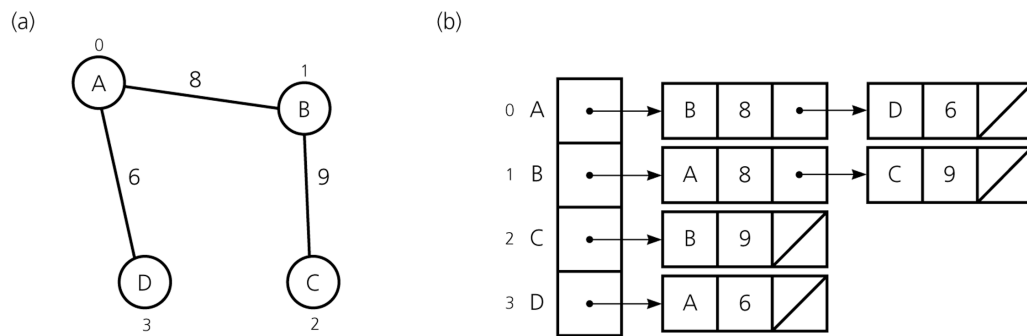
Adjacency list voor een gerichte graaf met n knopen genummerd $0, 1, \dots, n-1$

- een array met n gelinkte kettingen
- j zit in de i -de gelinkte ketting enkel en alleen indien er in de graaf een edge is tussen knoop i en knoop j
- voor een niet-gerichte graaf: behandel elke edge als 2 gerichte edges in tegengestelde richtingen



308

Adjacency list



309

Graaf bewerkingen

Twee veel voorkomende bewerkingen op grafen zijn:

1. bepaal of er een edge is van knoop i naar knoop j
2. vind alle knopen naburig aan een gegeven knoop i

Adjacency matrix

- de eerste bewerking is efficiënter

Adjacency list

- de tweede bewerking is efficiënter
- vereist vaak minder geheugen dan een adjacency matrix

310

Gegevensabstractie en -structuren

Thema 21 Eenvoudige sorteeralgoritmen en hun efficiëntie

<http://www.youtube.com/watch?v=lyZQPjUT5B4>

311

Sorteren

- een proces dat een verzameling gegevens in stijgende of dalende volgorde plaatst
- op basis van de zoek sleutels van de gegevenselementen

Soorten sorteeralgoritmen

- **internal** sort: alle gegevens kunnen samen in main memory
- **external** sort: niet alle gegevens kunnen samen in main memory, er is extra opslag nodig

De efficiëntie van sorteeralgoritmen

- Significante bewerkingen: vergelijken en verplaatsen
- Criteria: aantal vergelijkingen en aantal verplaatsingen (data moves)

312

Selection sort

- Algoritme
 - zoek het grootste item en zet het op z'n plaats (= swap)
 - zoek het 2^e grootste item en zet het op z'n plaats
 - enz.

Shaded elements are selected;
boldface elements are in order.

Initial array:

29	10	14	37	13
----	----	----	----	----

After 1st swap:

29	10	14	13	37
----	----	----	----	-----------

After 2nd swap:

13	10	14	29	37
----	----	----	-----------	-----------

After 3rd swap:

13	10	14	29	37
----	----	-----------	-----------	-----------

After 4th swap:

10	13	14	29	37
-----------	-----------	-----------	-----------	-----------

313

Selection sort algoritme voor n elementen

```
// A[0] .. A[lastUnsorted] is het niet gesorteerde deel
// zoek het element dat in A[lastUnsorted] moet komen
// d.i. het grootste element uit A[0] .. A[lastUnsorted]

for lastUnsorted = n-1 downto 1 do
{
    // bepaal largest
    largest = 0
    for index = 1 to lastUnsorted do
        if A[index] > A[largest]
            largest = index

    // verwissel A[largest] en A[lastUnsorted] van plaats
    temp = A[largest]
    A[largest] = A[lastUnsorted]
    A[lastUnsorted] = temp

    // vanaf A[lastUnsorted] is de array nu gesorteerd
}
```

314

Selection sort efficiëntie analyse

- Efficiëntie
 - worst case: $O(n^2)$
 - average case: $O(n^2)$

→ enkel geschikt voor kleine n

De efficiëntie hangt niet af van de originele volgorde van de gegevens

315

Selection sort efficiëntie analyse

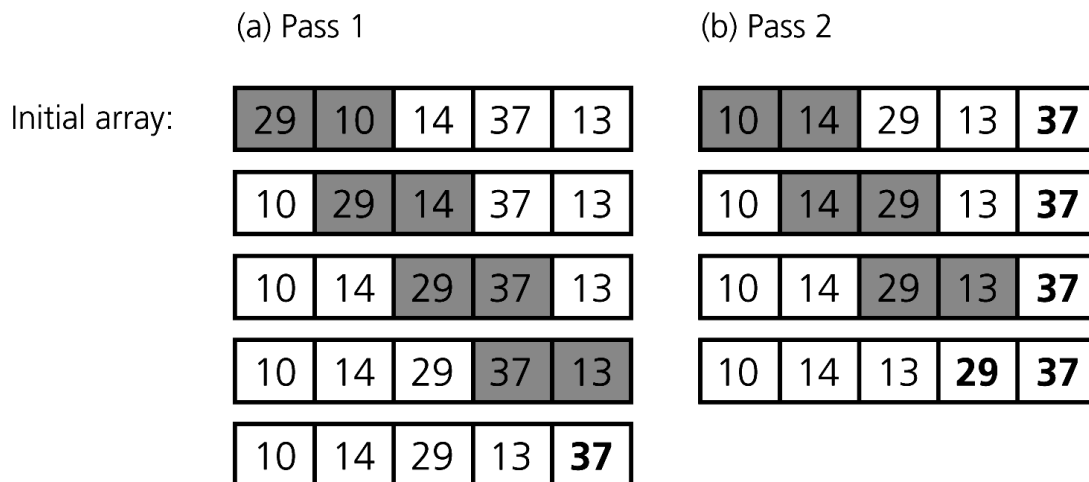
- zoek het grootste element in het niet gesorteerde deel
 - 1^e keer: $n-1$ vergelijkingen
 - 2^e keer: $n-2$ vergelijkingen
 - 3^e keer: $n-3$ vergelijkingen
 - ...
 - $n-1^{\text{e}}$ keer: $n-(n-1) = 1$ vergelijking
 - totaal: $n(n-1)/2$ vergelijkingen
- zet het gevonden element op z'n plaats
 - telkens 1 verwisseling of 3 verplaatsingen
 - $n-1$ keer
- totaal aantal significante bewerkingen: $n(n-1)/2 + 3(n-1) = n^2/2 + 5n/2 - 3$

316

Bubble sort

- Algoritme

- vergelijk (van links naar rechts) aanliggende elementen en swap als ze niet in volgorde zijn
 - borrel zo het grootste element naar z'n plaats
 - borrel dan het 2^e grootste element naar z'n plaats
 - dan het 3^e grootste
 - enz.



317

Bubble sort – algoritme 1

```
sorted = false
for pass = 1 to n-1 do           // A[0] .. A[n-pass] is het niet gesorteerde deel
    if not sorted                // de rest is gesorteerd
    {
        sorted = true
        for i = 0 to n-pass-1 do
            if A[i] > A[i+1]
            {
                sorted = false

                // verwissel A[i] en A[i+1] van plaats
                temp = A[i]
                A[i] = A[i+1]
                A[i+1] = temp
            }
        }
        // vanaf A[n-pass] is de array nu gesorteerd
    }
}
```

Wat is de functie van 'sorted'?

318

Bubble sort algoritme 2

```
exchanges = true
pass = 1                                // A[0] .. A[n-pass] is het niet gesorteerde deel
                                        // de rest is gesorteerd
while pass < n and exchanges do
{   exchanges = false
    for i = 0 to n-pass-1 do
        if A[i] > A[i+1]
        {   exchanges = true

            // verwissel A[i] en A[i+1] van plaats
            temp = A[i]
            A[i] = A[i+1]
            A[i+1] = temp
        }
    pass = pass + 1
}
```

Wat is de functie van 'exchanges'?

319

Bubble sort efficiëntie analyse

- Efficiëntie
 - worst case: $O(n^2)$
 - best case: $O(n)$

Worst case

- n-1 keer wordt het hele niet-gesorteerde deel opgeborreld
 - 1^e keer: n-1 vergelijkingen en n-1 verwisselingen of 3(n-1) verplaatsingen
 - 2^e keer: n-2 vergelijkingen en n-2 verwisselingen of 3(n-2) verplaatsingen
 - ...
 - n-1^e keer: 1 vergelijking en 1 verwisseling of 3 verplaatsingen
- totaal aantal significante bewerkingen: $4n(n-1)/2 = 2n^2 - 2n$ vergelijkingen en verplaatsingen

Best case

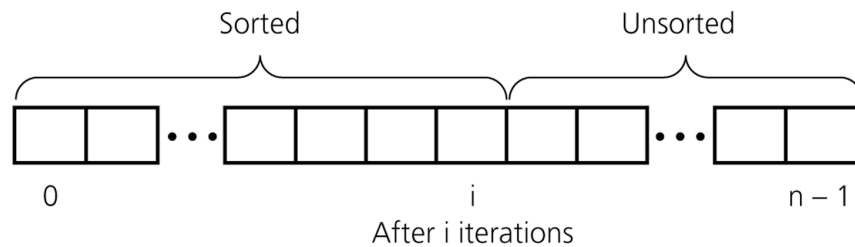
- totaal aantal significante bewerkingen: n-1 vergelijkingen

320

Insertion sort

- Algoritme

- verdeel de array in 2 delen: een gesorteerd en een niet-gesorteerd deel
- bij het begin bevat het gesorteerde deel 1 element
- neem telkens het eerste element uit het niet-gesorteerde deel en voeg het (na de nodige shifts om plaats te maken) op de juiste plaats toe in het gesorteerde deel



321

Insertion sort

Initial array:

29	10	14	37	13
----	----	----	----	----

Copy 10

29	29	14	37	13
----	----	----	----	----

Shift 29

10	29	14	37	13
----	----	----	----	----

Insert 10; copy 14

10	29	29	37	13
----	----	----	----	----

Shift 29

10	14	29	37	13
----	----	----	----	----

Insert 14; copy 37, insert 37 on top of itself

10	14	29	37	13
----	----	----	----	----

Copy 13

10	14	14	29	37
----	----	----	----	----

Shift 37, 29, 14

Sorted array:

10	13	14	29	37
----	----	----	----	----

Insert 13

322

Insertion sort algoritme

```
for firstUnsorted = 1 to n-1 do      // firstUnsorted is eerste plaats in
                                     // het niet gesorteerde deel
{   itemToInsert = A[firstUnsorted]
    // voeg itemToInsert op de juiste plaats in het gesorteerde gedeelte toe

    // zoek loc, de juiste plaats voor itemToInsert
    loc = firstUnsorted
    while (loc > 0 en A[loc-1] > itemToInsert)
    {   A[loc] = A[loc-1]              // shift naar rechts
        loc = loc - 1
    }
    A[loc] = itemToInsert              // itemToInsert staat op z'n plaats
}
```

Het rode vetgedrukte deel is een combinatie van swap en shifts.

323

Insertion sort efficiëntie analyse

- Efficiëntie
 - worst case: $O(n^2)$
 - goed voor kleine arrays omwille van eenvoud
 - zeer onefficiënt voor grote arrays

Worst case

- n-1 keer wordt het 1^e element uit het niet gesorteerde deel op z'n plaats gezet
- een element op z'n plaats zetten:
 - 1^e keer: 1 vergelijking, 1 shift en 2 verplaatsingen
 - 2^e keer: 2 vergelijkingen, 2 shifts en 2 verplaatsingen
 - 3^e keer: 3 vergelijkingen, 3 shifts en 2 verplaatsingen
 - ...
 - n-1^e keer: n-1 vergelijkingen, n-1 shifts en 2 verplaatsingen
- totaal aantal significante bewerkingen: $n(n-1) + 2(n-1) = n^2 + n - 2$

324

Radix sort

- Algoritme
 - behandelt elk item als een karakter string
 - verdeelt de items herhaaldelijk in groepjes volgens het i^{de} karakter van elk item

0123, 2154, 0222, 0004, 0283, 1560, 1061, 2150	Original integers
(156 0 , 215 0) (106 1) (022 2) (012 3 , 028 3) (215 4 , 000 4)	Grouped by fourth digit
1560, 2150, 1061, 0222, 0123, 0283, 2154, 0004	Combined
(000 4) (022 2 , 012 3) (215 0 , 215 4) (156 0 , 106 1) (028 3)	Grouped by third digit
0004, 0222, 0123, 2150, 2154, 1560, 1061, 0283	Combined
(000 4 , 106 1) (012 3 , 215 0 , 215 4) (022 2 , 028 3) (156 0)	Grouped by second digit
0004, 1061, 0123, 2150, 2154, 0222, 0283, 1560	Combined
(000 4 , 012 3 , 022 2 , 028 3) (106 1 , 156 0) (215 0 , 215 4)	Grouped by first digit
0004, 0123, 0222, 0283, 1061, 1560, 2150, 2154	Combined (sorted)

325

Radix sort algoritme voor n elementen met elk d karakters

```
for j = d downto 1
{
  initialiseer 10 lege groepjes
  zet de teller van elk groepje op 0
  for (i=0 to n-1)
  {
    k = jde karakter van het ide element
    plaats het ide element op het eind van groepje k
    verhoog de teller van groepje k met 1
  }
  voeg alle elementen samen tot 1 groep in de volgorde: eerst de elementen van
  groepje 0, dan die van groepje 1, dan van groepje 2, enz.
}
```

326

Radix sort efficiëntie analyse

- Efficiëntie
 - radix sort is $O(n)$
- Om n strings van d karakters te sorteren:
 - per karakter:
 - n verplaatsingen om groepjes te vormen
 - n verplaatsingen om de groepjes te combineren tot 1 groep
 - totaal $2n \cdot d$ verplaatsingen
 - geen vergelijkingen

→ $O(n)$

327

Gegevensabstractie en -structuren

Thema 22 Snelle recursieve sorteeralgoritmen en hun efficiëntie

328

Mergesort

- Een recursief sorteeralgoritme – divide and conquer
- Efficiëntie onafhankelijk van de initiële volgorde van de items
- Algoritme
 - halveer de array
 - sorteer elke helft mbv mergesort
 - merge de gesorteerde helften tot een gesorteerde array

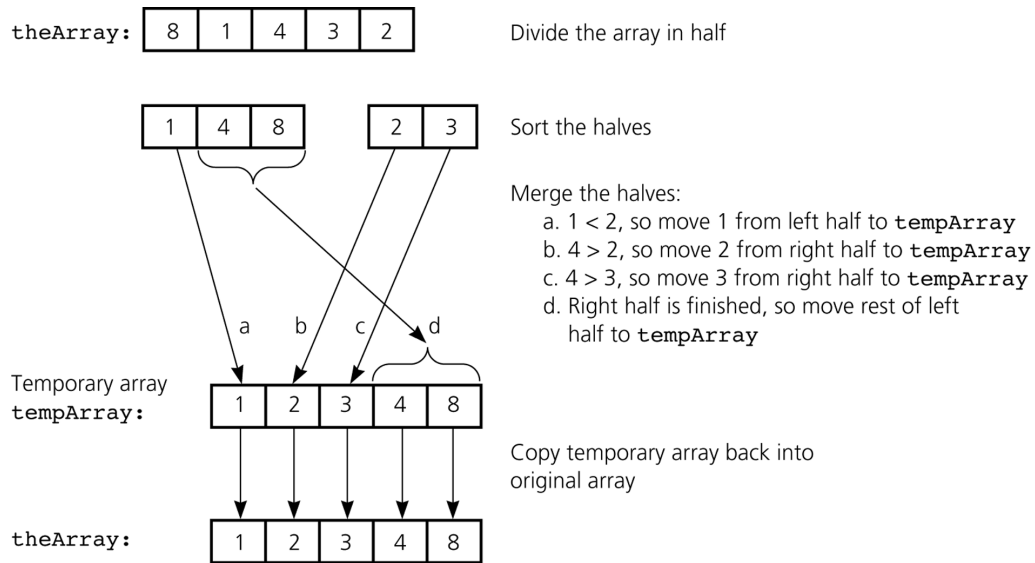
329

Mergesort

- Efficiëntie
 - worst case: $O(n \cdot \log_2 n)$
 - average case: $O(n \cdot \log_2 n)$
- Voordeel
 - mergesort is een uitzonderlijk snel algoritme
- Nadeel
 - mergesort heeft een 2e array nodig die even groot is als de originele

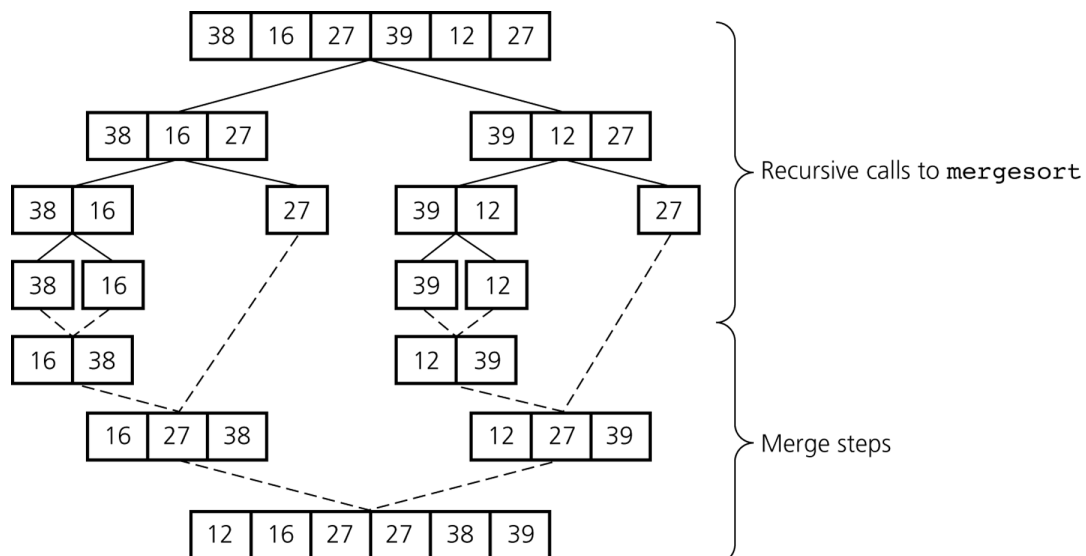
330

Mergesort



331

Mergesort



332

Mergesort pseudocode

mergesort (inout theArray, in first, in last)

```
if (first < last)
{
    mid = (first + last) / 2
    mergesort (theArray, first, mid)
    mergesort (theArray, mid+1, last)
    merge (theArray, first, mid, last)
}
```

333

merge (inout theArray, in first, in mid, in last)

```
first1 = first
last1 = mid
first2 = mid + 1
last2 = last
index = first1           // plaats in tempArray voor het volgende kleinste element
```

```
while ((first1 <= last1) and (first2 <= last2)) do
{
    if theArray[first1] < theArray[first2]
    {
        tempArray[index] = theArray[first1]
        first1 = first1 + 1
        index = index + 1
    }
    else
    {
        tempArray[index] = theArray[first2]
        first2 = first2 + 1
        index = index + 1
    }
}
// ...
```

334

```

while (first1 <= last1) do
{   tempArray[index] = theArray[first1]
    first1 = first1 + 1
    index = index + 1
}
while (first2 <= last2) do
{   tempArray[index] = theArray[first2]
    first2 = first2 + 1
    index = index + 1
}
for index = first to last do
    theArray[index] = tempArray[index]

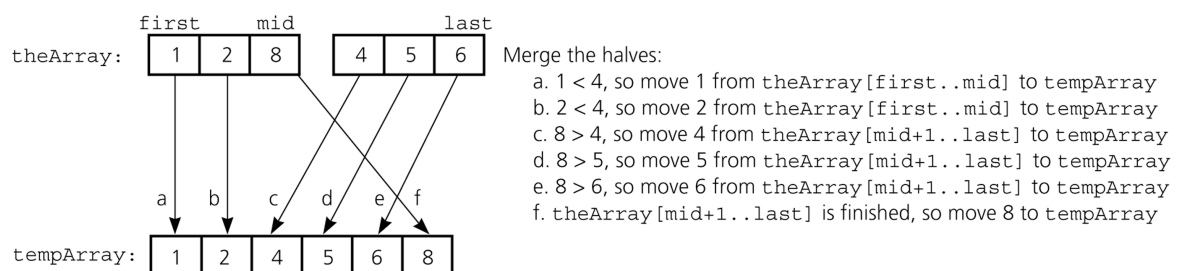
```

335

Mergesort - efficiëntie analyse

Eén merge met n elementen:

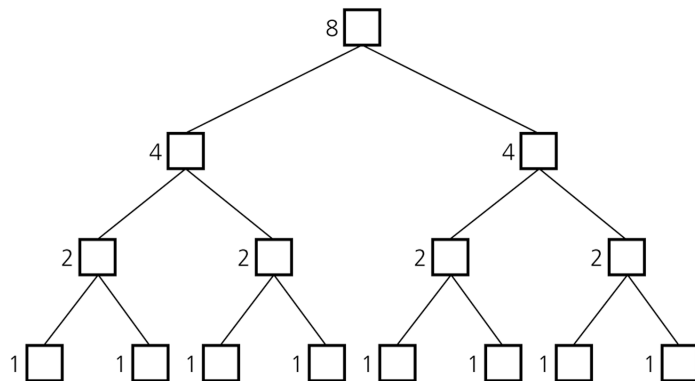
- maximum $n-1$ vergelijkingen
- n verplaatsingen van de originele array naar de hulparray
- n verplaatsingen van de hulparray naar de originele
- **totaal: $3n - 1$ vergelijkingen en verplaatsingen**
- een worst-case voorbeeld van de merge



336

Mergesort - efficiëntie analyse

Voorbeeld: niveaus van recursie voor mergesort met 8 items



Niveau 1: 2 recursieve oproepen naar mergesort met 4 items elk

Niveau 2: 4 recursieve oproepen naar mergesort met 2 items elk

Niveau 3: 8 recursieve oproepen naar mergesort met 1 item elk

Niveau 4: 8 basisgevallen

Algemeen bij n macht van 2 en k niveaus van recursie - boom heeft hoogte $k+1$

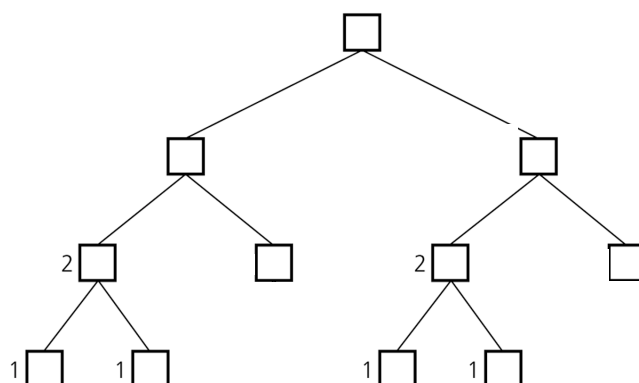
Niveau i : 2^i recursieve oproepen naar mergesort met $n/2^i$ items elk

Niveau $k+1$: $n = 2^k$ basisgevallen

337

Mergesort - efficiëntie analyse

Voorbeeld: niveaus van recursie voor mergesort met 6 items



Niveau 1: 2 recursieve oproepen naar mergesort met 3 items elk

Niveau 2: 4 recursieve oproepen naar mergesort met 1 of 2 items elk

Niveau 3: 4 recursieve oproepen naar mergesort met 1 item elk

Niveau 3&4: 6 basisgevallen – geen recursie

Algemeen bij n geen macht van 2 en k niveaus van recursie - boom heeft hoogte $k+1$

Niveau k : minder dan 2^k recursieve oproepen naar mergesort met 1 item elk

Niveau k & $k+1$: n basisgevallen – recursie stopt

338

Mergesort - efficiëntie analyse

Hoeveel niveaus van recursie zijn er met n elementen (zonder basisgevallen)?

Stel dat er k niveaus van recursie zijn.

1. stel n is een macht van 2, dan $n = 2^k \Leftrightarrow k = \log_2 n$

2. stel n is geen macht van 2,

dan $2^{k-1} < n < 2^k$

$\Leftrightarrow k-1 < \log_2 n < k$

$\Leftrightarrow k < \log_2 n + 1 < k + 1$

$\Leftrightarrow k = \text{het gehele deel van } (\log_2 n + 1) = \lfloor \log_2 n + 1 \rfloor$

339

Mergesort - efficiëntie analyse

Hoeveel significante bewerkingen (SB) per niveau?

Niveau 1: 1 x merge met n elementen
 $\rightarrow 3n - 1$ SB

Niveau 2: 2 x merge met n/2 elementen elk
 $\rightarrow 2(3n/2 - 1) = 3n - 2$ SB

Niveau 3: 4 x merge met n/4 elementen elk
 $\rightarrow 4(3n/4 - 1) = 3n - 4$ SB

Niveau m: 2^{m-1} x merge met $n/2^{m-1}$ elementen elk
 $\rightarrow 2^{m-1}(3n/2^{m-1} - 1) = 3n - 2^{m-1}$ SB

Dus elk niveau is $O(n)$.

Er zijn $\log_2 n$ of $\lfloor \log_2 n + 1 \rfloor$ niveaus.

Conclusie: mergesort is $O(n \cdot \log_2 n)$.

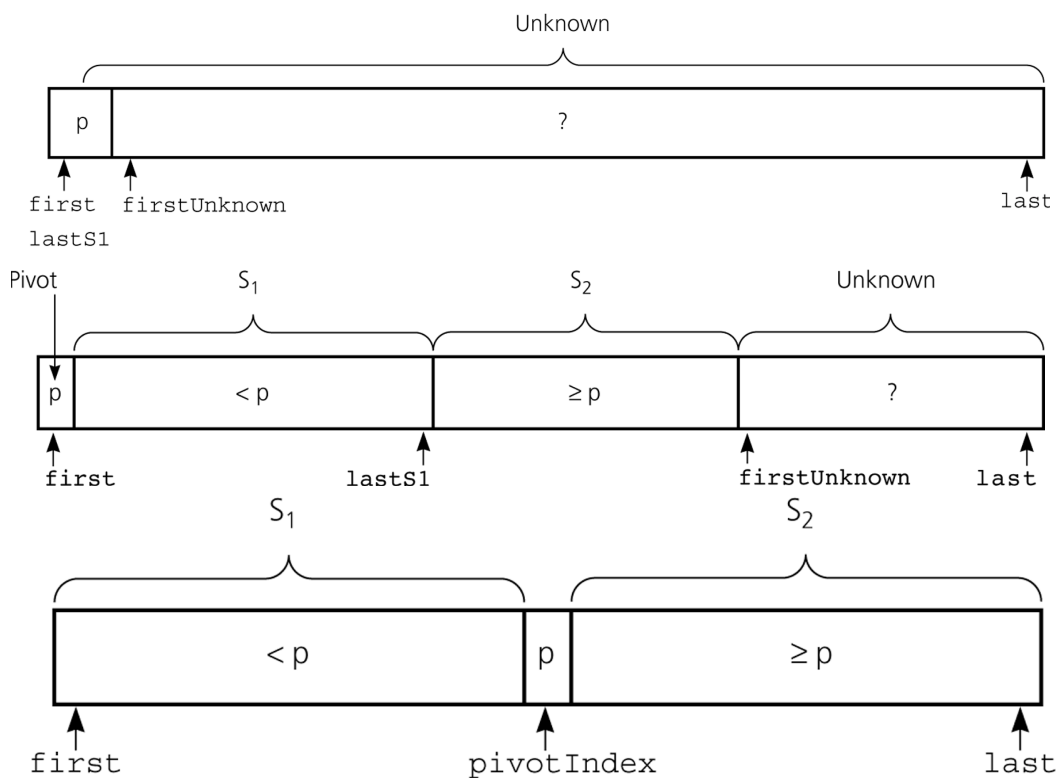
340

Quicksort

- Een recursief sorteeralgoritme – divide and conquer
- Algoritme
 - kies een spilwaarde (pivot)
 - verdeel de array rond de spilwaarde
 - items < spil
 - items $\geq$ spil
 - $\rightarrow$ spil is nu op z'n juiste gesorteerde plaats
 - sorteer het linkerdeel mbv quicksort
 - sorteer het rechterdeel mbv quicksort

341

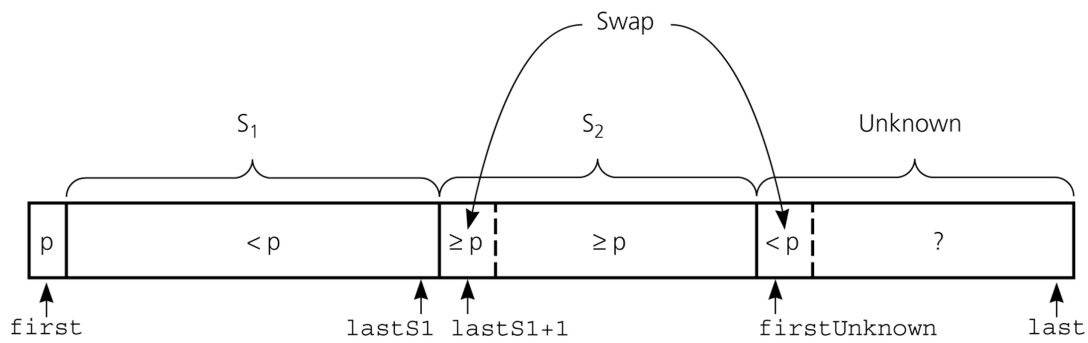
Quicksort



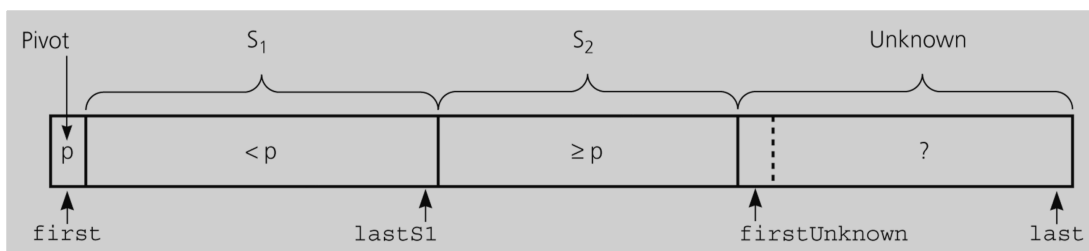
342

Quicksort

Plaats het element op positie `firstUnknown` in S_1 : swap

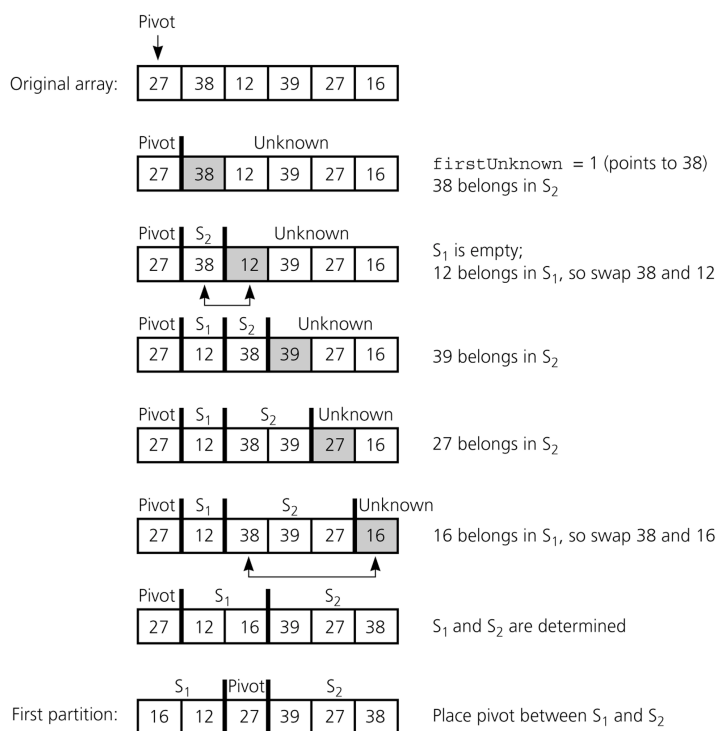


Plaats het element op positie `firstUnknown` in S_2 : `++firstUnknown`



343

Quicksort



344

Quicksort

- Efficiëntie
 - average case: $O(n \cdot \log_2 n)$
 - worst case: $O(n^2)$
 - als de array al gesorteerd is en het kleinste item is de spil

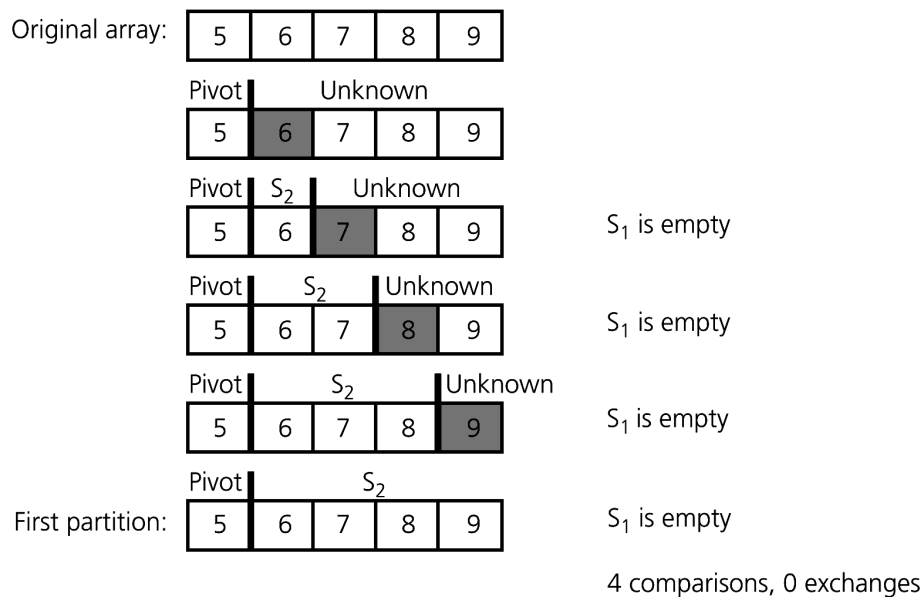
→ quicksort is meestal uitzonderlijk snel

Quicksort – efficiëntie analyse

Dit komt aan bod in het opleidingsonderdeel Algoritmen en complexiteit.

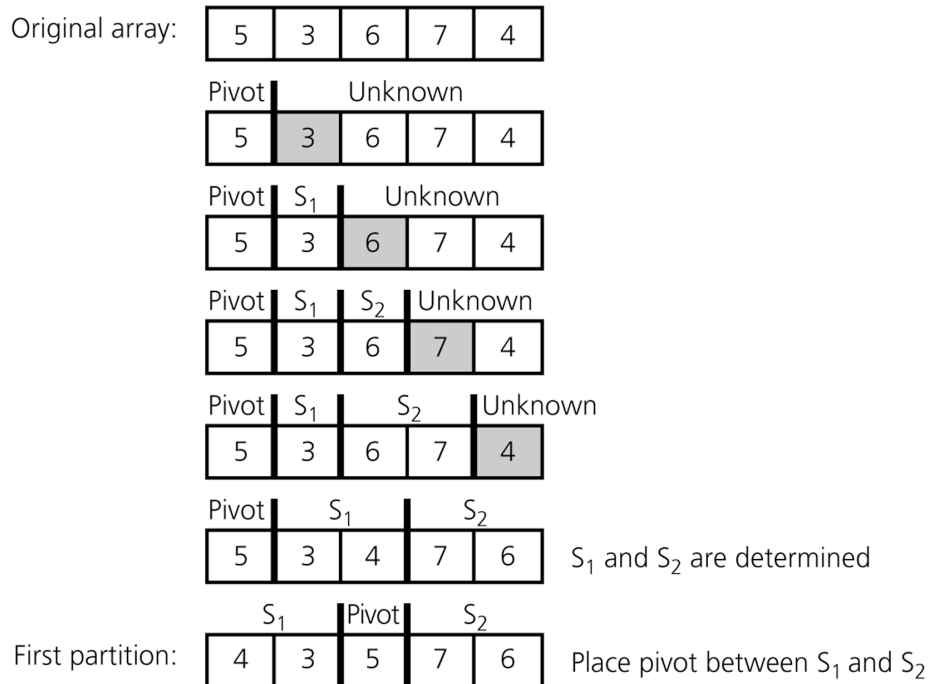
345

Quicksort – worst-case partitioning



346

Quicksort – average-case partitioning



347

quicksort (inout theArray, in first, in last)

```
// sorteert theArray[first .. last]
if (first < last)
{
    partition(theArray, first, last, pivotIndex)
    // sorteer S1
    quicksort(theArray, first, pivotIndex-1)
    // sorteer S2
    quicksort(theArray, pivotIndex+1, last)
}
```

partition (inout theArray, in first, in last, out pivotIndex)

```
// verdeelt theArray in: S1-spil-S2
Kies de spil en swap met theArray[first]
p = theArray[first]           // p is de spil
```

```
lastS1 = first
firstUnknown = first + 1
```

348

```

// verdeel alle items over S1 en S2
while (firstUnknown <= last)
{
    if (theArray[firstUnknown] < p)
    {
        // voeg item toe aan S1
        swap theArray[firstUnknown] with theArray[lastS1+1]
        lastS1 = lastS1 + 1
        firstUnknown = firstUnknown + 1
    }
    else // voeg item toe aan S2
        firstUnknown = firstUnknown + 1
}

// plaats spil tussen S1 en S2
swap theArray[first] met theArray[lastS1]
pivotIndex = lastS1

```

349

Treesort

Sorteer een array van integers in stijgende volgorde, gebruik makend van het ADT binaire zoekboom.

treesort (inout anArray: ArrayType, in n: integer)

// sorteer de n integers in de array anArray in stijgende volgorde

```

{
    insert elk element uit anArray in een binaire zoekboom bTree,
    traverse bTree in inorder en
    kopieer alle bezochte elementen in inorder volgorde in anArray
}

```

- Efficiëntie
 - worst case: $O(n^2)$
 - average case: $O(n \cdot \log_2 n)$

350

Efficiëntie heapsort

Heapsort vs. mergesort

- beiden $O(n \log n)$ in worst en average cases
- maar, heapsort heeft geen 2^e array nodig

Heapsort vs. quicksort

- quicksort is $O(n \log n)$ in average case
- quicksort heeft slechte worst case efficiëntie: $O(n^2)$
- en toch gaat de voorkeur meestal naar quicksort
- zie later in opleidingsonderdeel Algoritmen en complexiteit

351

Overzicht sorteeralgoritmen en hun efficiëntie

Approximate growth rates of time required for eight sorting algorithms

	<u>Worst case</u>	<u>Average case</u>
Selection sort	n^2	n^2
Bubble sort	n^2	n^2
Insertion sort	n^2	n^2
Merge sort	$n \times \log n$	$n \times \log n$
Quick sort	n^2	$n \times \log n$
Radix sort	n	n
Tree sort	n^2	$n \times \log n$
Heap sort	$n \times \log n$	$n \times \log n$

352

Gegevensabstractie en -structuren

Thema 23 Gegevensverwerking in externe opslag

353

Externe methoden

Voordelen van externe opslag

1. opslag: gegevens in een bestand 'overleven' de uitvoering van het programma

vb. de 'save' bewerking van een tekstverwerker

- plaatst het document in een bestand
- laat toe het programma te verlaten en het document later terug in te lezen

2. externe opslag: meestal is er veel meer externe opslag dan intern geheugen beschikbaar

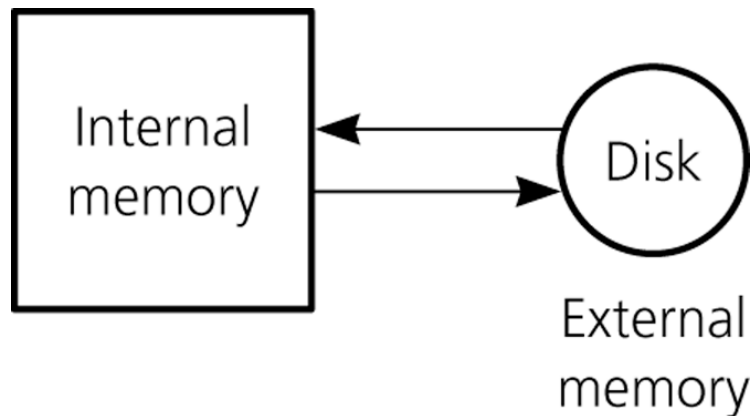
vb. een tabel met 1 miljoen elementen van gemiddelde grootte past waarschijnlijk niet in z'n geheel in het interne geheugen, maar kan zonder probleem op een externe schijf opgeslagen worden

➔ Wanneer we werken met tabellen die te groot zijn voor het interne geheugen hebben we technieken nodig om de extern opgeslagen gegevens te verwerken (vb. zoeken, sorteren) .

354

Stel dat het computergeheugen is opgesplitst in 2 delen:

1. **intern geheugen** – bevat de lopende programma's en non-file gegevens
2. **externe opslag** – bevat de permanente **bestanden**

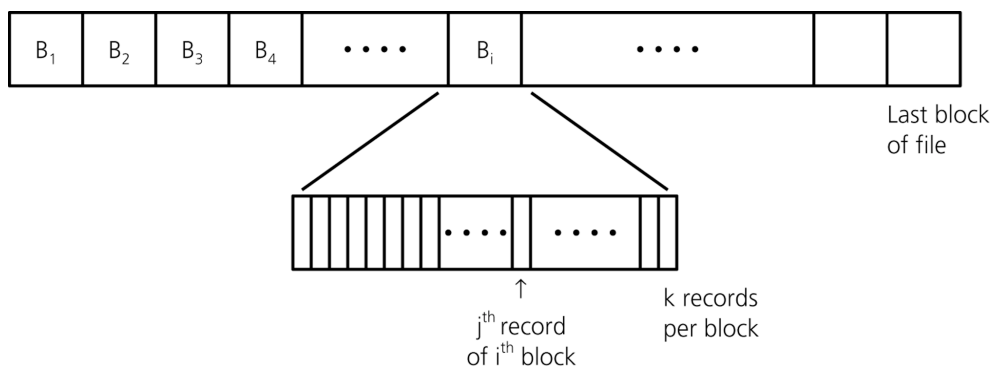


355

Een bestand bevat **datarecords** – we veronderstellen voor de eenvoud dat alle datarecords in eenzelfde bestand van hetzelfde type zijn.

De datarecords in een bestand zijn georganiseerd in **blokken**:

- de **blokgrootte** hangt af van
 - de hardware configuratie en
 - de systeemsoftware van de computer
- het **aantal datarecords per blok** hangt af van de grootte van de records
- een bepaald **blok inlezen** uit het bestand: **input**
- gegevens **wegschrijven** in een bepaald blok: **output**
- input en output via het **bloknummer** (= nummer volgens lineaire volgorde)



356

Alle input uit en output naar een bestand gebeurt op het **niveau van de blokken**

- blokken van records inlezen en wegschrijven
- je kan geen data records inlezen uit een bestand en wegschrijven naar een bestand.

Buffers zijn tijdelijke opslagplaatsen voor gegevens.

Stel dat *buf* een buffer is die alle records uit een blok kan bevatten – de mogelijke lees- en schrijfp opdrachten voor blokken (input/output) zijn dan:

- *buf.readBlock(dataFile, i)* - leest het *i*^{de} blok van het bestand *dataFile* en plaatst het in object *buf*
- *buf.writeBlock(dataFile, i)* – schrijft de inhoud van *buf* in het *i*^{de} blok van het bestand *dataFile*
- *buf.writeBlock(dataFile, n+1)* – voegt een nieuw blok toe aan *dataFile* indien het aantal blokken gelijk is aan *n* → het bestand kan dynamisch groeien

357

Voorbeeld: Werknemer Smith krijgt €1000 opslag. Stel dat Smith's record in blok *i* zit.

Pseudocode:

```
// read block i from file dataFile into buffer buf
buf.readBlock(dataFile, i)
Find the entry buf.getRecord(j) that contains the record whose search key is "Smith"

// increase the salary portion of Smith's record
(buf.getRecord(j)).setSalary((buf.getRecord(j)).getSalary() + 1000)

// write changed block back to file dataFile
buf.writeBlock(dataFile, i)
```

Efficiëntie van algoritmen

Het inlezen en wegschrijven van blokken vereist veel meer tijd dan het verwerken van de gegevens uit de blokken in het interne geheugen.

- beperk het aantal I/O bewerkingen zoveel mogelijk
- de I/O bewerkingen zijn de dominerende factor in het bepalen van de efficiëntie van een algoritme

358

Het sorteren van gegevens in een extern bestand

Beschouw het volgende sorteerbelem:

- een extern bestand F bevat 1600 werknemer records
- sorteer deze records volgens het identificatienummer
- elk blok bevat 100 records; het bestand bevat dus 16 blokken $B_1, B_2, \dots, B_{16}$
- stel dat het programma op elk moment slechts net genoeg interne geheugen kan gebruiken om 300 records (3 blokken) te stockeren

→ het bestand is te groot voor het interne geheugen

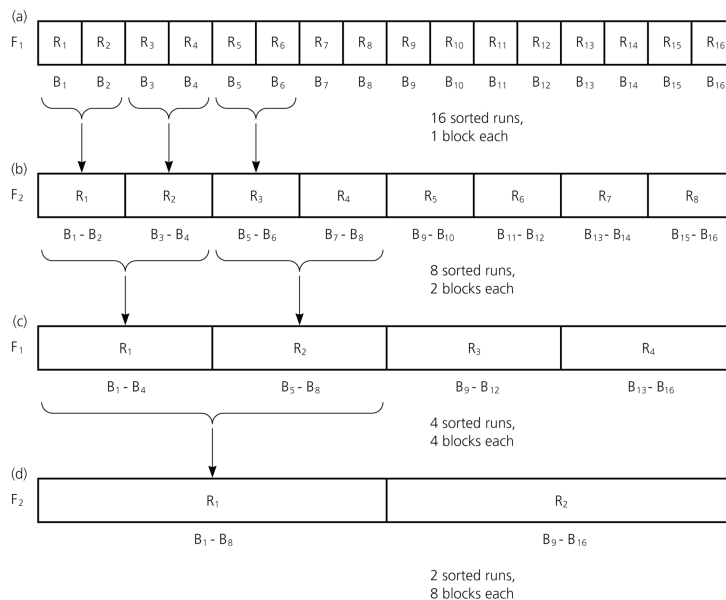
→ **externe mergesort**

Externe mergesort - Fase 1: sorteer elk blok

- lees een blok B uit F in het interne geheugen
- sorteer B's datarecords met een interne sorteermethode
- schrijf het gesorteerde blok naar F_1 (een werkbestand) vooraleer het volgende blok in te lezen
- na het verwerken van alle 16 blokken bevat F_1 16 **gesorteerde runs** $R_1, R_2, \dots, R_{16}$



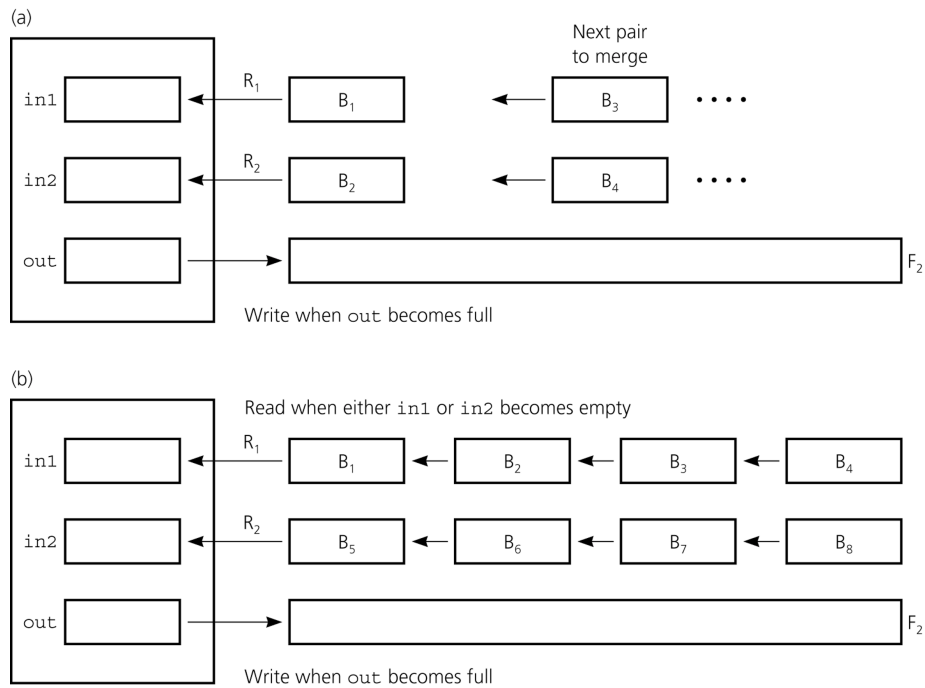
359



Externe mergesort - Fase 2: een opeenvolging van mergestappen

- elke mergestap:
 - voegt paren van gesorteerde runs samen tot grotere gesorteerde runs
 - verdubbelt het aantal blokken per gesorteerde run
 - halveert het aantal gesorteerde runs
 - merges runs stukje per stukje (zie volgende figuur)
- de laatste mergestap voegt 2 gesorteerde runs samen tot een enkele gesorteerde run die alle datarecords bevat.

360



361

```
externalMergesort(in unsortedFile: String, out sortedFile: String)
// sorts a file by using an external mergesort.
// Precondition: unsortedFile is the name of an external file to be sorted.
// SortedFile is the name that the function will give to the resulting sorted file.
// Postcondition: the file named sortedFile is sorted. The original file is
// unchanged. Both files are closed.
// Calls: blockSort, mergeFile, and copyFile.
// Simplifying assumptions: the number of blocks in the unsorted file is an
// exact power of 2.
```

```
inFile := file unsortedFile
outFile := file sortedFile
```

```
// Phase 1: sort file block by block and count the blocks
blockSort(inFile, tempFile1, numberOfBlocks)
```

362

```

// Phase 2: merge runs of size 1, 2, 4, 8, ..., numberOfBlocks/2
// uses two temporary files and a toggle that keeps track of files for each merge step
toggle := 1

for (size := 1 to numberOfBlocks/2 with increments of size)
{   // uses 2 temporary files and a toggle that keeps track of the source file
    // and the destination file

    if (toggle = 1)
        mergeFile(tempFile1, tempFile2, size, numberOfBlocks)
    else
        mergeFile(tempFile2, tempFile1, size, numberOfBlocks)

    toggle := -toggle
}   // end for

// copy the current temporary file to outFile
if (toggle = 1)
    copyFile(temp File1, outFile)
else
    copyFile(tempFile2, outFile)
// end externalMergesort

```

363

blockSort(in inFile: File, out outFile: File, out numberOfBlocks: integer)

```

// Sorts each block of records in a file (d.i. fase 1)
// Precondition: The file variable inFile is associated with the file to be sorted
// Postcondition: The file associated with the file variable outFile contains the
// blocks of inFile. Each block is sorted; numberOfBlocks is the number of
// blocks processed. Both files are closed.
// Calls: readBlock and writeBlock for direct access I/O; sortArray.

prepare inFile for input
prepare outFile for output
numberOfBlocks := 0
while (more blocks in inFile remain to be read)
{   numberOfBlocks := numberOfBlocks + 1
    buffer.readBlock(inFile, numberOfBlocks)
    sortArray(buffer) // sort with some internal sort
    buffer.writeBlock(outFile, numberOfBlocks)
}   // end while
close inFile and outFile
// end blockSort

```

364

mergeFile(in inFile: File, out outFile: File, in runSize: integer,
in numberOfBlocks: integer)

```
// Performs one mergestep of phase 2. Merges pairs of runs from one file to
// another (d.i. in de 'sorted runs figuur' de overgang van (a) naar (b) of
// van (b) naar (c) of van (c) naar (d) of van (d) naar de volledig gesorteerde file)
// Precondition: inFile is an external file that contains numberOfBlocks
// blocks and sorted runs of runSize blocks each.
// Postcondition: outFile contains the merged runs of inFile. Both files are
// closed.
// Calls: mergeRuns
```

```
prepare inFile for input
prepare outFile for output
for (firstBlockOfFirstRun := 1 to numberOfBlocks
    with increments of 2*runSize)
{ // merge de run met als eerste blok firstBlockOfFirstRun met de
  // volgende run (waarvan het eerste blok firstBlockOfFirstRun+runSize is)
  mergeRuns(inFile, outFile, firstBlockOfFirstRun, runSize)
} // end for
close inFile and outFile
// end mergeFile
```

365

mergeRuns(in fromFile: File, out toFile: File, in start: integer, in runSize: integer)

```
// merges two consecutive sorted runs of size runSize in a file – zie vorige figuur
// Precondition: fromFile is an external file of sorted runs of size runSize open
// for input. toFile is an external file of sorted runs open for output.
// the first sorted run to be merged
// - starts at block: start
// - ends at block: start + runSize – 1
// the second sorted run to be merged
// - starts at block: start + runSize
// - ends at block: start + 2*runSize – 1
// Postcondition: the merged runs from fromFile are appended to toFile. The
// files remain open.
```

```
// initialize the input buffers for runs 1 and 2
in1.readBlock(fromFile, first block of Run 1)
in2.readBlock(fromFile, first block of Run 2)
```

366

```

// merge until one of the runs is finished. Whenever an input buffer is
// exhausted, the next block is read. Whenever the output buffer out is full, it
// is written.

while (neither run is finished)
{   place the smaller "leading edge" of in1 and in2 in the next position of out
    if (out is full)
        out.writeBlock(toFile, next block of toFile)
    if (in1 is exhausted and blocks remain in Run 1)
        in1.readBlock(fromFile, next block of Run 1)
    if (in2 is exhausted and blocks remain in Run 2)
        in2.readBlock(fromFile, next block of Run 2)
} // end while

// assertion: exactly one of the runs is finished

```

367

```

// append the remainder of the unfinished input buffer to the output buffer
while (in1 is not exhausted)
    place next item of in1 into the next position of out
while (in2 is not exhausted)
    place next item of in2 into the next position of out

out.writeBlock(toFile, next block of toFile)

// finish off the remaining complete blocks
while (blocks remain in Run 1)
{   in1.readBlock(fromFile, next block of Run 1)
    in1.writeBlock(toFile, next block of toFile)
} // end while
while (blocks remain in Run 2)
{   in2.readBlock(fromFile, next block of Run 2)
    in2.writeBlock(toFile, next block of toFile)
} // end while
// end mergeRuns

```

368

Externe tabellen

Hoe gegevens in externe opslag organiseren zodat de ADT tabel bewerkingen retrieve, insert, delete en traverse efficiënt kunnen worden uitgevoerd?

Volgende onderwerpen komen aan bod:

1. datarecords gesorteerd op zoeksleutel
2. indexeren van externe bestanden
3. externe hashing
4. B-bomen
5. multiple indexing (zie Thema 24)

369

Datarecords gesorteerd op zoeksleutel

- één van de eenvoudigste implementaties van externe tabellen
- de datarecords zijn opgeslagen in de blokken in de volgorde van de zoeksleutels, bv. door gebruik te maken van de externe mergesort.

Traverse is heel eenvoudig

```
traverseTable( in dataFile: File, in numberOfBlocks: integer,  
               in recordsPerBlock: integer, in visit: function)  
// Traverses the sorted file dataFile in sorted order, calling the function visit  
// once for each item  
  
// read each block of file dataFile into an internal buffer buf  
for (blockNumber := 1 to numberOfBlocks)  
{   buf.readBlock(dataFile, blockNumber)  
  
    // visit each record in the block  
    for (recordNumber := 1 to recordsPerBlock)  
        visit(buf.getRecord(recordNumber – 1))  
} // end for
```

370

Retrieve kan gebruik maken van binair zoeken

```
tableRetrieve(  in dataFile: File, in recordsPerBlock: integer, in first: integer,
                in last: integer, in searchKey: KeyItemType,
                out tableItem: TableItemType): boolean
// searches blocks first through last of file dataFile, copies into tableItem the
// record whose search key is searchKey, and returns true. The function returns
// false if no such item exists.

if (first > last or nothing is left to read from dataFile)
    return false
else
{   // read the middle block of file dataFile into array buf
    mid := (first + last)/2
    buf.readBlock(dataFile, mid)
```

371

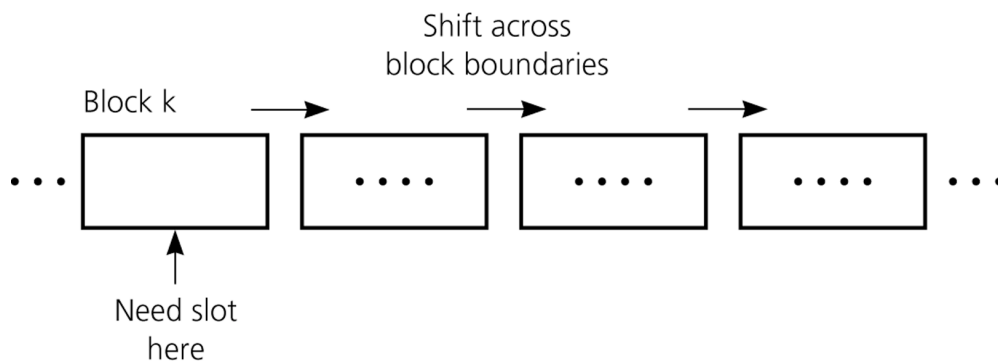
```
if ((searchKey >= (buf.getRecord(0)).getKey()) and
    searchKey <= (buf.getRecord(recordsPerBlock-1)).getKey()))
{   // desired block is found
    search buffer buf for record buf.getRecord(j) whose search key equals
    searchKey
    if (record is found)
    {   tableItem := buf.getRecord(j)
        return true
    }
    else
        return false
} // end if

// else search appropriate half of the file
else if (searchKey < (buf.getRecord(0)).getKey())
    return tableRetrieve(dataFile, recordsPerBlock, first, mid-1, searchKey,
                        tableItem)
else
    return tableRetrieve(dataFile, recordsPerBlock, mid+1, last, searchKey,
                        tableItem)

} // end if
```

372

Insert en delete kunnen vele dure I/O bewerkingen tot gevolg hebben om datarecords te shiften

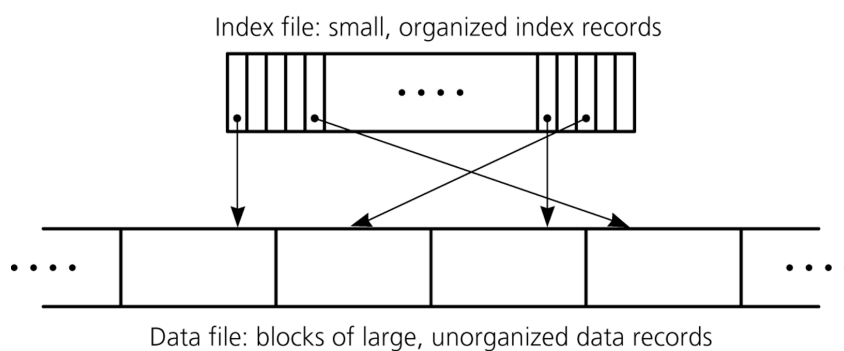


→ deze techniek (datarecords opslaan volgens zoek sleutel) is enkel nuttig voor externe tabellen waarin zelden elementen worden toegevoegd en waaruit zelden elementen worden verwijderd.

373

Indexeren van externe bestanden

Voeg een **indexbestand** toe als index voor het databestand en sorteer het indexbestand ipv het databestand.



374

→ het **databestand**

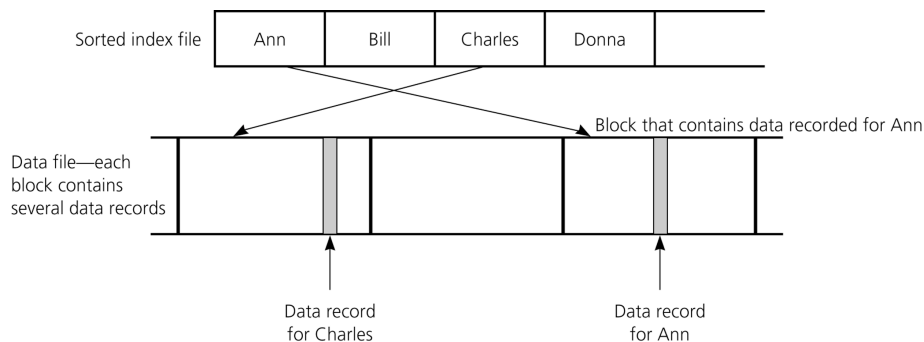
- bevat de **datarecords**
- is niet gesorteerd

→ het **indexbestand**

- bevat één **indexrecord** per datarecord in het databestand
- is gesorteerd volgens de zoek sleutels

Elke **indexrecord** bevat:

- een **sleutel**, de zoek sleutel van het overeenkomstige datarecord
- een **datapointer**, het bloknummer van het blok in het databestand dat het overeenkomstige datarecord bevat



375

Voordelen van een indexbestand

- een indexrecord is veel kleiner dan een datarecord
 - het indexbestand is veel kleiner dan het databestand
 - veel minder I/O nodig
- datarecords
 - zijn niet gesorteerd en
 - hoeven dus niet geshift te worden bij
 - delete: locatie in freelist plaatsen
 - insert: element in lege locatie volgens freelist of achteraan toevoegen
 - enkel indexrecords worden geshift
- je kan verschillende indexbestanden gebruiken op hetzelfde databestand (d.i. multiple indexing, zie Thema 24)

376

Als het indexbestand de indexrecords sequentieel volgens sleutel bevat, kan retrieve binair zoeken op het indexbestand:

```
tableRetrieve( in tIndex: File, in tData: File, in searchKey: KeyItemType,  
               out tableItem: TableItemType): Boolean
```

```
// Retrieves into tableItem the record whose search key equals searchKey,  
// where tIndex is the index file and tData is the data file. The operation  
// returns true if the record exists, false otherwise.
```

```
if (no blocks are left in tIndex to read)  
    return false
```

```
else
```

```
{ // read the middle block of index file into object buf  
  mid := number of middle block of index file tIndex  
  buf.readBlock(tIndex, mid)
```

377

```
if ((searchKey >= (buf.getRecord(0)).getKey()) and  
    (searchKey <= (buf.getRecord(indexrecordsPerBlock-1)).getKey()))  
{ // desired block of index file is found  
  search buf for index record buf.getRecord(j) whose key = searchKey  
  if (index record buf.getRecord(j) is found)  
  {   blockNum := buf.getRecord(j).datapointer  
      data.readBlock(tData, blockNum)  
      search data for data record data.getRecord(k) whose  
          key = searchKey  
      tableItem := data.getRecord(k)  
      return true  
  }  
  else  
      return false  
}  
else if (tIndex is one block in size)  
    return false // no more blocks in file  
else if (searchKey < (buf.getRecord(0)).getKey())  
    return tableRetrieve(first half of tIndex, tData, searchKey, tableItem)  
else  
    return tableRetrieve(2nd half of tIndex, tData, searchKey, tableItem)  
} // end if
```

378

Een niet-gesorteerd databestand met een *sequentieel* gesorteerd indexbestand is

- efficiënter dan een gesorteerd databestand maar
- verre van bevredigend in vele toepassingen

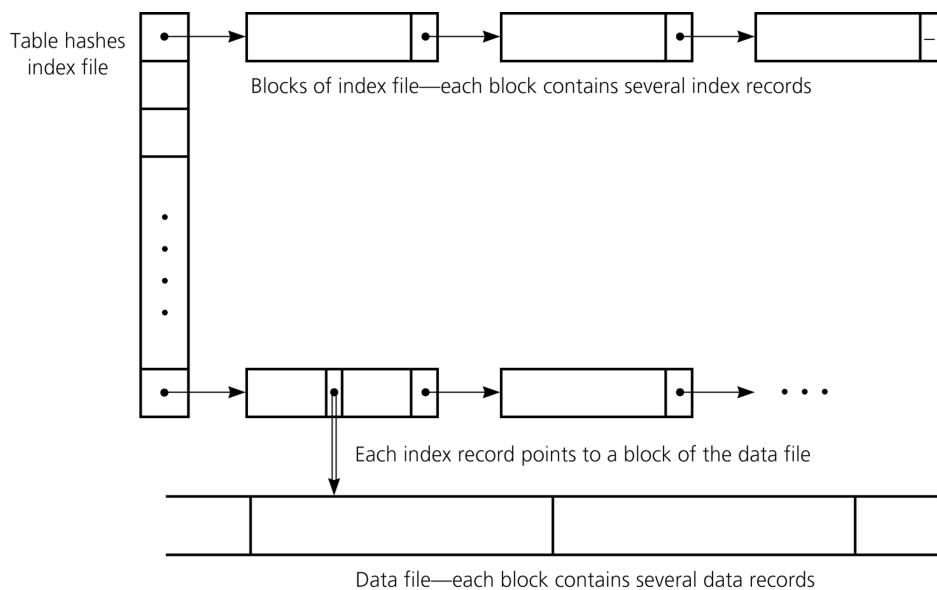
Er zijn veel betere implementaties mogelijk gebruik makend van

- hashing
- zoekbomen.

379

Externe hashing

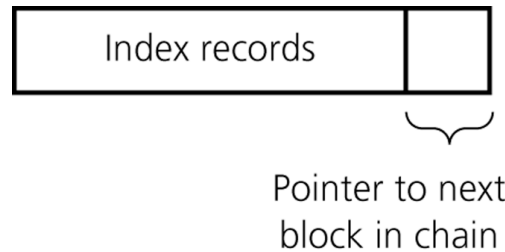
Hash het indexbestand ipv het databestand:



Elk blok van de gelinkte ketting in `hashTable[i]` bevat indexrecords waarvan de sleutel (en dus ook de zoek sleutel van het overeenkomstig data record) gehashed wordt op `i`.

380

Om de gelinkte ketting te vormen bevat elk blok in het indexbestand bovendien een blokpointer, d.i. het bloknummer van het volgende blok in de ketting of –1 als null pointer:



381

TableRetrieve(in tIndex: File, in tData: File, in searchKey: KeyItemtype,
out tableItem: TableItemtype): Boolean

// Retrieves into tableItem the item whose search key equals searchKey, where
// tIndex is the index file, which is hashed, and tData is the data file. The
// function returns true if the record exists, false otherwise.

```

// apply the hash function to the search key
i := hash(searchKey)
// find the first block in the chain of index blocks – these blocks contain index
// records that hash into location i
p := hashTable[i]
// if p = -1, no values have hashed into location i
if (p <> -1)
    buf.readBlock(tIndex, p)
// search for the block with the desired index record
while (p <> -1 and buf does not contain an index record with key searchKey)
{
    p := number of next block in chain
    // if p = -1, you are at the last block in the chain
    if (p <> -1)
        buf.readBlock(tIndex, p)
} // end while
  
```

382

```

// retrieve the data item if present
if (p <> -1)
{
    // let buf.getRecord(j) be the index record with key searchKey
    blockNum := number of the data-file block to which buf.getRecord(j) points
    data.readBlock(tData, blockNum)
    find data record data.getRecord(k) whose search key is searchKey
    tableItem := data.getRecord(k)
    return true
}
else
    return false

```

383

Externe hashing, insert

Stel dat we een nieuw datarecord met zoek sleutel searchKey wensen toe te voegen:

- voeg het datarecord toe aan het (niet-gesorteerde) databestand
 - als er ergens midden in het databestand een lege plek is als gevolg van een eerdere delete bewerking kan het datarecord daarin geplaatst worden
 - anders wordt het nieuwe datarecord achteraan in het laatste blok toegevoegd, of indien nodig als eerste datarecord in een nieuw blok dat achteraan het databestand wordt toegevoegd
- voeg een corresponderend indexrecord toe aan het indexbestand
 - stel dat p het bloknummer is van het blok in het databestand dat het nieuwe datarecord bevat
 - voeg aan het indexbestand een nieuw indexrecord <searchKey, p> toe in hashTable[hash(searchKey)], een gelinkte ketting van blokken (in een lege plek of achteraan)

384

Externe hashing, delete

Stel dat we het datarecord met zoek sleutel searchKey wensen te verwijderen:

- verwijder het corresponderende indexrecord uit het indexbestand
 - zoek in de gelinkte ketting van blokken waarnaar hashTable[hash(searchKey)] verwijst, een indexrecord met sleutel searchKey
 - als er zo geen indexrecord is dan bevat het databestand ook geen datarecord met zoek sleutel searchKey
 - als het indexrecord <searchKey, p> gevonden is, onthoud dan het bloknummer p en verwijder dit indexrecord uit het indexbestand
- verwijder het datarecord uit het databestand
 - verwijder het datarecord met searchKey uit het blok met bloknummer p (door de locatie in een freelist te plaatsen)

Het aantal blokaccessen (I/O bewerkingen) is zeer laag voor de retrieve, insert en delete bewerkingen:

- hoogstens 1 blok uit het databestand
- hoogstens alle blokken uit 1 gelinkte hashketting van het indexbestand

385

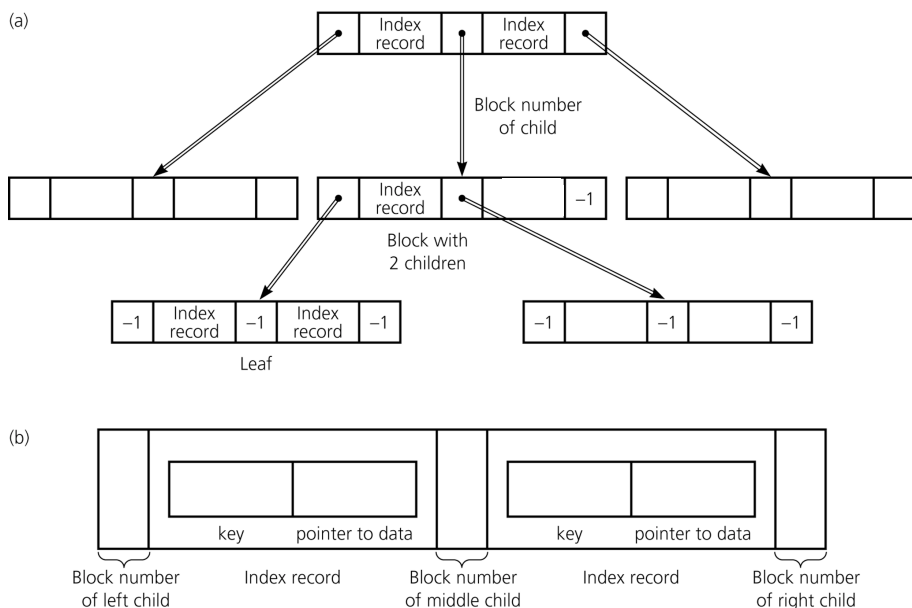
Externe hashing is

- zeer geschikt voor insert, delete en retrieve bewerkingen op grote externe tabellen.
- niet geschikt voor bewerkingen zoals
 - gesorteerd doorlopen (traversal),
 - het kleinste/grootste element zoeken,
 - ...→ gebruik in dit geval een externe zoekboom implementatie - B-bomen - ipv externe hashing

386

B-bomen

Organiseer het indexbestand (ipv het databestand) als een externe zoekboom



387

Figuur (a) is een B-boom van graad 3:

- de blokken van het indexbestand zijn georganiseerd in een 2-3 zoekboom
- de pointers naar de kinderen zijn bloknummers
- elk blok in het indexbestand is een knoop in de boom en bevat 3 pointers naar kinderen (-1 is de null pointer)
- elke knoop (blok van het indexbestand) kan 2 indexrecords van de vorm <sleutel, datapointer> bevatten en 3 kinderpointers

```
tableRetrieve( in tIndex: File, in tData: File, in rootNum: integer,
    in searchKey: KeyItem, out tableItem: TableItem): boolean
// Retrieves into tableItem the record whose search key equals searchKey.
// tIndex is the index file, which is organised as a 2-3 tree. rootNum is the
// block number (of the index file) that contains the root of the tree. tData is
// the data file. The function returns true if the record exists, false otherwise.
```

```
// read from index file into internal buf the block that contains the root of the 2-3 tree
buf.readBlock(tIndex, rootNum)
```

388

```

// search buf for the index record whose key equals searchKey
if (searchKey is in buf)
{
    blockNum := data pointer in the index record with key searchKey
    data.readBlock(tData, blockNum)
    find data record data.getRecord(k) whose search key equals searchKey
    tableItem := data.getRecord(k)
    return true
}

else if (the root is a leaf)
    return false

// else search the appropriate subtree
else
{
    child := block number of root of appropriate subtree
    return tableRetrieve(tIndex, tData, child, searchKey, tableItem)
} // end if

```

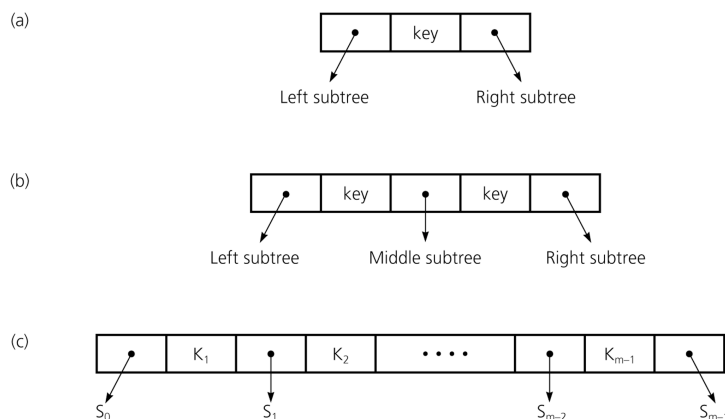
389

Definitie algemene zoekboom

Als een knoop N in een zoekboom **m kinderen** heeft, dan moet N **m-1 zoek sleutelwaarden** bevatten die de waarden in de deelbomen correct scheiden:

Stel N heeft deelbomen $S_0, S_1, \dots, S_{m-1}$ en sleutelwaarden $K_1, K_2, \dots, K_{m-1}$ met $K_1 < K_2 < \dots < K_{m-1}$. Dan geldt:

- alle waarden in S_0 zijn kleiner dan K_1
- voor alle i tussen 1 en $m-2$: alle waarden in S_i zitten tussen K_i en K_{i+1}
- alle waarden in S_{m-1} zijn groter dan K_{m-1}



390

Indien elke knoop uit een boom aan deze eigenschap voldoet, kan het volgende veralgemeend zoekalgoritme gebruikt worden.

```
tableRetrieve( in tIndex: File, in tData: File, in rootNum: integer,  
               in searchKey: KeyItemtype, out tableItem: TableItemtype): boolean  
// Retrieves into tableItem the record whose search key equals searchKey.  
// tIndex is the index file, which is organised as a search tree. rootNum is the  
// block number (of the index file) that contains the root of the tree. tData is  
// the data file. The function returns true if the record exists, false otherwise.  
  
// read from index file into internal array buf the block that contains the root of the tree  
buf.readBlock(tIndex, rootNum)  
  
// search buf for the index record whose key value equals searchKey  
if (searchKey is one of the  $K_i$  in the root)  
{   blockNum := data pointer in the index record with key searchKey  
    data.readBlock(tData, blockNum)  
    find data record data.getRecord(k) whose search key equals searchKey  
    tableItem := data.getRecord(k)  
    return true  
}
```

391

```
else if (the root is a leaf)  
    return false  
  
// else search the appropriate subtree  
else  
{   determine which subtree  $S_i$  to search  
    child := block number of the root of  $S_i$   
    return tableRetrieve(tIndex, tData, child, searchKey, tableItem)  
} // end if
```

392

Wat is het ideale maximum aantal kinderen m in een zoekboom?

Het beste resultaat wordt bekomen als:

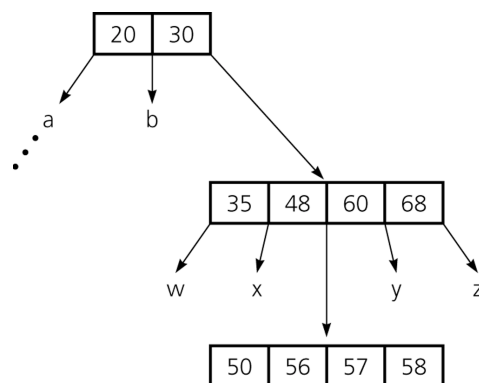
m de grootst mogelijke oneven integer is zodat m kinderpointers en m-1 indexrecords <sleutel, datapointer> in één enkel blok van het indexbestand passen. (Als m oneven is zijn de algoritmen eenvoudiger.)

Definitie. Een **B-boom van graad m** is een algemene zoekboom met volgende eigenschappen:

- alle bladeren zitten op hetzelfde niveau
- elke knoop bevat tussen $m-1$ en $\lfloor m/2 \rfloor$ indexrecords
- elke interne knoop bevat 1 kind meer dan het indexrecords bevat
- de wortel mag ook minder records/kinderen bevatten: minimum 1 indexrecord en 2 kinderen

393

Een B-boom van graad 5:

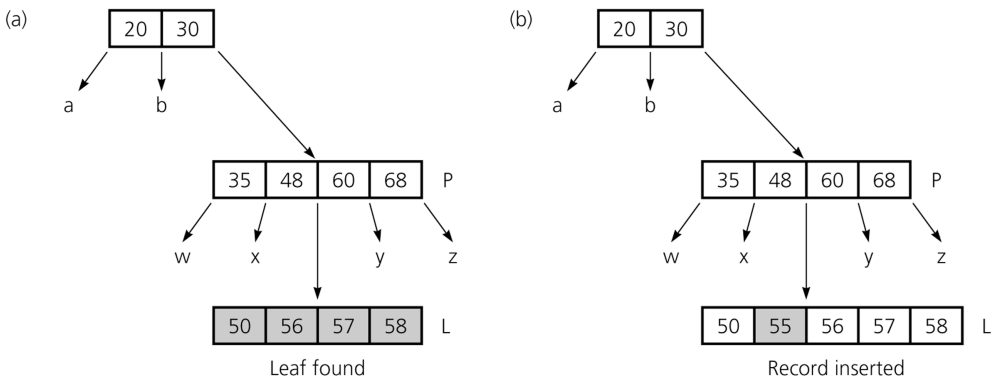


Een B-boom van graad 3 komt overeen met een 2-3 boom.

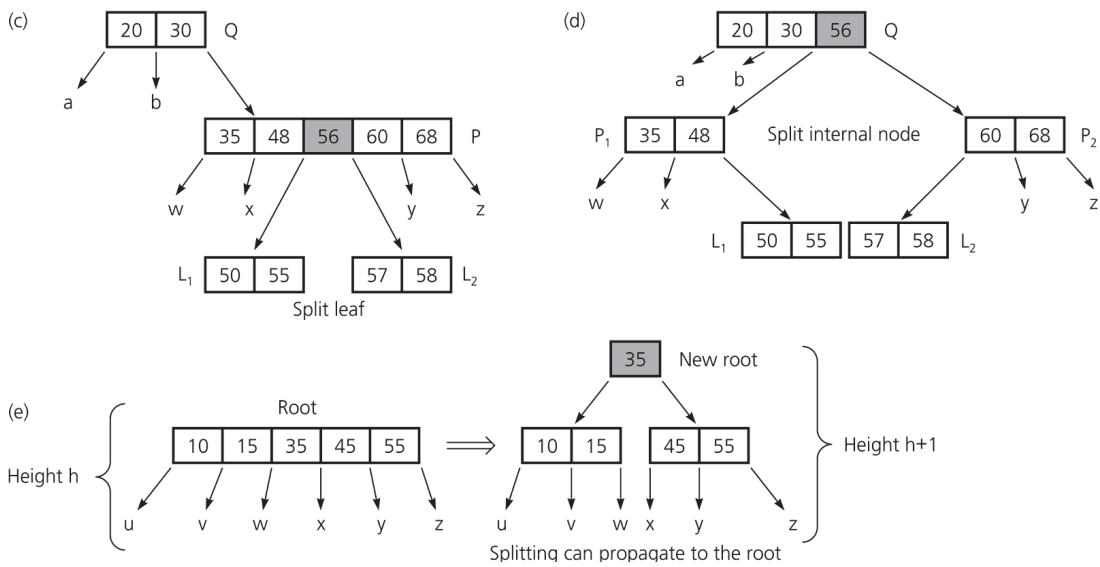
De insert en delete algoritmen voor B-bomen zijn veralgemeningen van de algoritmen voor 2-3 bomen en maken op analoge manier gebruik van splitsen resp. samenvoegen.

394

Insert 55



395

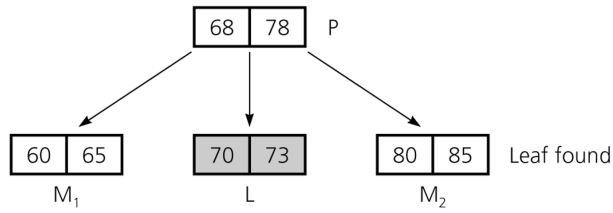


De hoogte neemt enkel toe bij het splitsen van de wortel.

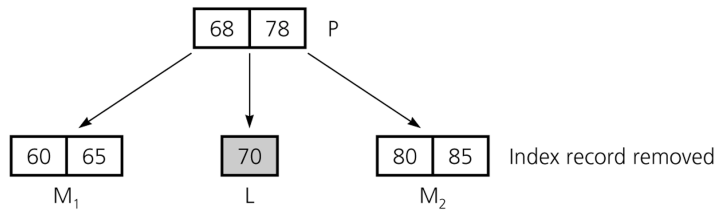
396

Delete 73 uit een B-boom van graad 5 (minstens 2 items per knoop)

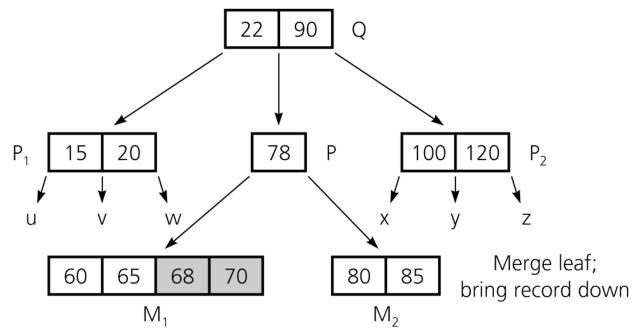
(a)



(b)

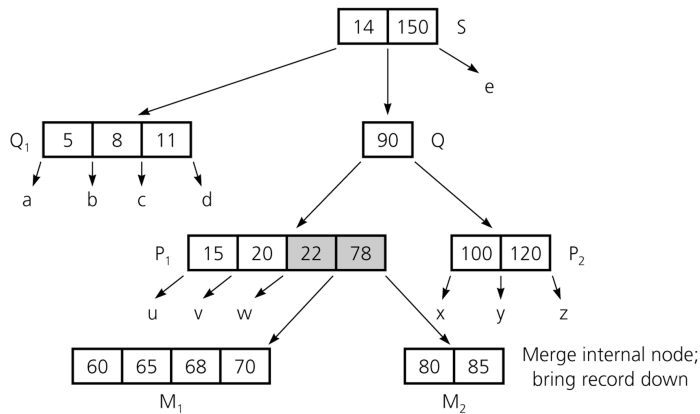


(c)

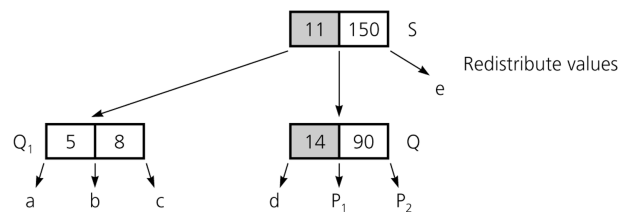


397

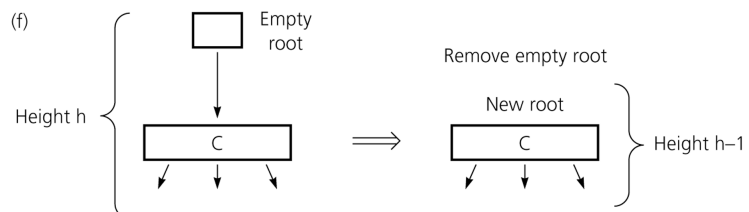
(d)



(e)



(f)



398

Het doorlopen in volgorde – traversals

Inorder traversal van een B-boom indexbestand

Dit gebeurt met een minimum aantal I/O bewerkingen want elk blok van het indexbestand wordt slechts éénmaal ingelezen (het databestand hoeft niet gebruikt te worden):

Traverse(in blockNum: integer, in m: integer)

```
// Traverses in sorted order an index file that is organized as a B-tree of
// degree m. blockNum is the block number of the root of the B-tree in the
// index file.
```

```
if (blockNum <> -1)
{   // read the root into internal array buf
    buf.readBlock(indexFile, blockNum)
    // traverse the children
```

399

```
    // traverse  $S_0$ 
    p := the block number of the 0th child of buf
    traverse(p, m)
    for (i:=1 to m-1)
    {   display key  $K_i$  of buf
        // traverse  $S_i$ 
        p := the block number of the ith child of buf
        traverse(p, m)
    } // end for
} // end if
```

Dit algoritme veronderstelt dat er voldoende intern geheugen is om een recursieve stack van h blokken - met h de hoogte van de boom - in te lezen. In veel toepassingen is dit redelijk, zo niet kan een ander algoritme gebruikt worden.

400

Inorder traversal van een databestand geïndexeerd met een B-boom

Indien de volledige datarecords dienen getoond te worden (en niet enkel de zoek sleutel) is de B-boom implementatie niet efficiënt omdat dan telkens het overeenkomstige blok van het databestand dient ingelezen te worden:

Volgende lijn uit het vorige algoritme

display key K_i of buf

wordt dan

$p :=$ the datapointer in the i^{th} index record of buf

data.readBlock(dataFile, p)

extract from data the data record whose search key equals K_i

display the data record

→ het aantal blokaccessen (I/O bewerkingen) in het databestand is gelijk aan het aantal datarecords! Dit is meestal onaanvaardbaar – voor dit soort toepassingen is het beter het databestand zelf gesorteerd te houden.

401

Gegevensabstractie en -structuren

Thema 24 Gegevensstructuren combineren

402

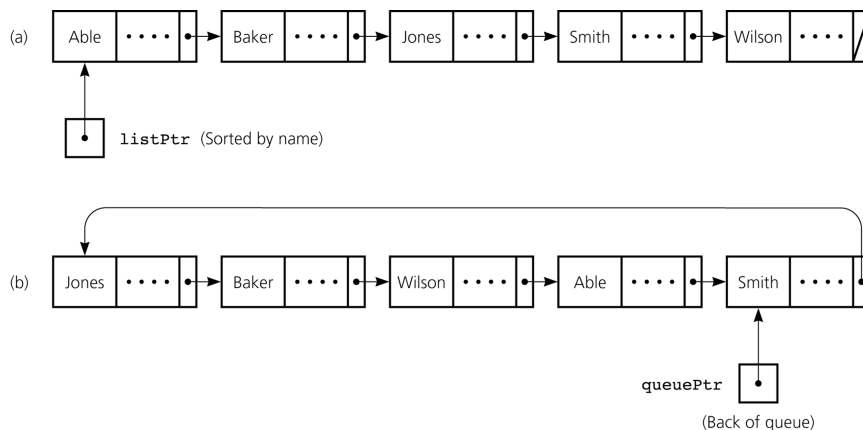
Gegevens met verschillende organisaties

Veel toepassingen combineren verschillende vormen van gegevensbeheer op dezelfde verzameling gegevens.

Voorbeeld: (a) het regelmatig afdrukken van een klantenlijst gesorteerd op naam
(b) het bijhouden van een wachtlijst met klanten

→ gebruik 2 **onafhankelijke gegevensstructuren**,

- voor (a) een gesorteerde gelinkte ketting en
- voor (b) een queue.

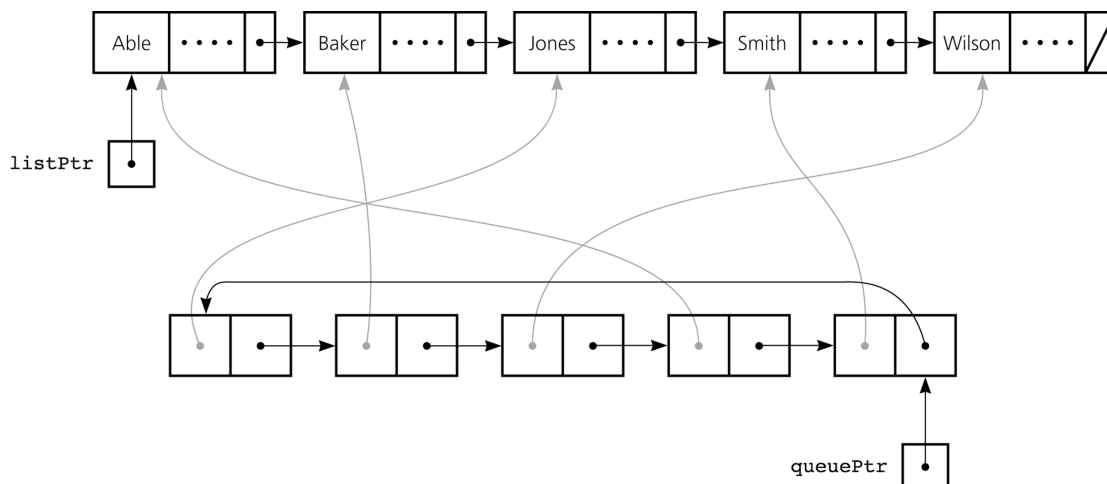


403

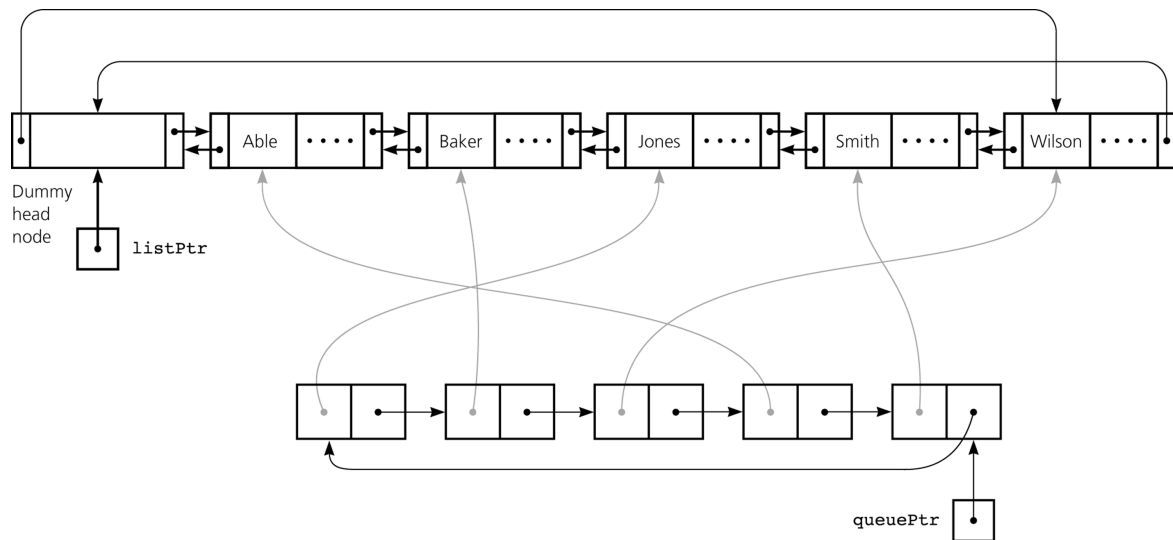
Maar: verschillende onafhankelijke gegevensstructuren

- verspillen geheugen en
- ondersteunen niet alle bewerkingen efficiënt

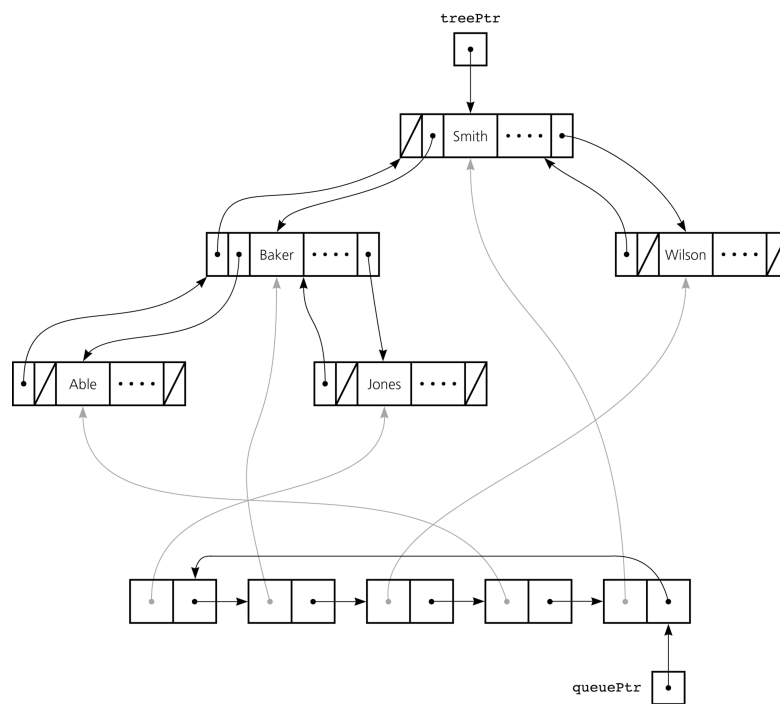
→ beter **onderling afhankelijke gegevensstructuren** ter ondersteuning van verschillende soorten gegevensbeheer, zoals in volgende figuren:



404



405



406

Multiple indexing en externe opslag

Voorbeeld

Een databestand bevat een verzameling werknemersrecords waarop volgende retrievals nodig zijn:

RetrieveN(in aName: NameType): ItemType

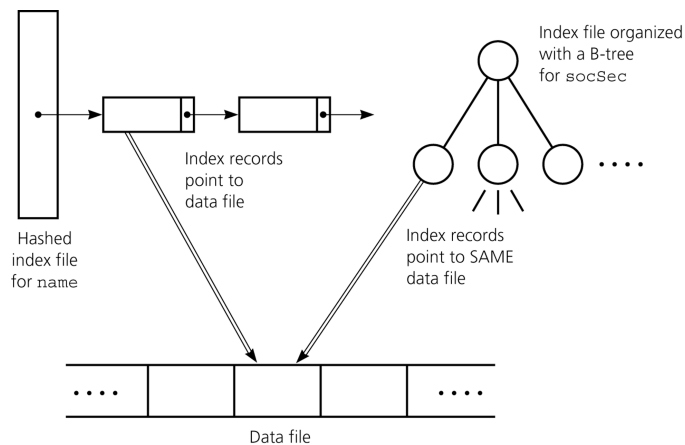
// Retrieves the item whose search key contains the name aName

RetrieveS(in ssn: SSNType): ItemType

// Retrieves the item whose search key contains the social security number ssn

Een mogelijke oplossing houdt 2 onafhankelijke indexbestanden bij, bv. zoals in volgende figuur:

407



Merk op: retrieve gebruikt telkens slechts 1 indexbestand maar insert en delete bewerken steeds beide indexbestanden

408